example-network-topology — full documentation

Example Network Topology

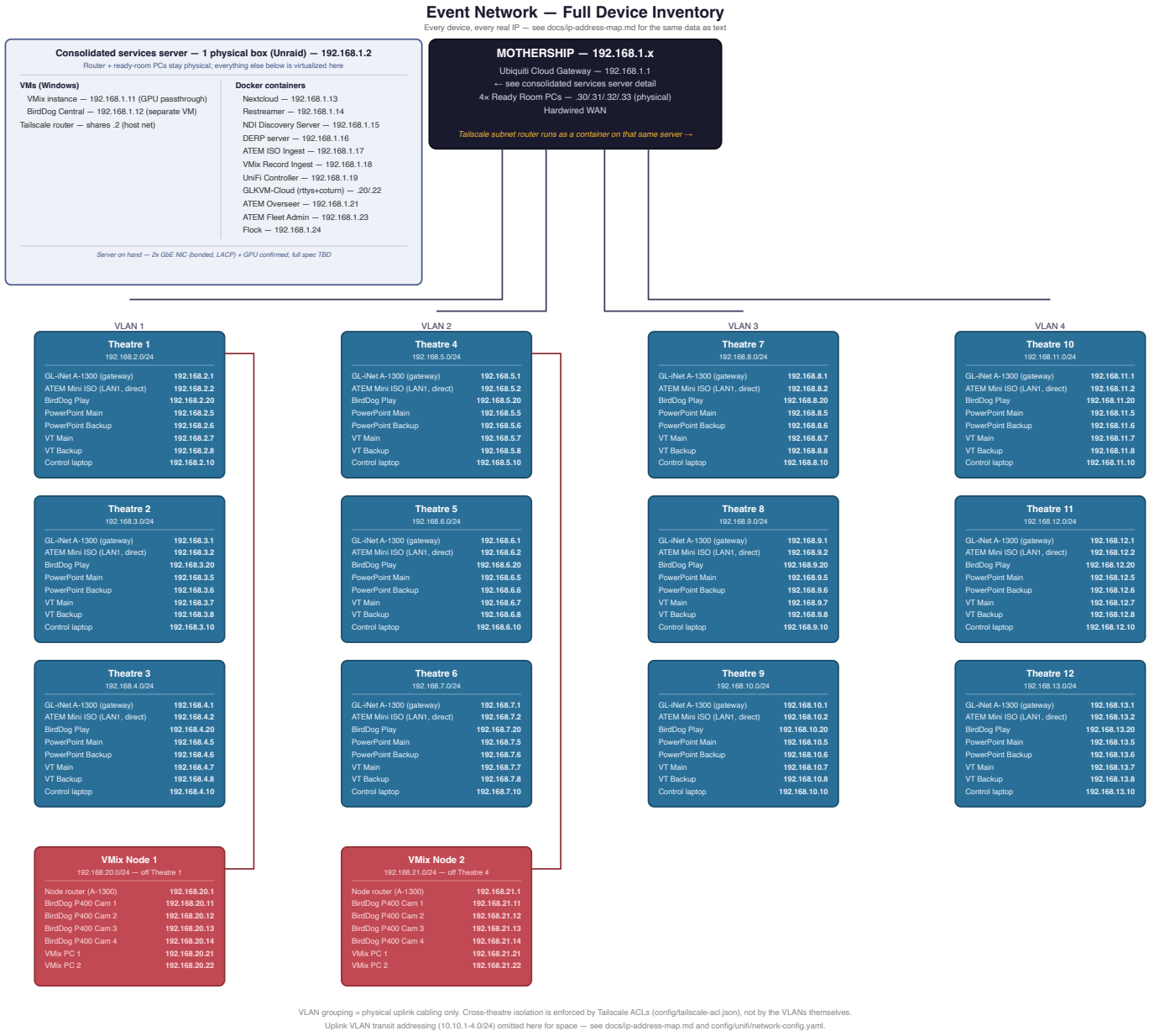
A complete, fully-worked network design for a 12-theatre live event: every theatre streams its program feed and syncs its files back to a central "mothership," all riding over a single Tailscale mesh, with no reliance on venue Wi-Fi or a conference-provided network.

This is an anonymized reference design, published as a worked example of the whole process — topology, bandwidth modelling, hardware selection with rationale, generated configs, deployment runbook. It's a real design for a real event, with the identifying details generalized: all domains are placeholders on the IANA-reserved `example.net` (substitute your own), and nothing here names the event, client, or venue. Product names, prices, and throughput figures are real and cited.

Status: design finalized, nothing built yet. Every hardware decision is locked in, every device has a real IP, throughput has been checked against real-world figures and the venue's actual VLAN constraints, and Tailscale/GL-iNet/UniFi configs are generated and ready to apply — see `docs/open-questions.md` for the handful of things that still need real-world input before this goes from design to build, or `docs/deployment-runbook.md` for the full ordered build sequence.

The shape of it

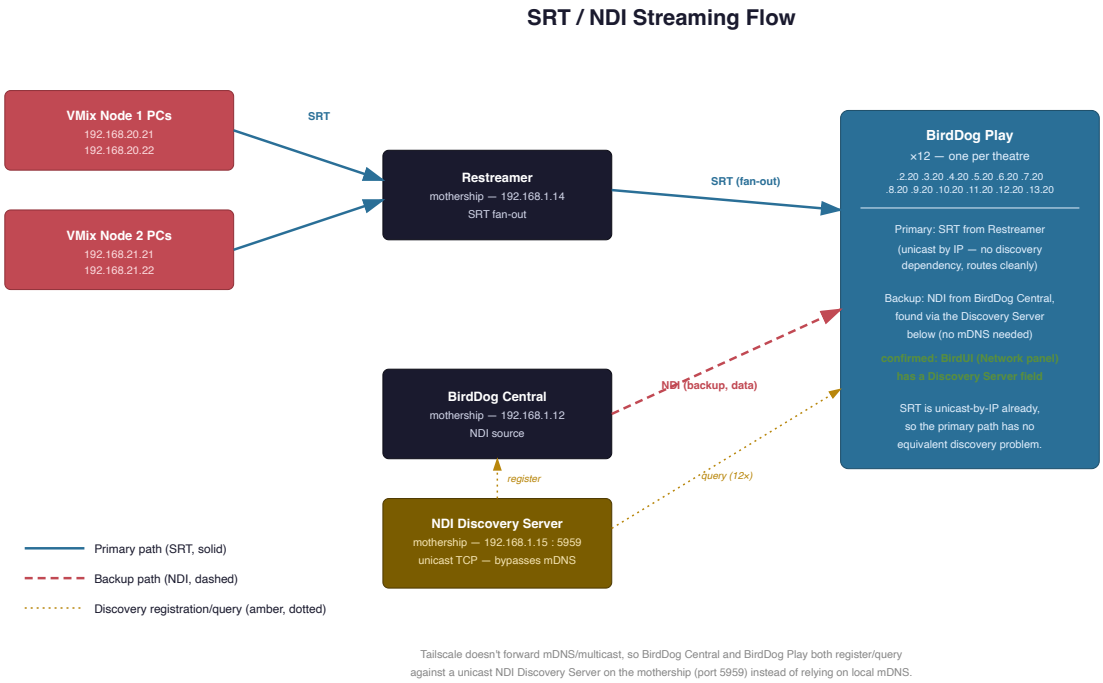
12 theatres, each its own isolated subnet, connect back to one mothership over Tailscale. Physical uplink cabling is grouped 3-theatres-per-VLAN purely for cable management — the VLANs don't provide isolation themselves, **Tailscale ACLs do**: every theatre can reach the mothership, but never another theatre, and never a VMix node. Two extra "VMix nodes" hang off Theatre 1 and Theatre 4 respectively, each with its own cameras, its own router, and its own place on the tailnet.



Every box in that diagram is a real device with a real address — 134 of them in total, counting the live-editing suite's own four. `docs/ip-address-map.md` has the same inventory as a plain text table, if that's easier to search or paste from.

Streaming: SRT primary, NDI backup

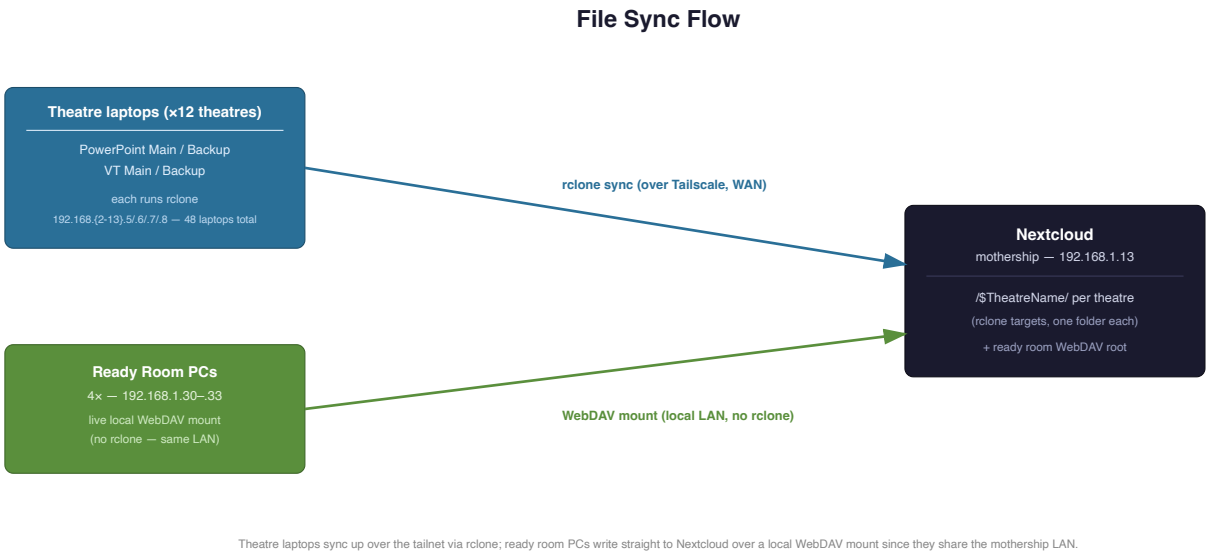
Each theatre's **BirdDog Play** receives the event's program feed two ways:



- **Primary — SRT**, unicast by IP from the mothership's Restreamer straight to each theatre's BirdDog Play. No discovery mechanism needed, so it's the more robust path — and it can originate directly from the ATEM's own built-in hardware streaming engine or from VMix, no computer required either way.
- **Backup — NDI**, from BirdDog Central. NDI normally discovers sources over local mDNS multicast, which Tailscale can't carry between theatres — so both ends instead register with a self-hosted **NDI Discovery Server**, a unicast-TCP service built into NDI 5+ specifically for this WAN/VPN case. Confirmed working on BirdDog Play's own firmware (BirdUI's Network panel has a Discovery Server field built in).

Why BirdDog Play at all, rather than a laptop running VLC? Zero on-site config, native dual-protocol support, broadcast-grade unattended reliability, and three ways to manage all 12 units centrally from the mothership — BirdUI, BirdDog Central, and Flock (a purpose-built fleet manager for exactly this device). Full comparison in `docs/birddog-play-rationale.md`.

File sync: rclone up, WebDAV local



Theatre laptops sync up over the tailnet via rclone; ready room PCs write straight to Nextcloud over a local WebDAV mount since they share the mothership LAN.

Theatre laptops (PowerPoint and VT, main and backup — 48 across all 12 theatres) each run `rclone`, syncing their theatre's folder up to Nextcloud over the tailnet. The mothership's 4 ready-room PCs skip rclone entirely and mount Nextcloud directly over local WebDAV, since they already share its LAN.

ATEM ISO ingest: near-real-time, no manual chunking

Each theatre's ATEM Mini Extreme ISO records up to 9 H.264 streams (8 camera ISOs + program) to a USB SSD — and, unlike most of this design, there's no laptop involved: the mothership pulls directly from the ATEM's **built-in FTP server** over Tailscale subnet routing, transferring only the bytes appended since the last check. That matters because the file is *constantly growing*: a naive whole-file re-sync on a schedule doesn't survive the math for a real session (a 3-hour, 6-stream recording is 50-80GB — re-uploading that *whole thing* every 10 minutes takes longer than 10 minutes, falling permanently behind). Two documented ways to get that incremental transfer: FTP's own `REST` /resume command directly (the default — plain rsync can't be used for this leg, since it needs SSH or an rsync daemon and the ATEM only speaks FTP), or `rclone mount` (presenting the ATEM's FTP share as a local path) paired with `rsync --append`, which transfers only the new tail without needing rsync's usual full-file checksum pass. Either way, the pull writes straight into the folder Nextcloud has mounted as External Storage, so there's no second upload step and no need to slice the video file itself — full design in `docs/atem-iso-ingest.md`.

VMix record ingest: same mechanism, a Windows SMB share instead

The 4 VMix PCs (2 per node) get the same treatment, just retargeted at a Windows SMB share instead of an FTP server — SMB is a genuine network filesystem protocol, so mounting it is the native way to access it here, if anything a better fit than the ATEM's FTP-only case rather than a stretch of the same pattern. `rclone mount` presents each PC's share as a local path, and `rsync --append` transfers only the new tail, same reasoning as the ATEM ingest's alternative approach. Each VMix node has its own dedicated A-1300 uplink — not shared with the theatre it physically sits near — so this traffic never competes with that theatre's ATEM ingest or SRT feed. Full design in `docs/vmix-record-ingest.md`.

Self-hosted admin tooling: UniFi Controller + GLKVM-Cloud

Two self-hosted services on the consolidated server, both following the same rule the rest of this design does — no dependency on a vendor's own cloud for managing this design's own infrastructure:

- **UniFi Controller** (self-hosted UniFi Network Application) gives a local, independent management console for the Cloud Gateway, separate from relying on `cloud.ui.com` — the Cloud Gateway ships with its own built-in controller and works fine without this, so it's optional, but not depending on a vendor cloud to administer the mothership's own router fits the pattern everything else here follows.
- **GLKVM-Cloud** gives centralized, browser-based SSH terminal and web-admin proxy access to all 14 GL-iNet routers at once, instead of 14 separate sessions — using GLKVM-Cloud's documented support for embedded OpenWrt devices, not its flagship KVM-hardware feature set (there's no physical KVM unit anywhere in this design). Full design, including a real WAN-port-forward collision this surfaced with the DERP server and how it was resolved, in `docs/glkvm-cloud.md`.

Fleet management: ATEM Overseer + Fleet Admin, and Flock

Three more mothership services, this time for the theatre hardware itself rather than the network infrastructure — all separate projects (`atem-overseer`, `atem-fleet-admin`, `flock`), not built from source in this repo:

- **ATEM Overseer** is a fleet-wide monitoring/tally dashboard — every theatre's ATEM sends it a dedicated low-bitrate monitoring stream (10 Mbps, folded into the bandwidth model below), giving one central multiview view of all 12 theatres instead of walking to each one.
- **ATEM Fleet Admin** provisions many ATEMs at once from model-aware config forms, either pushed live over the network (reaching all 12 theatres the same way ATEM ISO Ingest does — Tailscale subnet routing, no separate path needed) or exported as loadable XML + a media folder.
- **Flock** does the same fleet-manager job as ATEM Fleet Admin, but for the 12 theatres' BirdDog Play units — LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits. Each BirdDog Play also sends Flock its own 10 Mbps SRT preview stream, same treatment as Overseer's — also folded into the bandwidth model below.

Bandwidth: real numbers, right-sized VLANs

The 4 uplink VLANs turned out to be **venue-supplied, not our infrastructure** — 1 Gbps each, with expensive ports, which flips the goal from "isolate cleanly" to "minimum VLAN count at adequate performance." The full model, grounded in real figures (SRT ~5.8-10.4 Mbps via long-GOP H.264 + SRT/WireGuard overhead, ATEM ISO ingest ~62-94 Mbps plus the ATEM Overseer and Flock monitoring/preview streams — ~10.4 Mbps apiece — together dominating per-theatre upstream, full NDI at 1080p50 a very different ~130 Mbps roughly regardless of content), is in `docs/bandwidth-analysis.md`. Headlines:

- The current 4-VLAN plan runs at 27-39% utilization under normal operation — still comfortable, though real-time ATEM ingest plus both monitoring streams have eaten a genuine chunk of what used to be a much wider margin.
- **Consolidating to 2 VLANs** is still workable but has real eroded margin (53-78%, down from 41-65% before either monitoring stream existed) for normal SRT-primary operation — worth actively re-weighing now, not just noting. A **mass NDI fallback** (e.g. a Restreamer failure pushing all 12 theatres onto NDI at once) pushes 2 VLANs to 90% with no room left; 3-4 VLANs stay comfortable through that same event.

- **1 VLAN no longer works with real-time ATEM ingest, full stop** — it now exceeds capacity under normal expected load (106%), not just an unlucky worst-case day. Deferring ATEM ingest to end-of-session pulls (rather than real-time) is required to make 1 VLAN viable at all, not just a nice-to-have lever anymore.
- **A separate, previously-missed finding**: each theatre's own A-1300 has a tighter ceiling (~170 Mbps) than the shared VLAN — a theatre running ATEM ingest and an NDI fallback simultaneously can exceed *its own router's* capacity regardless of VLAN count. Mitigation: pause that theatre's ATEM ingest during an NDI fallback (cheap, config-only) — still works with both monitoring streams added, but with less spare margin than before. A higher-throughput GL-iNet model would also fix it, but not cleanly — see the doc for why.

Why Tailscale avoids DERP relay here

Every device in this design — all 12 theatres, both VMix nodes, the mothership — sits behind the *same* shared venue internet connection. That's an unusually favorable setup for Tailscale: direct connections are likely either over the true private LAN path or via a NAT hairpin through that one shared public IP, with public DERP relay only as a last resort. As extra insurance, a self-hosted DERP server (reachable at `derp.example.net:443`) runs on the mothership itself, so even relayed traffic never actually leaves the venue. Full reasoning in `docs/tailscale.md`.

Hardware, confirmed

Role	Hardware	Why
Mothership router	Ubiquiti **Cloud Gateway**	Can't run Tailscale natively — that role moves to a container on the services server instead
Consolidated services server	**Unraid**, single physical box (already on hand)	Hosts VMix + BirdDog Central as separate Windows VMs (only VMix gets GPU passthrough), plus Nextcloud/Restreamer/NDI Discovery Server/DERP/ATEM ISO Ingest/VMix Record Ingest/UniFi Controller/GLKVM-Cloud/ATEM Overseer/ATEM Fleet Admin/Flock as Docker containers. Chosen over TrueNAS SCALE for its more mature GPU-passthrough track record. Calculated target spec (storage pools, RAM, CPU, GPU tier, NIC validation) in `docs/server-specification.md`
Theatre + VMix node routers	GL-iNet **A-1300** (Slate Plus) ×14	12 theatre routers + both VMix node routers, same model throughout. ~170 Mbps WireGuard, comfortable headroom for normal operation (~55% combined); 2 LAN ports let the ATEM Mini Extreme ISO sit on its own dedicated port, away from the rest of the room's traffic — it carries the heaviest sustained load (near-real-time ISO ingest). Its own ceiling is the tighter constraint during an NDI fallback though (128% combined with ATEM ingest, resolved by pausing that theatre's ingest during the fallback) — margin worth watching if any further theatre-to-mothership stream ever gets added, see `docs/bandwidth-analysis.md`. Full case for GL-iNet + Tailscale specifically — segmentation, multi-WAN, pay-per-device WiFi economics, other uses beyond this event, and the same ceiling reasoning — in `docs/gl-inet-rationale.md`
Theatre playback	**BirdDog Play** ×12	Zero-config, native SRT+NDI, centrally fleet-managed — see above
Server NICs	**Bonded**, 802.3ad/LACP	SRT bandwidth isn't constant — LACP spreads the many simultaneous flows this box handles (12 theatres' SRT fan-out, rclone, Nextcloud, DERP) across both links for real aggregate headroom

See `docs/open-questions.md` for the reasoning behind every one of these, including the couple of things assumed rather than confirmed (and the fallback if an assumption turns out wrong).

Live editing: a separate subsystem, not the theatre network

2× MacBook Pro editing workstations plus dedicated fast storage at the mothership, editing event content as it arrives rather than waiting until after the event — deliberately on its **own dedicated 10GbE LAN** (`192.168.22.x`), not part of the Tailscale mesh or the theatre-facing network the rest of this repo is sized for. Editing needs

sustained multi-gigabit throughput per editor; putting that on the existing bonded 1GbE would either starve the editors or risk contention with the live ingest pipelines still running during the event.

Key decisions, each evaluated rather than assumed:

- **Dedicated NAS, not Nextcloud's own storage** — the same rclone/rsync pull that ingests each theatre's ATEM/VMix footage into Nextcloud writes a second copy straight to the edit-suite NAS at the same time, no chained re-sync. Decouples editing entirely from the live show's infrastructure.
- **A dedicated Mac mini running both the Resolve Project Server and Remote Render** — needed because the whole workflow below is one shared show project both editors work against, not two independent copies (confirmed **not** possible on a Blackmagic Cloud Store appliance itself, which is storage-only). Combining both roles on one box isn't documented as forbidden by Blackmagic but is thinly precedented in practice — see Decision 2 in `docs/live-editing.md` for the full honesty-check.
- **Blackmagic Cloud's internet sync service — not needed here.** A genuinely separate product from the Cloud Store hardware, built for cross-site collaboration; this is a single-venue setup with no second site to sync with.
- **A pre-built Resolve project structure, generated by resolve-configurator** — a companion project to this design — scaffolds a bin per theatre-day and an empty timeline per session ahead of time, plus writes the exact smart-bin rules as a one-time-apply recipe (Resolve's own scripting API can't create real smart bins). Once footage lands on the NAS and that recipe's been applied, each session's bin auto-fills with just its own clips — editors open a session, drag in, top-and-tail, export.

Also covers a complementary DIT (Digital Imaging Technician) ingest workflow for physical media — the ATEM's own USB SSD and any standalone camera cards — comparing ShotPut Pro, OffShoot, Silverstack, and the companion tools FoolCat (camera reports) and o/PARASHOOT (safe card reformatting). Full design in `docs/live-editing.md`.

Subnet map

Mothership on `192.168.1.x`; theatres contiguous from `.2.x` through `.13.x`, grouped 3 per physical VLAN purely for uplink cabling.

Theatre	Subnet	VLAN (uplink grouping)
1	192.168.2.x	VLAN 1
2	192.168.3.x	VLAN 1
3	192.168.4.x	VLAN 1
4	192.168.5.x	VLAN 2
5	192.168.6.x	VLAN 2
6	192.168.7.x	VLAN 2
7	192.168.8.x	VLAN 3
8	192.168.9.x	VLAN 3
9	192.168.10.x	VLAN 3
10	192.168.11.x	VLAN 4
11	192.168.12.x	VLAN 4
12	192.168.13.x	VLAN 4

VMix Node 1 (off Theatre 1): `192.168.20.x`. VMix Node 2 (off Theatre 4): `192.168.21.x`. See `docs/bandwidth-analysis.md` for whether this 4-VLAN grouping is the right count to actually build, or whether to consolidate.

Operations: building it, running it, and the road not taken

Three docs turn the design into something an on-site crew can actually execute:

- `docs/deployment-runbook.md` — the full ordered build sequence, Phase 0 (confirmations) through Phase 7 (pre-event verification), each step linking back to the doc with the detail. This is the doc to have open on build day.
- `docs/troubleshooting.md` — the gotchas surfaced *during design* (unreachable theatres, DERP relay when direct was expected, NIC bonds that won't form, ingest falling behind, missing monitoring previews), each as symptom → cause → fix, so they don't have to be rediscovered under show pressure.
- `docs/topology-alternative-tailscale-switches.md` — the road not taken: plain switches + Tailscale on generic Linux boxes instead of GL-iNet routers. Documented with working configs (`config/alt-tailscale-switch/`) so the decision can be revisited with evidence, but not recommended — the doc explains why.

Before build — open items

Design is finalized; these are the headline items still needing real-world input before the config in this repo gets applied (the complete list, including a few smaller ones, lives in `docs/open-questions.md`; the ordered build sequence is `docs/deployment-runbook.md`).

- [ ] Check the existing consolidated services server against the calculated target spec in `docs/server-specification.md` (CPU/RAM/GPU headroom, storage pool layout).
- [ ] Add the `derp.example.net` DNS record for the self-hosted DERP server.
- [ ] Verify the DERP hairpin isn't caught by the uplink-VLAN firewall block before relying on it.
- [ ] Confirm the real ATEM ISO bitrate/active-channel count and empirically verify partial-file playability.
- [ ] Confirm what VMix is actually recording and set up real SMB shares/credentials on all 4 VMix PCs.
- [ ] Confirm the final VLAN count with the venue, and decide whether a mass-NDI-fallback event needs to survive at full quality (drives 3-4 vs. 2 VLANs) — `docs/bandwidth-analysis.md` .
- [ ] Add the `kvm.example.net` DNS record and confirm `GLKVM_ACCESS_IP` 's exact format for the WAN-remapped GLKVM-Cloud ports — `docs/glkvm-cloud.md` .
- [ ] Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's real container images, and whether the ATEM/BirdDog Play can actually originate their monitoring/preview streams alongside their primary feeds — `docs/open-questions.md` #11.
- [ ] Implement the second write destination (the edit-suite NAS) in the actual ingest scripts — documented in `docs/live-editing.md` but not yet a code change — `docs/open-questions.md` #15.
- [ ] Decide how much footage the live-editing subsystem needs access to, confirm the Mac mini's combined Project Server + Remote Render role holds up under real load, and whether any standalone cameras are in scope — `docs/live-editing.md` , `docs/open-questions.md` #12-14.

Everything in this repo

Design docs

- `docs/topology.md` — full network layout: subnets, VLAN grouping, mothership, VMix nodes, theatre wiring
- `docs/server-specification.md` — calculated minimum spec for the consolidated server (storage pools, RAM, CPU, GPU tier, NIC validation)
- `docs/ip-address-map.md` — every device on the network, with its real IP (134 devices, including the live-editing subsystem's NAS, 2 MacBook Pros, and Project Server/Remote Render Mac mini)

- `docs/streaming-flow.md` — SRT/NDI streaming path, the NDI Discovery Server plan
- `docs/birddog-play-rationale.md` — why BirdDog Play over a laptop + VLC for theatre playback
- `docs/gl-inet-rationale.md` — why GL-iNet + Tailscale for segmentation/routing, multi-WAN/pay-per-device WiFi economics, and other uses (LED processors, mixing desks, mesh) beyond this event
- `docs/live-editing.md` — post-production subsystem at the mothership: dedicated NAS/network, Resolve collaboration decisions, the resolve-configurator-driven edit workflow, DIT ingest tool comparison
- `docs/file-sync-flow.md` — rclone + WebDAV sync to Nextcloud
- `docs/atem-iso-ingest.md` — near-real-time ATEM ISO ingest into Nextcloud, no manual chunking
- `docs/vmix-record-ingest.md` — same mechanism, targeting each VMix PC's SMB share
- `docs/glkvm-cloud.md` — self-hosted GLKVM-Cloud for centralized remote administration of the 14 GL-iNet routers
- `docs/bandwidth-analysis.md` — real-world throughput math against the venue's 1 Gbps VLANs, and how many are actually needed
- `docs/tailscale.md` — DERP-avoidance reasoning, the self-hosted DERP server, verification commands
- `docs/open-questions.md` — every decision made, how it was reached, and what's still genuinely open
- `docs/topology-alternative-tailscale-switches.md` — plain switch + Tailscale-on-Linux alternative to GL-iNet, not recommended but documented
- `docs/deployment-runbook.md` — ordered build checklist synthesizing every doc/config into one sequence
- `docs/troubleshooting.md` — known gotchas surfaced during design, symptom → cause → fix

Diagrams (`diagrams/`) — topology (full device inventory), streaming-flow, file-sync-flow, live-editing-dataflow (dual-write into the edit suite), and live-editing-project-structure (resolve-configurator → smart-bin recipe → editor workflow), all as standalone SVGs

Offline exports (`export/`) — every doc as a self-contained HTML file and an A4 PDF, plus a combined `full-documentation` version of each — see `export/README.md`

Generated configs

- `config/docker-compose.yml` — the entire consolidated server's Docker stack in one file, see `config/README.md`
- `config/tailscale-acl.json` — ACL policy (cross-theatre isolation) + the self-hosted DERP region
- `config/tailscale-up-commands.sh` — per-role `tailscale up` template
- `config/tailscale-up-all-devices.sh` — the same, fully instantiated for all 15 tailnet nodes
- `config/gl-inet/` — UCI network config for all 14 routers (12 theatres + 2 VMix nodes), generated from templates
- `config/unifi/` — Cloud Gateway VLAN/DHCP/firewall/port-forward reference
- `config/atem-iso-ingest/` — the FTP-pull ingest script (and rclone+rsync alternative) plus Nextcloud External Storage setup
- `config/vmix-record-ingest/` — the same rclone+rsync combo, targeting each VMix PC's SMB share
- `config/glkvm-cloud/` — the self-hosted GLKVM-Cloud docker-compose stack (`rttys` + `coturn`)
- `config/alt-tailscale-switch/` — configs for the plain-switch/generic-Linux alternative, not recommended but documented

Topology

Theatre nodes (×12)

Each theatre is its own subnet, NAT'd by its own GL-iNet **A-1300 (Slate Plus)** router. The router runs Tailscale as a subnet router (LAN + WAN side routing enabled), advertising the theatre's `/24`.

Physical wiring: the A-1300 has 1 WAN + 2 LAN gigabit ports. Both LAN ports are used:

- **LAN 1 → ATEM Mini Extreme ISO directly.** Dedicated port, so the router's other LAN traffic (rclone syncs, control laptop, etc.) never shares a switch segment with it. The ATEM carries the heaviest sustained traffic of any theatre device — the near-real-time ISO ingest pull (see `docs/atem-iso-ingest.md`) is a continuous 50-90 Mbps flow, well above BirdDog Play's — so it gets the dedicated port.
- **LAN 2 → 8-port unmanaged Netgear switch**, which fans out to everything else: BirdDog Play, PowerPoint Main/Backup, VT Main/Backup, Control laptop (5 devices, 3 spare switch ports).

All devices stay on the same `192.168.X.0/24` — this is a physical port split for traffic separation, not a VLAN/subnet split.

Role	Address	Physical connection
GL-iNet A-1300 router (LAN gateway)	`192.168.X.1`	—
ATEM Mini Extreme ISO	`192.168.X.2`	A-1300 LAN 1 (direct)
BirdDog Play	`192.168.X.20`	Netgear switch → A-1300 LAN 2
PowerPoint Main	`192.168.X.5`	Netgear switch → A-1300 LAN 2
PowerPoint Backup	`192.168.X.6`	Netgear switch → A-1300 LAN 2
VT Main	`192.168.X.7`	Netgear switch → A-1300 LAN 2
VT Backup	`192.168.X.8`	Netgear switch → A-1300 LAN 2
Control laptop	`192.168.X.10`	Netgear switch → A-1300 LAN 2

`X` per the subnet map in the top-level README: Theatre 1 → `2`, Theatre 2 → `3`, … Theatre 12 → `13`.

The ATEM's built-in FTP server (its ISO recording drive) is pulled continuously by the mothership over Tailscale subnet routing — see `docs/atem-iso-ingest.md`.

Alternative to the GL-iNet router, documented but not recommended: plain switch + Tailscale on generic Linux — see `docs/topology-alternative-tailscale-switches.md`.

Uplink VLAN grouping

Theatres are grouped 3-per-physical-VLAN purely to organize uplink cabling back to the mothership — **this grouping does not itself provide cross-theatre isolation**. Each GL-iNet A-1300 already NATs its own theatre LAN, and once every router joins the same tailnet, routes are reachable tailnet-wide unless Tailscale ACLs restrict them (see `config/tailscale-acl.json`).

These VLANs are venue-supplied, not our infrastructure — each capped at 1 Gbps, each port expensive. The 4-VLAN/3-theatres-each split below is safe but conservative; the real bandwidth math supports consolidating to

2 VLANs (6 theatres each) with genuine margin to spare — see `docs/bandwidth-analysis.md` for the full model and recommendation.

VLAN	Theatres	Subnets
1	1, 2, 3	.2.x, .3.x, .4.x
2	4, 5, 6	.5.x, .6.x, .7.x
3	7, 8, 9	.8.x, .9.x, .10.x
4	10, 11, 12	.11.x, .12.x, .13.x

Mothership — `192.168.1.x`

Ubiquiti **Cloud Gateway**; all 4 VLANs terminate here via separate links. Also hosts the 4× "ready room" computers (physical, untouched) and the hardwired WAN.

Consolidated services server (Unraid). Everything else that used to be described as separate mothership boxes — Nextcloud, Restreamer, the VMix instance, BirdDog Central, the NDI Discovery Server, a self-hosted DERP server, the ATEM ISO ingest pipeline, the VMix record ingest pipeline, a self-hosted UniFi Controller, self-hosted GLKVM-Cloud, ATEM Overseer + ATEM Fleet Admin (fleet monitoring and provisioning for the 12 theatre ATEMs), and now Flock (BirdDog Play fleet manager) — runs on a single physical server instead, virtualized with Unraid. The server already exists (no procurement needed) — known so far: **2× gigabit NICs (bonded — see below), a "decent" discrete GPU.** Full spec (CPU/motherboard, RAM, exact GPU model) is TBD — see `docs/open-questions.md` and the calculated target minimum spec (storage pool layout, RAM, CPU, GPU tier, all worked from this box's actual workload) in `docs/server-specification.md`.

Workload	Runs as	Notes
VMix instance	Windows VM, GPU passthrough	Windows-only; benefits from hardware encode
BirdDog Central	Windows VM (separate from the VMix VM)	Windows-only (tech specs); kept on its own VM so it can't take down the live VMix instance if it hangs
Nextcloud	Docker container	official image + MariaDB + Redis (see `config/docker-compose.yml`)
Restreamer	Docker container	official `datarhei/restreamer` image
NDI Discovery Server	Docker container	see `docs/streaming-flow.md`
DERP server	Docker container	self-hosted relay fallback — see `docs/tailscale.md`
ATEM ISO Ingest	Docker container	pulls all 12 theatres' ATEM ISO recordings via FTP, dual-writing to Nextcloud and the edit-suite NAS in the same pass — see `docs/atem-iso-ingest.md` and `docs/live-editing.md`
VMix Record Ingest	Docker container	pulls all 4 VMix PCs' recordings via SMB, same dual-write pattern — see `docs/vmix-record-ingest.md`
UniFi Controller	Docker container	self-hosted UniFi Network Application — local management/backup console for the Cloud Gateway, independent of cloud.ui.com — see `config/unifi/README.md`
GLKVM-Cloud (`rttys` + `coturn`)	2 Docker containers	self-hosted remote administration (web UI + SSH terminal) for the 14 GL-iNet routers — see `docs/glkvm-cloud.md`
ATEM Overseer	Docker container	fleet monitoring/tally dashboard for all 12 theatres' ATEMs — receives each theatre's monitoring stream, see `docs/bandwidth-analysis.md`
ATEM Fleet Admin	Docker container	bulk ATEM provisioning — model-aware config forms, live network apply via `atem-connection` over the same Tailscale subnet routing the ISO ingest pipeline uses, or XML+media folder export
Flock	Docker container	BirdDog Play fleet manager — LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits; receives each theatre's BirdDog Play preview stream, see `docs/bandwidth-analysis.md`

Workload	Runs as	Notes
Tailscale subnet router	Docker container, host networking	advertises `192.168.1.0/24`; replaces the earlier "Nextcloud host" plan now that Nextcloud itself is a container on this same box

Bonded NICs. Decided: bond the 2 gigabit NICs (802.3ad/LACP) rather than split traffic across them — SRT bandwidth isn't constant, and LACP's hash-based distribution spreads the many *simultaneous* flows this box handles (12 theatres' SRT fan-out, each to a different destination, the 12 incoming Overseer monitoring streams and Flock SRT previews, plus rclone/Nextcloud/BirdDog Central/NDI Discovery Server/DERP traffic) across both links by destination-IP/port hash — giving real aggregate headroom for bursty traffic, not just failover. Note this does **not** speed up any single flow; the benefit comes from having many distinct flows to hash across.

Assuming the Cloud Gateway supports LAG on the ports the Unraid server lands on (only UDM Pro/SE/Pro Max, UXG Enterprise, and EFG support port aggregation — not base Cloud Gateway models). If it turns out this one doesn't: drop an intermediate LACP-capable switch in between and bond through that instead — the bond doesn't care which device terminates it, only that something in the path speaks LACP, so this is a cheap fallback rather than a blocker.

One thing that does need doing regardless of which device terminates the bond: **LACP rate/hash policy must match on both ends.** Ubiquiti gear hardcodes LACP rate `fast` and hash policy `layer3+4`; Unraid's bonding defaults differ (`slow` rate, layer2 hash) and need to be set to match, or the bond won't form correctly.

Why Unraid over TrueNAS SCALE: both now support Docker and GPU-passthrough VMs, but this box needs two GPU/Windows-adjacent VMs alongside several containers, and Unraid has the more mature, better-documented GPU passthrough workflow plus a more polished one-click container experience (Community Applications) — useful since this may be maintained on-site by AV staff rather than a Linux admin. Trade-off: Unraid requires a paid license (Lifetime tier is a $129 one-time cost); TrueNAS SCALE is free. Only VMix needs the passthrough GPU — BirdDog Central is NDI routing/control, not video encode, so it runs as a plain VM.

Tailscale note: Cloud Gateway (UniFi-OS-based) generally can't run Tailscale natively without an unofficial container hack, so the mothership subnet-router role runs as a container on the Unraid box instead of the Cloud Gateway itself. See `docs/open-questions.md`.

VMix nodes (×2)

Each has its own **GL-iNet A-1300** router (same model as the theatre routers) and its own Tailscale connection — a separate node on the tailnet, not part of the theatre it physically sits next to, just uplinked near it:

Node	Uplinks near	Subnet	Contents
VMix Node 1	Theatre 1	`192.168.20.x`	4× BirdDog P400 cameras, 2× VMix PCs
VMix Node 2	Theatre 4	`192.168.21.x`	4× BirdDog P400 cameras, 2× VMix PCs

That makes **14 GL-iNet A-1300s total** across the network — 12 theatre routers plus these 2. Both LAN ports are bridged together on the VMix node routers (unlike the theatre routers, there's no single traffic-sensitive device here that needs its own dedicated port the way the ATEM does).

Each VMix PC's own recording is pulled to the mothership the same way the ATEMs' recordings are — near-real-time, over this node's own dedicated uplink (not the adjacent theatre's) — see `docs/vmix-record-ingest.md`.

Edit suite — `192.168.22.x`

Post-production subsystem at the mothership: 2× MacBook Pro editing workstations, a Mac mini running the DaVinci Resolve Project Server and Remote Render, and a dedicated fast-storage NAS, on their own 10GbE LAN — **not** on the Tailscale mesh (same room as the mothership, no distance for Tailscale to bridge) and not sharing the theatre-facing network's bandwidth budget. Routed to the mothership LAN only so the ingest containers can dual-write footage to the edit-suite NAS in the same pull that feeds Nextcloud — there is no separate recording-pool-to-NAS sync step. Full design, including the storage and project-server decisions and the DIT physical-media ingest workflow, in `docs/live-editing.md`.

See `diagrams/topology.svg` for the full visual layout — every theatre, every device, every real IP — and `docs/ip-address-map.md` for the same inventory as a text table. Per-device configs: all 14 routers in `config/gl-inet/`, the Cloud Gateway in `config/unifi/`, and Tailscale in `config/tailscale-up-all-devices.sh`.

Consolidated services server — minimum specification

The server itself is already on hand (see `docs/topology.md`) — this document calculates the **target minimum spec** to check that hardware against, working forward from the actual workload this box runs (`docs/topology.md`'s VM/container table) rather than a generic "decent server" guess. Where a figure depends on something this repo hasn't pinned down (the mothership's own VMix instance's camera/channel count, the event's actual day count/hours), that's flagged explicitly rather than assumed — see `docs/open-questions.md` for the running list.

Storage — three pools, not one array

Unraid's traditional parity array is built around mixed-size spinning disks with a single-write-at-a-time parity bottleneck, absorbed by an SSD cache pool that the overnight "mover" drains onto the array — a good fit for bulk archival storage, a poor fit for a box whose entire storage workload here is already flash. Recommendation: skip the legacy array entirely and use three separate Unraid **pools** (ZFS, available since Unraid 6.12), each sized and mirrored for what it actually holds.

Container/VM pool

Holds `docker.img`, all container `appdata` (including the three databases — MariaDB, MongoDB, Redis — that want fast, low-latency small-I/O access), and both Windows VM vdisks.

Item	Estimate
VMix instance vdisk	~100 GB (OS + app + local scratch)
BirdDog Central vdisk	~80 GB (OS + app)
`docker.img`	~50 GB
Combined `appdata` (12 containers + the Tailscale subnet router + 3 DBs)	~58 GB
Total	**~288 GB**

Minimum: 500 GB usable, mirrored NVMe (2× ~512GB+ NVMe) — comfortable headroom over the ~288 GB estimate, and mirrored because rebuilding two Windows VMs from scratch mid-event is real downtime, not just an inconvenience. NVMe specifically (not SATA SSD) because VM and database performance is latency-sensitive, not throughput-bound.

Drive class: mainstream consumer NVMe with onboard DRAM cache (not a DRAM-less budget drive) is genuinely sufficient here — no need for the enterprise tier the recording pool below needs. VM/appdata churn is nowhere near enough sustained write volume to meaningfully test even consumer-grade endurance ratings, and the workload cares about consistent low-queue-depth latency (how snappy a container restart or a DB query feels), which good consumer NVMe already delivers.

Content pool — 500 GB (as specified)

Nextcloud's own storage: theatre laptop content synced via rclone (established at 10-20 GB/theatre × 12 theatres = **120-240 GB**, see `docs/bandwidth-analysis.md`), plus Nextcloud's own app/database footprint and versioning overhead (flagged as a real multiplier in `docs/atem-iso-ingest.md`'s versioning caveat — version retention settings directly affect how much of this budget versioning eats into). 500 GB gives roughly 2-4x headroom over

the raw 120-240 GB content estimate, which comfortably absorbs version history without needing an aggressive retention policy from day one.

Recommendation: mirrored, SATA SSD is adequate (no NVMe-level latency requirement here) — mirrored less for data-loss prevention (every laptop is still the source of truth for its own content even if this copy is lost) and more for **availability**: a lost content pool mid-event means re-running the initial bulk sync for all 12 theatres, which is itself only a ~20-40 minute operation at 50-100 Mbps (`docs/bandwidth-analysis.md`) but is a real, avoidable disruption during a live event.

Drive class: same reasoning as the container/VM pool — mainstream consumer SATA SSD with DRAM cache, the cheapest reasonable tier that still isn't a DRAM-less budget part. Gentle, sporadic write pattern; no case for spending more here.

Recording pool — 8 TB, all-flash (as specified)

This is where the calculation actually matters — both *why* it has to be flash and *whether 8 TB is enough*.

Why all-flash, not spinning disk — it's not about raw throughput. Combined worst-case write load onto this pool, from `docs/bandwidth-analysis.md`'s own established figures:

Source	Per-unit worst case	Count	Total
ATEM ISO ingest	~90.4 Mbps (94 Mbps incl. WireGuard tax)	12 theatres	~1,085 Mbps
VMix Record ingest	~30 Mbps (assumed, unconfirmed — see `docs/open-questions.md`)	4 PCs	~120 Mbps
Combined			**~1,205 Mbps ≈ 151 MB/s**

151 MB/s sustained is well within a single 7200 RPM HDD's *rated sequential* throughput on paper — so the case for flash isn't the aggregate number, it's the **access pattern**. That 151 MB/s isn't one stream, it's **16 independent files being appended to concurrently** (12 ATEM + 4 VMix, each on its own ~60-90s poll cycle — see `docs/atem-iso-ingest.md` and `docs/vmix-record-ingest.md`), with Nextcloud's own targeted `occ files:scan` indexing calls layered on top of that. A spinning disk pays a real seek penalty (~5-15ms) every time it switches between those 16+ scattered write targets — under this specific concurrent-scattered pattern, achievable HDD throughput collapses to a small fraction of its rated sequential number, while any flash drive (no seek penalty) handles 151 MB/s of concurrent random-ish writes without strain. The "all-flash" requirement is about the write *pattern* this design creates, not the volume.

Does 8 TB actually cover the event? This is the one figure genuinely blocked on an input this repo doesn't have: **actual event duration (days × active-recording hours/day) isn't established anywhere in this design.** Working from the same combined-load figures:

	Combined rate	8 TB covers
Realistic (5-7 active ATEM channels/theatre, per `docs/bandwidth-analysis.md`)	~99.4 MB/s ≈ 349 GB/hr	**~23.5 hours**
Worst case (all 9 ATEM channels/theatre)	~150.6 MB/s ≈ 530 GB/hr	**~15.5 hours**

At a typical ~8-10 hour active-program day, that's roughly **1.5-3 days** of continuous worst-to-realistic-case recording before the pool fills — workable for a short event, tight for a longer one, and this assumes every theatre is actually near its "realistic" active-channel count for the whole day, which per the original design intent ("likely only 4-6 [channels] will actually have any data") may be conservative. **Confirm actual event length against this table before treating 8 TB as settled** — if it's a multi-day event without a periodic archive-off step already planned, either the day count needs to fit the budget above, or a nightly archive-to-cold-storage step needs adding to free capacity for the next day. The ingest pipelines write directly into Nextcloud's External Storage

location and, in the same pull, the edit-suite NAS (`docs/atem-iso-ingest.md`, `docs/live-editing.md`) — neither write has an offload stage that frees *this* pool. Tracked in `docs/open-questions.md`.

Redundancy — worth adding capacity for, not assumed in the 8 TB figure. This pool is the authoritative archive: the dual-write to the edit-suite NAS (`docs/live-editing.md`) does give the footage a second landing spot at ingest time, but that copy may be a curated subset (open-questions #12) and one of the candidate NAS units is RAID 0 — so a failure here still risks footage that can't be re-shot. Two ways to get 8 TB *usable* with 1-drive fault tolerance:

Option	Raw capacity needed	Trade-off
2× 8 TB NVMe, mirrored	16 TB raw	Simplest to reason about, fastest resilver, 50% capacity overhead
4× ~2.7 TB NVMe, RAID-Z1	~10.8 TB raw	75% capacity efficiency vs. 50%, slower resilver on failure, more drives to manage

Recommend the mirror for the same reason Unraid was chosen over TrueNAS SCALE elsewhere in this design (`docs/topology.md`) — operational simplicity, since this may be maintained by AV staff rather than a storage specialist, matters more here than squeezing out the last few TB of efficiency.

Endurance. Since this repo already treats the router hardware as reusable across events (see `docs/gl-inet-rationale.md`), the same applies here — a full pool fill is roughly 8 TB of actual flash writes (these are byte-range *appends*, not whole-file rewrites, so total written ≈ total recorded, not a multiple of it), and a mirrored pair each independently absorb that same 8 TB per event. Even a deliberately-generous worst-case estimate — ~530 GB/hr sustained for a full 24 h/day across ~20 event-days a year, beyond what the pool could even hold without nightly archive-off — comes to roughly 260 TB/year of actual writes to the pool — comfortably inside what even the *lower* enterprise endurance tier (see below) is rated for over a normal warranty period, so endurance headroom isn't actually the tight constraint here once the right drive class is picked.

Drive class: this is the one pool where consumer-grade flash — even a good, DRAM-cached, high-TBW consumer drive — isn't the right call, and endurance isn't the reason. The deciding factor is **power-loss protection (PLP)**: capacitor-backed circuitry that flushes a drive's write cache to permanent storage if power drops mid-write. Consumer/prosumer NVMe drives don't have this. A generator hiccup, a tripped breaker, or a UPS transfer glitch mid-event — exactly the kind of thing a live event's own power setup can produce — risks losing whatever was sitting in the drive's write cache at that instant, on the one pool in this whole design with no second copy of the data anywhere else. That's the actual argument for **enterprise-class NVMe** here specifically (not "enterprise is always better," which isn't true for the other two pools above) — server-validated drives with capacitor-backed PLP, provisioned for sustained high-queue-depth writes without the cache-cliff behavior consumer drives show once their fast write cache fills under prolonged concurrent load. Enterprise drives are also sold in more than one endurance tier (commonly around 1 drive-write-per-day vs. 3× that for a heavier-duty tier) — the lighter of the two tiers already has large margin over this workload's realistic annual write volume, so there's no need to pay for the heavier tier's endurance on top of the PLP requirement that's actually driving this choice.

Write caching

With every pool already flash (no legacy spinning array behind anything), the classic Unraid "SSD cache pool absorbing writes before the overnight mover" pattern doesn't apply here — there's no slow array for it to shield. What actually buffers the bursty, 16-concurrent-stream ingest pattern described above is:

- **RAM** — the OS page cache smooths write bursts before they hit the drives (see the RAM section below, which already budgets for this).

- **The drives' own onboard DRAM cache** — a real purchasing consideration: choose NVMe drives with onboard DRAM (not DRAM-less/QLC-only budget drives), since sustained multi-stream concurrent writes are exactly the workload DRAM-less drives handle worst.

NIC — validating the already-decided 2× bonded GbE

`docs/topology.md` already settled on 2× gigabit NICs, bonded (802.3ad LACP). Checking that against the heaviest combined load this design has identified — the mass-NDI-fallback disaster scenario from `docs/bandwidth-analysis.md`, layered on top of full ATEM ingest:

Direction	Load	Total
Inbound (theatres → mothership)	ATEM ingest worst case (~1,123 Mbps) + VMix ingest (~120 Mbps) + ATEM Overseer (~125 Mbps) + Flock (~125 Mbps)	~1,493 Mbps
Outbound (mothership → theatres), all 12 theatres on NDI fallback simultaneously	12 × ~130 Mbps	~1,560 Mbps

(Inbound total updated from an earlier ~1,205 Mbps once ATEM Overseer's and Flock's monitoring/preview streams were added — see `docs/topology.md` — each contributing ~125 Mbps fleet-wide on top of what was already accounted for.)

Both directions run independently over a full-duplex link, so they don't stack against each other — but each direction needs to fit across the bond's two physical 1 Gbps links via LACP's per-flow hashing (each *individual* flow is capped at 1 Gbps, since LACP doesn't split one flow across both links). With 14+ independent theatre flows in each direction (12 ATEM ingest + 12 Overseer + 12 Flock inbound, each individually well under 1 Gbps — worst case ~94 Mbps ATEM, ~10.4 Mbps apiece for Overseer/Flock, ~130 Mbps NDI outbound), the hash has plenty of separate flows to distribute across both links without any single one bottlenecking — **the existing 2× 1GbE bonded design still checks out, including against the worst-case disaster scenario**, not just normal operation. Margin has narrowed since this was first validated (~1,205 → ~1,493 Mbps inbound, both still comfortably under the bond's ~2,000 Mbps aggregate), the same erosion `docs/bandwidth-analysis.md` tracks everywhere else Overseer and Flock touch this design — see that doc's own worked-through comparison of this exact disaster scenario with and without pausing ATEM ingest fleet-wide, which is the more consequential mitigation than the NIC choice itself.

One load deliberately NOT counted against this bond: the dual-write to the edit-suite NAS. The same ingest containers that produce the inbound figures above also write every recording out again to the edit-suite NAS at `192.168.22.2` (`docs/live-editing.md`) — up to another ~1,243 Mbps of outbound at worst case. If that leg transited the theatre-facing bond it would stack on top of the NDI-fallback outbound and break the conclusion above — so it must **not**: the edit suite is its own 10GbE LAN, and this box needs a separate edit-LAN-facing interface (a 10GbE NIC, or at minimum its own dedicated port) for that traffic, counted in the spec alongside the 2× bonded theatre-facing GbE.

Worth it anyway: 2× 2.5GbE, if the box/switch already support it at no real added cost. Doesn't change the conclusion above, but many current motherboards ship 2.5GbE onboard already, and it buys real margin against the one thing this analysis can't fully account for — LACP hash distribution isn't perfectly even in practice, and real headroom above a "checks out, but not by a huge margin" number is cheap insurance if it's already sitting on the board.

RAM

Workload	Estimate	Basis
VMix instance VM	32 GB	vMix's own "Mid" tier guidance (4-6 cameras, HD multi-cam) — see GPU section for the caveat on which tier actually fits
BirdDog Central VM	8 GB	Routing/control app, not encode — lighter than VMix
Nextcloud + MariaDB + Redis	6 GB	InnoDB buffer pool + PHP workers + Redis cache
UniFi Controller + MongoDB	3 GB	Manages only the Cloud Gateway (the 14 GL-iNet routers are GLKVM-Cloud's, not UniFi's) — MongoDB's WiredTiger cache doesn't need much at this scale
ATEM Overseer + Flock	5 GB	Unlike the other admin/control-plane containers below, these two are actually decoding video — up to 12 concurrent theatre streams apiece for their monitoring dashboards (see `docs/bandwidth-analysis.md`), not just pushing config or metadata
Restreamer, NDI Discovery, DERP, both ingest containers, GLKVM-Cloud (rttys+coturn), ATEM Fleet Admin, Tailscale router	9 GB	9 lightweight containers, ~1 GB each budgeted
Unraid OS + ZFS ARC (3 pools, ~8.5 TB combined)	8 GB	Soft ZFS guidance is roughly 1 GB RAM per TB of pool for decent ARC hit rate — this is a floor, not a hard requirement, ZFS is adaptive
Subtotal	**~71 GB**	
Minimum, with headroom	**96 GB**	71 GB doesn't map to a clean DIMM configuration on an 8-channel platform (see CPU section) — 96 GB is the next practical capacity above it with real margin, not just rounding up to the subtotal
Recommended	**128 GB**	Matches vMix's own "64 GB for heavy Instant Replay" guidance with room to spare, and gives ZFS ARC meaningfully more room across all three pools

ECC, not just capacity. All three storage pools above are ZFS — ZFS leans on RAM for checksumming and its ARC read cache, and a bit flip in ordinary (non-ECC) RAM can get silently written into a checksum or into the recording pool's parity/mirror data before anyone notices, which defeats the whole point of using ZFS for the one pool holding unrepeatable footage. ECC RAM catches and corrects that class of error before it propagates. Pair this with a platform that has *validated* ECC support (see the CPU section below) — ECC only protects against silent corruption if it's actually enabled and working, which isn't guaranteed on every board that merely accepts ECC modules electrically.

CPU

Workload	Estimate	Basis
VMix instance	8 cores dedicated	vMix's official "Mid" tier: "Core i7 / Core Ultra 7, 3.5-4.5 GHz, 8+ cores" for 4-6 camera HD multi-cam
BirdDog Central	2 cores	Routing/control, not encode
Docker stack (12 containers + the Tailscale subnet router + 3 DBs)	5-7 cores shared	Mostly I/O-bound; Restreamer's SRT relay (remux, not transcode, per this design's primary path) and ATEM Overseer + Flock (each decoding up to 12 concurrent monitoring streams) are the heaviest individual containers, still light relative to VMix's own budget
Unraid OS/array overhead	2 cores	
Subtotal	**17-19 cores**	
Minimum	**16 cores / 32 threads, high sustained (not just boost) clock**	Rounds slightly below the subtotal's high end — see platform-class discussion below, which matters more than squeezing out the last 1-3 cores

The platform class matters more than the core count, and this is worth spelling out rather than just naming a chip. A 16-core desktop CPU is easy to find — the actual constraint is everything *around* it:

- **PCIe lanes.** Between the GPU and up to 8 NVMe drives spread across the three pools above, this box realistically wants 24-40+ usable PCIe lanes. Mainstream consumer desktop platforms (the socket a typical Ryzen 9 or Core i9/Ultra 9 sits in) top out around half that once you account for the GPU slot, chipset-shared bandwidth, and onboard I/O all competing for the same limited lane budget — workable if you're willing to compromise the number of drives or share lanes through bifurcation, tight otherwise. A **workstation/HEDT-class platform** (Threadripper PRO or a high-end Xeon W, not a consumer desktop chip) is the category that actually clears this without compromise — this is the same reason HEDT platforms exist at all, not a preference.
- **Validated ECC.** Some mainstream consumer boards technically accept ECC memory electrically, but whether ECC actually *functions* — gets detected, enabled, and does correction — varies board-to-board and isn't something to gamble on for the pool holding irreplaceable footage. Workstation platforms build ECC support into the platform itself, officially validated, not a maybe.
- **Sustained clock over peak boost.** vMix is a continuously-loaded, real-time workload, not a bursty one — a CPU's *base* clock under sustained all-core load is a better predictor of live-production performance than its headline single-core boost number. Worth actually comparing base clocks across whichever specific chips are shortlisted, not just core count.

Trade-off worth being upfront about: workstation/HEDT platforms cost meaningfully more (CPU + motherboard together) than the mainstream desktop alternative, and idle power draw for 24/7 operation runs higher too. That's a real cost, not a rounding error — but the alternative doesn't actually satisfy this box's lane and ECC requirements, so it's not a straightforward "save money" option, it's a different, smaller build with real compromises attached.

Also required, not a separate line item: IOMMU (Intel VT-d) or AMD-Vi support on both the CPU and motherboard, for GPU passthrough to the VMix VM — already flagged as unconfirmed in `docs/open-questions.md` #1; whatever CPU/board combination is checked against this spec, confirm this specifically. Both platform classes above support it; this is really only a risk on very old hardware.

GPU

VMix strongly recommends a dedicated NVIDIA GPU for NVENC hardware encode — already the existing design's own reasoning for GPU passthrough (`docs/topology.md`: "Windows-only; benefits from hardware encode"). Current vMix-published tiers (source):

Tier	GPU	Use case
Entry (1-2 cams)	RTX 5060 Ti (16 GB)	1080p stream + record
Mid (4-6 cams)	**RTX 5070 Ti**	HD multi-cam, 1080p replay
Demanding (8+ cams / 4K)	RTX 5080+	4K productions, multicorder

Genuinely can't pin an exact minimum here, and worth being upfront about why: the 2 VMix *nodes* clearly handle "4× BirdDog P400 cameras, 2× VMix PCs" each (`docs/topology.md`) — but what the mothership's own separate **VMix instance** VM actually processes isn't documented anywhere in this repo. Recommend the **Mid tier (RTX 5070 Ti-equivalent performance/encode class)** as a reasonable working target absent that detail — check the existing "decent GPU" already on hand (`docs/topology.md` / `docs/open-questions.md`) against this table once the mothership VMix instance's actual role/channel count is confirmed.

Card class: a workstation/professional-line GPU at that performance tier, not the consumer gaming card itself — this is a genuinely different recommendation from just naming a GeForce tier, and worth the reasoning:

- **Chassis fit.** Consumer gaming GPUs are built for open-air desktop-tower cooling (multiple large fans dumping heat sideways into the case, relying on front-to-back case airflow to clear it). A server chassis rarely has that airflow profile. Workstation cards in this class commonly ship as compact, single-slot, blower-style coolers that pull air in and exhaust it straight out the card's own bracket — the form factor is built for a server/rack enclosure, not retrofitted into one.
- **Driver behavior under continuous load.** Consumer GPU drivers are tuned for gaming sessions — burst to a high boost clock, then throttle. A continuously-loaded encode workload running for a multi-day event wants a driver branch built for sustained clocks under indefinite load, which is what the workstation/enterprise driver track is actually validated for.
- **ECC VRAM**, same rationale as ECC system RAM above — protects against silent corruption during a multi-day continuous run.
- **Licensing.** Consumer GPU license terms typically carry restrictive language around "datacenter"-style deployment that doesn't clearly define whether a single VM on owned hardware counts — a real, if loosely-worded and thinly-enforced, gray area for running a gaming card inside a server chassis long-term. The workstation/professional line's license doesn't carry that ambiguity; it's built for exactly this deployment shape. Worth noting this is a licensing question, not a technical one — passthrough itself works fine on either card class.
- The actual NVENC encode silicon is typically the **same generation** across the consumer and workstation lines at a comparable performance tier — this isn't a "buy workstation for better encoding" argument, the encode quality/capability is essentially equivalent. It's specifically about fit for continuous server operation.

Worth knowing when pricing this out: the historical cost gap between a workstation-class card and its consumer-tier equivalent has narrowed substantially in 2026 amid broader GPU memory/component pricing pressure — confirm current pricing before assuming the workstation option carries the large multiplier it used to.

Summary — minimum spec to check the existing server against

Component	Minimum	Why this class
CPU	16 cores / 32 threads, workstation/HEDT platform (Threadripper PRO or Xeon W class, not a mainstream desktop chip), strong sustained clock	PCIe lane count for GPU + up to 8 NVMe drives, validated ECC support
Motherboard	Workstation/HEDT chipset, IOMMU/AMD-Vi capable, validated ECC support, enough PCIe 4.0/5.0 lanes for 3 NVMe pools + GPU	Same reasoning as CPU — the two are a package
RAM	96 GB minimum, 128 GB recommended, ECC, populate all available channels	ZFS data integrity across all 3 pools; matches vMix's own heavy-use guidance
GPU	NVENC-capable, workstation/professional line at an RTX 5070 Ti-equivalent tier (unconfirmed exact tier — see above)	Server-chassis cooling fit, sustained-clock driver, ECC VRAM, cleaner passthrough licensing
Container/VM pool	500 GB usable, mirrored, consumer-grade DRAM-cached NVMe	Latency-sensitive VM/DB workload; consumer endurance is more than enough
Content pool	500 GB usable, mirrored, consumer-grade DRAM-cached SATA SSD	Gentle Nextcloud file I/O; no case for spending more
Recording pool	8 TB usable, mirrored (16 TB raw), enterprise-grade NVMe with power-loss protection	PLP is the deciding factor — this pool is the authoritative archive (the edit-suite NAS dual-write copy may be a subset, and may be RAID 0 — see `docs/live-editing.md`); confirm event duration against the capacity table above first
Network	2× 1GbE, bonded LACP (already decided, validated above) — 2× 2.5GbE if free to obtain — **plus a separate edit-LAN-facing interface (10GbE) for the dual-write leg**	Checks out even against the mass-NDI-fallback disaster scenario, provided the edit-suite dual-write never rides the theatre-facing bond

Everything above is a calculated **target**, not a purchase order — the actual box is already on hand per `docs/topology.md`. Next step is checking its real spec against this table and updating `docs/open-questions.md` #1 with the result.

IP address map

Authoritative, fully-instantiated device inventory for the whole network — every physical and virtual device, with its real IP. This is the source of truth; other docs reference this file rather than repeating addresses.

Uplink VLANs (Cloud Gateway transit networks — WAN side of each theatre's GL-iNet)

These transit subnets are a proposal — tags and ranges can be changed freely before deployment. They exist purely so each of the 4 physical VLAN groups has a working transit subnet; GL-iNet WAN ports don't need a predictable static address, DHCP is fine, since Tailscale rides over whatever address they get.

VLAN	802.1Q tag	Subnet	Cloud Gateway address	Serves
Uplink 1	101	10.10.1.0/24	10.10.1.1	Theatre 1–3 GL-iNet WAN ports
Uplink 2	102	10.10.2.0/24	10.10.2.1	Theatre 4–6 GL-iNet WAN ports
Uplink 3	103	10.10.3.0/24	10.10.3.1	Theatre 7–9 GL-iNet WAN ports
Uplink 4	104	10.10.4.0/24	10.10.4.1	Theatre 10–12 GL-iNet WAN ports

Mothership LAN — 192.168.1.0/24

Device	IP	Notes
Cloud Gateway (LAN gateway)	192.168.1.1	
Unraid server (host management IP)	192.168.1.2	Tailscale subnet router container shares this IP (host networking)
VMix instance	192.168.1.11	Windows VM, GPU passthrough
BirdDog Central	192.168.1.12	Windows VM, separate from VMix VM
Nextcloud	192.168.1.13	Docker container, macvlan
Restreamer	192.168.1.14	Docker container, macvlan
NDI Discovery Server	192.168.1.15	Docker container, macvlan, port 5959
DERP server	192.168.1.16	Docker container, macvlan, port 443 + STUN 3478/udp — see `docs/tailscale.md`
ATEM ISO Ingest	192.168.1.17	Docker container, macvlan — pulls all 12 theatres' ATEM ISO recordings via FTP, see `docs/atem-iso-ingest.md`
VMix Record Ingest	192.168.1.18	Docker container, macvlan — pulls all 4 VMix PCs' recordings via SMB, see `docs/vmix-record-ingest.md`
UniFi Controller	192.168.1.19	Docker container, macvlan, port 8443 (UI) + 8080 (device inform) — self-hosted UniFi Network Application, independent of cloud.ui.com
GLKVM-Cloud (`rttys`)	192.168.1.20	Docker container, macvlan, port 443 (UI) + 5912 (device) + 10443 (WebSocket) — self-hosted remote administration for the 14 GL-iNet routers, see `docs/glkvm-cloud.md`
ATEM Overseer	192.168.1.21	Docker container, macvlan — fleet monitoring/tally dashboard for all 12 theatres' ATEMs, receives each theatre's monitoring stream (see `docs/bandwidth-analysis.md`)
GLKVM-Cloud (`coturn`)	192.168.1.22	Docker container, macvlan, port 3478 tcp+udp — TURN relay for the above
ATEM Fleet Admin	192.168.1.23	Docker container, macvlan — bulk ATEM provisioning (model-aware config forms → live network apply via `atem-connection`, or XML+media folder export)

Device	IP	Notes
Flock	192.168.1.24	Docker container, macvlan — BirdDog Play fleet manager (LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits); receives each theatre's BirdDog Play preview stream, see `docs/bandwidth-analysis.md`
Ready Room PC 1	192.168.1.30	
Ready Room PC 2	192.168.1.31	
Ready Room PC 3	192.168.1.32	
Ready Room PC 4	192.168.1.33	

Theatres 1–12

Same 8-device pattern in every theatre; only the subnet octet (X) changes. GL-iNet A-1300 LAN 1 goes directly to the ATEM Mini Extreme ISO, LAN 2 feeds the Netgear switch carrying the rest (including BirdDog Play) — see `docs/topology.md` for the physical wiring.

Theatre 1 — 192.168.2.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.2.1
BirdDog Play	192.168.2.20
ATEM Mini Extreme ISO	192.168.2.2
PowerPoint Main	192.168.2.5
PowerPoint Backup	192.168.2.6
VT Main	192.168.2.7
VT Backup	192.168.2.8
Control laptop	192.168.2.10

Theatre 2 — 192.168.3.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.3.1
BirdDog Play	192.168.3.20
ATEM Mini Extreme ISO	192.168.3.2
PowerPoint Main	192.168.3.5
PowerPoint Backup	192.168.3.6
VT Main	192.168.3.7
VT Backup	192.168.3.8
Control laptop	192.168.3.10

Theatre 3 — 192.168.4.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.4.1
BirdDog Play	192.168.4.20
ATEM Mini Extreme ISO	192.168.4.2
PowerPoint Main	192.168.4.5

Device	IP
PowerPoint Backup	192.168.4.6
VT Main	192.168.4.7
VT Backup	192.168.4.8
Control laptop	192.168.4.10

Theatre 4 — 192.168.5.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.5.1
BirdDog Play	192.168.5.20
ATEM Mini Extreme ISO	192.168.5.2
PowerPoint Main	192.168.5.5
PowerPoint Backup	192.168.5.6
VT Main	192.168.5.7
VT Backup	192.168.5.8
Control laptop	192.168.5.10

Theatre 5 — 192.168.6.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.6.1
BirdDog Play	192.168.6.20
ATEM Mini Extreme ISO	192.168.6.2
PowerPoint Main	192.168.6.5
PowerPoint Backup	192.168.6.6
VT Main	192.168.6.7
VT Backup	192.168.6.8
Control laptop	192.168.6.10

Theatre 6 — 192.168.7.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.7.1
BirdDog Play	192.168.7.20
ATEM Mini Extreme ISO	192.168.7.2
PowerPoint Main	192.168.7.5
PowerPoint Backup	192.168.7.6
VT Main	192.168.7.7
VT Backup	192.168.7.8
Control laptop	192.168.7.10

Theatre 7 — 192.168.8.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.8.1

Device	IP
BirdDog Play	192.168.8.20
ATEM Mini Extreme ISO	192.168.8.2
PowerPoint Main	192.168.8.5
PowerPoint Backup	192.168.8.6
VT Main	192.168.8.7
VT Backup	192.168.8.8
Control laptop	192.168.8.10

Theatre 8 — 192.168.9.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.9.1
BirdDog Play	192.168.9.20
ATEM Mini Extreme ISO	192.168.9.2
PowerPoint Main	192.168.9.5
PowerPoint Backup	192.168.9.6
VT Main	192.168.9.7
VT Backup	192.168.9.8
Control laptop	192.168.9.10

Theatre 9 — 192.168.10.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.10.1
BirdDog Play	192.168.10.20
ATEM Mini Extreme ISO	192.168.10.2
PowerPoint Main	192.168.10.5
PowerPoint Backup	192.168.10.6
VT Main	192.168.10.7
VT Backup	192.168.10.8
Control laptop	192.168.10.10

Theatre 10 — 192.168.11.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.11.1
BirdDog Play	192.168.11.20
ATEM Mini Extreme ISO	192.168.11.2
PowerPoint Main	192.168.11.5
PowerPoint Backup	192.168.11.6
VT Main	192.168.11.7
VT Backup	192.168.11.8
Control laptop	192.168.11.10

Theatre 11 — 192.168.12.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.12.1
BirdDog Play	192.168.12.20
ATEM Mini Extreme ISO	192.168.12.2
PowerPoint Main	192.168.12.5
PowerPoint Backup	192.168.12.6
VT Main	192.168.12.7
VT Backup	192.168.12.8
Control laptop	192.168.12.10

Theatre 12 — 192.168.13.0/24

Device	IP
GL-iNet A-1300 (LAN gateway)	192.168.13.1
BirdDog Play	192.168.13.20
ATEM Mini Extreme ISO	192.168.13.2
PowerPoint Main	192.168.13.5
PowerPoint Backup	192.168.13.6
VT Main	192.168.13.7
VT Backup	192.168.13.8
Control laptop	192.168.13.10

VMix nodes

Each has its own router — GL-iNet A-1300, same model as the theatre routers — and its own Tailscale connection, not part of the theatre it physically sits next to.

VMix Node 1 — 192.168.20.0/24 (off Theatre 1)

Device	IP
Node 1 router (GL-iNet A-1300)	192.168.20.1
BirdDog P400 Camera 1	192.168.20.11
BirdDog P400 Camera 2	192.168.20.12
BirdDog P400 Camera 3	192.168.20.13
BirdDog P400 Camera 4	192.168.20.14
VMix PC 1	192.168.20.21
VMix PC 2	192.168.20.22

VMix Node 2 — 192.168.21.0/24 (off Theatre 4)

Device	IP
Node 2 router (GL-iNet A-1300)	192.168.21.1
BirdDog P400 Camera 1	192.168.21.11
BirdDog P400 Camera 2	192.168.21.12

Device	IP
BirdDog P400 Camera 3	192.168.21.13
BirdDog P400 Camera 4	192.168.21.14
VMix PC 1	192.168.21.21
VMix PC 2	192.168.21.22

Edit suite — 192.168.22.0/24

Post-production subsystem at the mothership — dedicated 10GbE LAN, not part of the Tailscale mesh (same room as the mothership, no reason to route it over the tailnet). See `docs/live-editing.md` for the full design.

Device	IP	Notes
Edit suite NAS	192.168.22.2	Blackmagic Cloud Store or generic 10GbE NAS — see `docs/live-editing.md` Decision 1
MacBook Pro — Editor 1	192.168.22.11	10GbE via Thunderbolt adapter/dock
MacBook Pro — Editor 2	192.168.22.12	10GbE via Thunderbolt adapter/dock
Mac mini — Project Server + Remote Render	192.168.22.20	Dedicated always-on machine, never an editor's own laptop — see `docs/live-editing.md` Decision 2

Device count

- Mothership: 20 devices (1 gateway + 1 server host + 2 VMs + 12 containers + 4 ready-room PCs)
- Theatres: 12 × 8 devices = 96
- VMix nodes: 2 × 7 devices = 14
- Edit suite: 4 devices (NAS + 2 MacBook Pros + Mac mini Project Server/Remote Render node), see `docs/live-editing.md` Decision 2
- **Total: 134 devices** across the network (excluding the 4 uplink VLAN transit addresses, which aren't per-device).

Streaming flow — SRT / NDI

Primary path (SRT)

```
VMix PCs (both nodes) --SRT--> Restreamer (mothership) --SRT (fan-out)--> BirdDog Play ×12 (one per theatre)
```

SRT is unicast-by-IP, so this path has no discovery dependency and should route cleanly over the tailnet.

Backup path (NDI)

```
BirdDog Central (mothership) --NDI--> BirdDog Play ×12 (one per theatre)
```

Bandwidth note: full NDI (not HX) at 1080p50 is ~125 Mbps per stream, roughly constant regardless of content — 12-22x SRT's figures, and without SRT's "static content is cheap" discount. Fine for a single theatre falling back — *provided that theatre's ATEM ISO ingest is paused for the duration*, since NDI plus the ingest together exceed the theatre's own A-1300 WireGuard ceiling — and a real capacity question if this path ever has to carry all 12 theatres at once (e.g. a Restreamer failure). See `docs/bandwidth-analysis.md` for the full impact at per-router, per-VLAN, and mothership-NIC level.

Caveat, and the fix: NDI relies on mDNS multicast for discovery, which does not cross a Tailscale-routed boundary — Tailscale is point-to-point WireGuard and doesn't forward multicast/mDNS (still an open feature request, not something MagicDNS papers over). Plain local mDNS discovery will not find BirdDog Central across theatres on its own.

Plan: run an NDI Discovery Server on the mothership. NDI 5+ added this specifically for WAN/VPN scenarios — it's unicast TCP (default port **5959**), so it works fine across the tailnet with no multicast involved:

- Run the discovery server as a small service on the mothership, alongside BirdDog Central.
- Point BirdDog Central (sender) and all 12 BirdDog Play units (receivers) at that server's IP. On Windows/macOS this is the NDI Tools Access Manager "Advanced" tab; on Linux it's `~/.ndi/ndi-config.v1.json`.
- Once a sender has a discovery server configured, it stops using mDNS entirely — it's visible only to finders pointed at the same server.
- Tailscale MagicDNS can supply a stable hostname for the discovery server's address instead of hardcoding its Tailscale IP, if the receiving device's config field accepts a hostname — a convenience, not a substitute for the discovery server itself.

Confirmed on BirdDog Play: BirdUI (the PLAY's web admin — browse to its IP, default password `birddog`) has a Network panel with NDI-specific settings including a Discovery Server toggle. Switch it ON, enter a comma-delimited list of Discovery Server IP address(es), click APPLY. BirdDog's own docs describe this as the mechanism for locating NDI sources "on different subnets" — exactly this cross-theatre case — and it supports multiple servers for failover. Sources: BirdDog PLAY User Guide, BirdUI User Guide. Change the default admin password before the event — BirdUI grants full device config access.

See `diagrams/streaming-flow.svg` — primary path solid, backup path dashed. For why BirdDog Play is the theatre-side receiver at all (vs. a laptop

- VLC), see `docs/birddog-play-rationale.md`.

Why BirdDog Play (not a laptop + VLC) for theatre playback

Each theatre's screen is driven by a **BirdDog Play** — a dedicated, purpose-built NDI/SRT decoder — rather than a general-purpose laptop running a media player. This documents the reasoning, alongside the closest realistic alternative.

Zero-config in the theatre

BirdDog Play needs no on-site setup beyond physical connection: plug it into the theatre screen and the network, point it at a source, done. No OS to patch, no application to configure per-session, nothing for on-site staff to babysit beyond power and a cable. That matters at this scale — 12 theatres, unattended for most of a live event.

Protocol support: SRT primary, NDI backup, both natively

- **SRT** — the primary path in this design (see `docs/streaming-flow.md`) is bandwidth-efficient by design (long-GOP H.264, see `docs/bandwidth-analysis.md` for real figures). In this design it originates from VMix via the mothership's Restreamer, but the protocol keeps options open: the ATEM Mini Extreme ISO also has a **built-in hardware streaming engine** that can stream SRT directly over Ethernet with no computer involved (Blackmagic tech specs), unused here but available as a fallback origin. BirdDog Play receives either without caring which one is sending.
- **NDI** — the backup path (BirdDog Central → BirdDog Play, via the NDI Discovery Server — see `docs/streaming-flow.md`) is handled natively too, with no separate plugin or configuration step.

Remote control from the mothership

Three ways to manage all 12 units centrally, none of which require physically visiting a theatre:

- **BirdUI** — each unit's own web admin, reachable directly (already the mechanism used to configure the NDI Discovery Server setting — `docs/streaming-flow.md`).
- **BirdDog Central** — already part of this design as a mothership VM (`docs/topology.md`) — centralized routing/control across the fleet.
- **Flock** — a fleet-management tool for exactly this device: LAN discovery, tag-based grouping, full BirdUI-parity settings per device, and batch edits across a group (Rust + Docker). Purpose-built for managing many BirdDog Play units as one fleet rather than 12 individual devices, which is precisely this deployment's shape.

Comparison: BirdDog Play vs. laptop + VLC

The realistic alternative is a laptop in each theatre running VLC, pointed at the same SRT/ NDI sources.

	BirdDog Play	Laptop + VLC
Setup	Plug in, done	Install/patch OS, install VLC, configure stream URL per source

	BirdDog Play	**Laptop + VLC**
SRT	Native, purpose-built decode	Native from VLC 3+ (SRT CookBook) — the one protocol VLC handles cleanly
NDI	Native, purpose-built decode	Requires a separate third-party plugin (NDI for VLC); reliably *outputting* VLC's playback as NDI is well-documented, but consistently *receiving and playing* an NDI source as input is far less so
Remote management	BirdUI, BirdDog Central, or Flock — purpose-built for this device	Nothing built in — would need RDP/VNC/TeamViewer bolted on, none of it purpose-built for stream monitoring/control
Unattended reliability	Broadcast-grade embedded device, designed to run 24/7 unattended	General-purpose OS: updates, driver issues, VLC hangs on a dropped stream sometimes need a manual restart
Recovery from a dropped stream	Automatic reconnect	Not guaranteed — depends on VLC settings, may need manual intervention
Footprint/power	Small, low power, unobtrusive	Full laptop: bigger, hotter, more power, another thing that can be bumped/closed/updated mid-show
Attack surface	Purpose-built embedded device	Full general-purpose OS — much larger surface
Cost per unit	Lower, purpose-built	A laptop is not free even if "already owned" — it's now unavailable for anything else, and carries more ongoing maintenance

Recommendation

BirdDog Play, as already implemented throughout this design. The laptop + VLC path is real and technically works for SRT specifically, but loses on every axis that matters for 12 unattended theatres running for the duration of a live event: zero-config deployment, clean NDI support, purpose-built fleet management (BirdUI/BirdDog Central/Flock), and broadcast-grade reliability without needing on-site intervention when something drops.

Why GL-iNet — segmentation, Tailscale overlay, multi-WAN, and beyond this event

This design already uses GL-iNet A-1300s throughout (see `docs/topology.md`, `docs/open-questions.md`). This document is the fuller "why" — the architectural reasoning behind putting a small router in every theatre instead of a flatter network, why Tailscale is the routing layer between them rather than site-to-site VLANs alone, the WAN-flexibility features this build doesn't currently use but is worth knowing about, and — since this is genuinely reusable hardware, not a one-event purchase — where else this same GL-iNet + Tailscale pattern earns its keep beyond a 12-theatre live event.

Network segmentation: one subnet per theatre

Each theatre is its own isolated `/24` (`192.168.2.0/24` through `192.168.13.0/24` — see `docs/ip-address-map.md`), NAT'd behind its own GL-iNet router, rather than 12 theatres' worth of devices sharing one flat network. Three concrete benefits fall out of that, beyond the obvious "keeps the IP plan tidy":

Individual DHCP servers. Each router runs its own DHCP scope for its own theatre. A DHCP problem — exhaustion, a rogue second DHCP server accidentally plugged in, a lease conflict — is contained to that one theatre's ~8 devices. On a flat network with one central DHCP server, the same fault takes out DHCP for the entire event at once. With 12 theatres + 2 VMix nodes + the mothership as 15 separate broadcast domains on the theatre-facing network (the edit suite's own 10GbE LAN is a further one — see `docs/live-editing.md`), there are 15 separate DHCP fault domains instead of one.

Multicast/broadcast containment. mDNS, SSDP, and NDI's own local discovery are all broadcast/multicast-based, which only works within a single L2 broadcast domain. Flatten 12 theatres onto one network and multicast traffic doesn't just fail to reach where it's needed — it also *floods everywhere it isn't*, since every discovery packet from every theatre reaches every other theatre's devices too. This design already runs into exactly that limitation on purpose: NDI's backup path needed a **unicast** NDI Discovery Server (see `docs/streaming-flow.md`) precisely because Tailscale's point-to-point routing (like any routed, non-bridged network) doesn't carry multicast across the theatre boundary. Segmentation isn't just tolerating that limitation — it's the reason cross-theatre multicast noise was never a problem to begin with, for anything that doesn't specifically need to be reachable across theatres.

Security — blast-radius containment. A compromised or simply misbehaving device in one theatre (malware on a presenter's laptop, a flaky ATEM firmware bug, an ARP-spoofing or DHCP-starvation attempt) is contained to that theatre's subnet. It cannot see, scan, or reach another theatre's devices or traffic, because there's no route between them — `config/tailscale-acl.json` only permits `theatre → mothership` and `mothership → theatre`, never `theatre → theatre`. This is the same point already made elsewhere in this repo — "VLAN grouping = physical uplink cabling only, cross-theatre isolation is enforced by Tailscale ACLs, not by the VLANs themselves" — worth restating here as the *reason* per-theatre segmentation is worth doing at all: it's not really about the subnets, it's about giving the ACL layer clean boundaries to enforce isolation across.

Tailscale as the overlay routing layer

Segmentation alone doesn't connect the theatres to the mothership — something has to route between the isolated subnets on purpose, while still keeping them isolated from each other. That's Tailscale's job here (full detail in `docs/tailscale.md`), and it's worth being explicit about why an overlay mesh is the right tool for that instead of relying on the venue's own physical network (VLAN trunking, static routes, a traditional site-to-site VPN concentrator):

- **Direct point-to-point tunnels, not routed via one central concentrator.** Every node-to-node WireGuard connection is negotiated independently and prefers a direct path when one exists — no single choke point every packet has to pass through, which matters for something latency-sensitive like SRT.
- **Doesn't depend on the underlying network cooperating.** A traditional VLAN-routed design needs the venue's switches trunking the right VLANs to the right ports, correctly, everywhere. Tailscale's NAT traversal (direct when possible, DERP relay as fallback — see `docs/tailscale.md`) works regardless of what the underlying physical topology actually looks like, which matters a lot when the venue's network isn't something this design controls in the first place (see `docs/bandwidth-analysis.md` — the venue-supplied-VLAN context that document was written against).
- **Isolation enforced at the overlay, not hoped-for at the physical layer.** The ACLs in `config/tailscale-acl.json` are what actually stop theatre-to-theatre traffic — independent of whether the venue's VLANs are configured exactly as planned. If a VLAN turns out flatter than expected, or a switch port is misconfigured, cross-theatre isolation still holds, because it was never depending on the physical segmentation to begin with.
- **No manual routing tables.** Each router advertises its own subnet (`--advertise-routes`) and accepts the others' where ACLs allow (`--accept-routes`) — see `config/tailscale-up-all-devices.sh`. Nobody is hand-maintaining a routing table across 14+ devices.
- **Identity independent of location.** A router's Tailscale identity is its WireGuard key, not an IP address on a specific network. That matters directly for the multi-WAN/failover discussion below: a router can switch from its wired uplink to WiFi or cellular mid-event, and every other node on the tailnet keeps reaching it under the same identity, with no reconfiguration anywhere else.

Multi-WAN: wired primary, WiFi/cellular failover

This build's current default is a single hardwired WAN per theatre (see `docs/topology.md`) — but GL-iNet's broader lineup supports genuine multi-WAN failover: multiple uplink sources (wired Ethernet, WiFi acting as a WAN client, a USB cellular modem) with automatic failover between them, no manual cable-swap or on-site intervention required. (Whether the A-1300 specifically has the automatic-failover feature is not documented — see the comparison table below; the Flint 2 documents it explicitly. Treat this as a lineup capability to verify per-model, not an assumed A-1300 feature.) For a deployment where most theatres run unattended for long stretches, that matters — a lost wired connection mid-session isn't something someone can necessarily walk over and fix before it costs real time.

Two failover directions are both worth having, and they're mirror images of each other:

- **Wired as primary, WiFi/cellular as backup** — this build's actual recommendation. If the venue's wired drop to a theatre fails (a bad cable, a switch fault, an ISP issue on the venue's side), the router falls back to venue WiFi or a cellular USB modem automatically. Bandwidth degrades under failover — that's expected and fine, it's a "keep the tailnet reachable and keep functioning at reduced quality" fallback, not a silent full-quality substitute.

- **WiFi as primary, for events that don't need this build's full streaming load.** Not every event this hardware gets reused for will run 12 theatres' worth of continuous SRT/NDI simulcast and near-real-time ISO ingest. For a lighter event — say, a conference room doing PowerPoint sync and basic control traffic, no live camera program feed — venue WiFi alone may comfortably cover it, and using each theatre's GL-iNet as a WiFi client eliminates the need for a wired drop to every room at all. Faster to set up, and avoids paying for (or waiting on) cabling runs the event genuinely doesn't need at that load level. This is the same "match infrastructure to actual load" reasoning `docs/bandwidth-analysis.md` already applies to VLAN count, just applied to WAN medium instead.

Limiting devices on pay-per-device venue WiFi

Some venues charge for WiFi **per connected device**, not by bandwidth tier. Connect every individual device in a theatre — the ATEM, BirdDog Play, both PowerPoint laptops, both VT laptops, the control laptop — directly to venue WiFi, and that's up to 7 billable devices per theatre. Put one GL-iNet router in each theatre instead, and the venue sees **one** device — the router — while everything behind it reaches the internet NAT'd through that single connection.

Worked example — real numbers from two major London exhibition venues' published 2025/2026 rate cards (cited as market examples of the pay-per-device model, not as this design's venue):

	ExCeL London	Olympia London (via eForce)
Single-device WiFi (per device)	£128 (advance) – £184 (late)	£171 (standard) – £205 (late)
Smallest bulk tier	10 devices: £647–932	6 devices: £519–623
7 devices, billed individually	£896–1,288	7×£171–205 = £1,197–1,435
7 devices, cheapest real bulk path	10-pack (over-buying): £647–932	6-pack + 1 single: £690–828
7 devices, hidden behind 1 router	**£128–184**	**£171–205**

Per theatre, that's roughly **5-7x cheaper** at ExCeL and **4-7x cheaper** at Olympia (the low end comparing against the cheapest bulk path, the high end against individual billing). Across all 14 GL-iNet routers in this design (12 theatres + 2 VMix nodes, each also with several devices behind it — see `docs/ip-address-map.md`), the gap runs into the thousands of pounds for the event, not a marginal saving.

Important — this isn't a loophole to just quietly exploit. Both venues' published technical policies explicitly prohibit exhibitors from bringing their own WiFi access points, mesh systems, or mobile hotspots onto the show floor without prior written authorization, and both reserve the right to detect and disconnect unauthorized hardware — this is standard, close-to-identical boilerplate at both ExCeL and Olympia, not an oversight. Using a GL-iNet router this way is a legitimate, common arrangement for professional AV vendors — but it means getting the venue's technical services team to authorize it ahead of time (registering the router, agreeing it's not itself broadcasting a public/guest SSID, etc.), not simply plugging one in on the day.

Sources: ExCeL Event Services Rate Card 2026 — IT, ExCeL Wireless Policy, London Book Fair 2025 eForce Internet Order Form (Olympia). Prices change; treat these as illustrative of the *shape* of the saving, confirm current figures with the actual venue before budgeting against them.

GLKVM-Cloud and remote administration

Already built into this design — see `docs/glkvm-cloud.md` for the full detail. In short: a self-hosted instance of GL.iNet's own GLKVM-Cloud platform gives centralized, browser-based SSH terminal and web-admin proxy

access to all 14 A-1300s at once, using GLKVM-Cloud's documented support for embedded OpenWrt devices — not its flagship KVM-hardware feature set, since there's no physical KVM unit in this design. Worth restating here as part of the broader case for this hardware family specifically: choosing GL-iNet doesn't just get 14 capable routers, it gets a fleet that's centrally manageable as one console instead of 14 separate admin sessions, for free, using tooling the vendor already publishes and this design already self-hosts.

Beyond this event: other places this pattern is useful

None of the following is used in this build — worth documenting anyway, since this is reusable hardware and the same GL-iNet-router-plus-Tailscale pattern generalizes well beyond a 12-theatre conference.

Remote administration of LED video processors (Novastar)

Novastar's LED processor/controller line is normally driven by a laptop physically present on the same local network as the hardware. A GL-iNet router with Tailscale turns that into remote access from anywhere — same principle as GLKVM-Cloud giving remote access to this design's own routers, applied to a different vendor's hardware.

Novastar MX30, via VMP. VMP (Vision Management Platform — not "Video Management Platform") is Novastar's control app for the COEX line the MX30 belongs to. Its own documentation describes adding a controller by **typing its IP address directly** ("Add Controller → enter the IP"), not by picking it off a broadcast-discovered list, and explicitly documents a router-mediated topology as one of its two supported connection methods ("PC and controller connected to the same LAN via a router... suitable for systems in complex environments"). Control traffic runs over TCP port 5200 (UDP 5201 for the UDP variant), per Novastar's own Central Control Protocol documentation — a unicast, IP-addressed protocol, not a broadcast-dependent one, which is exactly the traffic pattern that crosses a Tailscale tunnel cleanly. VMP's manual never explicitly addresses VPN/WAN use (neither endorsing nor ruling it out) — the underlying protocol lining up cleanly with how Tailscale actually works is a real signal, not a confirmed guarantee. Novastar's own sanctioned path for genuine internet-native access is a separate product, **VNNOX** (a hosted cloud platform) — worth knowing about as the vendor's alternative, though it's a different management layer entirely, not "VMP over the internet." Sources: VMP User Manual, COEX Wiki — Device Connection and Setting, Central Control Protocol Instructions V1.5.0.

A note on "MCTRL880": that specific model doesn't exist in Novastar's current or past product catalog — the closest real sending cards are the **MCTRL660 / MCTRL660 Pro**, **MCTRL600**, and **MCTRL4K**. The rest of this section uses MCTRL4K, since that's also the model the Tailscale question below is about specifically; the same reasoning applies to the MCTRL660/600 family.

NovaLCT + MCTRL4K over Tailscale — checked specifically, genuinely uncertain, not a confirmed "yes." No official Novastar documentation, forum thread, or case study addresses this exact combination — worth being upfront about that rather than asserting it works. What's actually known, and why it points toward "probably, with a caveat":

- NovaLCT ships in different variants for different Novastar product families. For Novastar's **Multimedia Player** line (standalone async players, not sending cards), the official manual explicitly documents a cross-subnet workaround: *"If the terminal and NovaLCT are not on the same network segment but they can be pinged... select Specify IP, enter an IP address and click Search."* That's a confirmed, official, IP-based (non-broadcast) connection path — but for a different product family than the MCTRL sending cards.
- For the **Synchronous Control System** variant of NovaLCT — the one that actually talks to MCTRL4K/660/600-class sending cards — the manual only says NovaLCT "connects to the sending card

automatically" once the hardware connection is normal, with no equivalent "Specify IP across a routed network" language found in any version checked. That phrasing suggests the initial device-discovery step may depend on local broadcast/ARP rather than a directable IP search — and Tailscale, being a routed L3 overlay rather than a bridged L2 network, **does not carry broadcast/multicast traffic between peers** (the same limitation this design already worked around for NDI — see `docs/streaming-flow.md`).
- Once a connection exists, though, MCTRL-family control traffic itself is unicast TCP to a known IP on port 5200 — confirmed by Novastar's own COEX protocol doc and independently corroborated by two separate community reverse-engineering projects (Bitfocus Companion's Novastar module, an independent Wireshark-based protocol writeup). That part should traverse a Tailscale tunnel without issue.

Practical read: the control *traffic* should work fine over Tailscale; the open question is purely whether NovaLCT's own device-*discovery* step for sending cards needs local broadcast to find the MCTRL4K in the first place. If NovaLCT's Synchronous Control System build has its own "add by IP" option (unconfirmed here — check the actual installed version), that sidesteps discovery entirely and is the path to try first. **The documented fallback that doesn't depend on any of this**: the MCTRL4K also exposes its own **web-based configuration UI**, reachable by pointing a browser at its IP — a plain HTTP-to-a-known-address connection with no broadcast dependency, and Novastar's own docs already describe reaching it this way (they just assume same-LAN, not a routed tunnel — no technical reason a browser would care about the difference). If NovaLCT itself doesn't cooperate over Tailscale, the web UI is the thing to actually try.

Remote WiFi access point for mixing desk control apps

A GL-iNet router can also just be a **portable, purpose-brought WiFi access point** — useful anywhere a venue's own WiFi is unreliable, absent, or not trusted for control traffic, letting an engineer run **Yamaha StageMix**, **Allen & Heath Qu-Pad**, or **Mixing Station** (a third-party app supporting many console brands) from an iPad without depending on house WiFi at all.

This is a genuinely different configuration mode from everything else in this document, worth being explicit about: everywhere else here, the GL-iNet router creates its **own routed, NAT'd subnet** with its own DHCP (that's the whole point of the per-theatre segmentation this doc opens with). For mixing-desk control, the safe default is the opposite — configure the router as a **plain bridged access point**, putting the iPad on the exact same L2 broadcast domain as the console, not a separate routed subnet behind it. That distinction is the crux of whether each app actually works:

App	Discovery mechanism	Manual IP fallback	Works across a routed subnet?
Yamaha StageMix (CL/QL-series consoles)	None — manual IP entry is the *only* method, and the app's own compatibility check assumes same-subnet addressing	Yes, but it's not actually a cross-subnet workaround here	**Not supported** per the official manual — no documented cross-subnet mode at all
Yamaha StageMix (TF-series and newer)	AUTO mode, mDNS-style, gated by iOS's Local Network permission	Yes — an explicit MANUAL mode	**Yes, explicitly documented** — connects across a different subnet, at the cost of losing live meter data
Allen & Heath Qu-Pad	Device list, almost certainly UDP broadcast per A&H's general control-protocol architecture	Not documented specifically for Qu-Pad (A&H's general platform docs say manual IP is the intended workaround when broadcast is blocked, but this isn't confirmed for Qu-Pad itself)	**Probably, but under-documented** — bench-test before relying on it live
Mixing Station	Varies by console brand — broadcast-based for Behringer X32/M32 (OSC/UDP) and Soundcraft (HiQNet, where the *mixer*	Yes, universally — the app always supports typing a console's IP	**Depends on the specific console's own protocol**, not on Mixing Station itself

App	Discovery mechanism	Manual IP fallback	Works across a routed subnet?
	initiates the connection to the app), inherited native behavior for others		

Bottom line: bridged AP mode is the one setup that works identically across all of these, no per-app or per-console workaround needed. A routed subnet is conditionally viable (TF-generation StageMix explicitly supports it; Mixing Station's manual-IP entry helps, but the underlying console protocol still might not) — treat it as something to test deliberately, not assume, if the router's other NAT/firewall features are wanted badly enough to be worth the added risk. For a one-off "get StageMix working from front of house" use case, bridged AP mode is the boring, reliable default. Sources: QL StageMix V8 User Guide, TF StageMix V4.5 User's Guide, Qu Series Reference Guide, Qu-Pad Help Guide, Mixing Station — Getting Started, Mixing Station — Soundcraft/HiQNet.

Mesh and peer-to-peer: fewer physical drops

Worth being precise about two different things both loosely called "mesh" here, since they operate at completely different layers and don't depend on each other:

- **Tailscale's own overlay mesh** — already how every router in this design reaches the mothership and each other, regardless of physical topology (see the Tailscale section above). It doesn't care whether the underlying WAN path is a dedicated wired drop, WiFi, cellular, or a hop through another GL-iNet unit — Tailscale just needs *some* IP reachability, wired or wireless, to establish its own tunnels on top.
- **GL-iNet's radio-layer repeater/WDS mode** — what this section is actually about: physically extending network reach between GL-iNet units over WiFi (or a wired Ethernet daisy-chain) instead of running a separate venue drop to every location. Worth naming accurately: GL-iNet markets this as repeater/extender (WDS) mode, not a self-healing mesh protocol in the eero/Orbi sense — one-directional dependency on whichever unit is upstream, not automatic multi-path failover between mesh nodes.

Running several adjacent theatres off a single network drop. One router lands the venue's wired uplink and acts as the "gateway" unit for a small cluster of adjacent theatres; the rest relay off it wirelessly (repeater mode) or via a wired Ethernet daisy-chain, while each theatre-side router still runs its own isolated subnet and DHCP scope exactly as described earlier in this document — the segmentation and Tailscale ACL isolation between theatres doesn't change, only how many theatres share one physical uplink. This directly trades against the "expensive per VLAN port" reality already central to `docs/bandwidth-analysis.md`: fewer wired drops needed from the venue, at a real cost.

That cost is worth stating plainly, because it partially undoes an argument made earlier in this document. The segmentation section above frames per-theatre subnets as independent fault domains — a DHCP problem or device fault in one theatre can't touch another. Meshing multiple theatres behind one shared uplink **reintroduces a shared fault point at the physical/WAN layer**: if the gateway unit's own uplink fails, every theatre relaying through it loses connectivity together, not just one. The subnets themselves stay isolated from each other (Tailscale ACLs don't change), but they stop being independently *reachable*. Worth treating as a deliberate, scoped trade-off — fewer drops needed, in exchange for correlated failure across whichever theatres share that one drop — not a strictly-better free option. Bandwidth is shared across the group too, same caveat as any repeater hop.

Going fully peer-to-peer, no wired backhaul at all. The extreme version of the same idea: no theatre gets a wired venue drop, and GL-iNet units relay to each other entirely over WiFi, hop by hop, until reaching whichever unit (or units) actually has a real WAN connection. Useful where running cable is genuinely impractical (temporary/pop-up spaces, venues that charge heavily per drop, awkward physical layouts) — but stacks the

same shared-fault-point concern from above across every hop in the chain, plus WiFi's own range/interference/multi-hop-latency limits, which get worse the more hops are strung together. Reasonable for a small cluster of nearby rooms; not something to lean on for 12 theatres at once without real testing against the venue's actual RF environment first.

GL-iNet travel router comparison

Prices and specs below are current as of when this doc was written (mid-2026) — GL-iNet runs frequent promo pricing, and this isn't a live-updated table, so treat exact figures as indicative and re-check before procurement. GBP pricing was only reliably obtainable for a couple of models (GL-iNet's regional store pages don't consistently expose static GBP pricing to an automated check) — USD is the more solid figure throughout.

Model	Price (USD)	WireGuard throughput	Ports	WiFi	Cellular	Multi-WAN failover	Notes
Slate Plus (GL-A1300)	$70–100 (promo vs. list — `docs/bandwidth-analysis.md` uses ~$100)	~170 Mbps	2× GbE LAN, 1× GbE WAN, 1× USB 3.0	WiFi 5	USB dongle only	Not documented	This build's current choice — see `docs/topology.md`. Smallest/cheapest of the group.
Beryl AX (GL-MT3000)	$99	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1× USB 3.0	WiFi 6	USB dongle only (needs an add-on board for a real slot)	Not documented	Only 1 LAN port — doesn't fit this build's "ATEM gets its own dedicated port" wiring without an added switch.
Flint 2 (GL-MT6000)	$170	~900 Mbps	4× GbE + 2× 2.5G	WiFi 6	None	**Yes** — explicit failover + load-balancing	Highest throughput of the non-cellular models; much larger/heavier, poor fit for 12-per-theatre portability.
Slate 7 (GL-BE3600)	$150–170	~540 Mbps	1× 2.5G LAN, 1× 2.5G WAN, 1× USB 3.0	WiFi 7 (dual-band only — no 6GHz/320MHz, so limited real-world WiFi 7 benefit)	USB dongle only	Yes, per its own user guide	Newest of the group; compact.
Spitz AX (GL-X3000)	$380	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1× USB 2.0	WiFi 6	**Built-in 5G/4G modem, dual Nano-SIM**	**Yes** — Ethernet/repeater/cellular/tethering	No internal battery — needs external 12V power. The dedicated cellular-failover choice if built-in modem matters more than portability.

Model	Price (USD)	WireGuard throughput	Ports	WiFi	Cellular	Multi-WAN failover	Notes
Puli AX (GL-XE3000)	$410	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1× USB 2.0	WiFi 6	**Built-in 5G modem, dual Nano-SIM**	**Yes**	Same cellular capability as Spitz AX, plus a **built-in 6400 mAh battery** — genuinely portable/untethered for short sessions, at a real price premium.

"Mesh" caveat: every model above advertises **repeater/extender (WDS) mode**, not a true self-healing mesh protocol between multiple units — worth being precise about this distinction rather than assuming parity with consumer mesh systems (eero, Orbi, etc.). See the mesh/peer-to-peer section above for what that means in practice for running several theatres off one drop.

For this build specifically, the Slate Plus stays the right call, but the margin behind that call has eroded a real amount as this design has grown — worth being precise rather than repeating the original reasoning unchanged. Under normal operation its 170 Mbps ceiling is still comfortable (~55% combined, per `docs/bandwidth-analysis.md`'s latest figures, which now include the ATEM Overseer and Flock monitoring/preview streams added after this comparison was first written). The scenario that actually bites is a theatre's NDI fallback plus its ATEM ingest running at the same time — that now hits **128% of the router's own ceiling** (was 116% before either monitoring stream existed), still fully resolved by the existing config-only mitigation (pause that theatre's ATEM ingest during the fallback — see the bandwidth doc), but with less spare margin left in that mitigation each time a new upstream/downstream load gets added. **The 2 LAN ports for the dedicated-ATEM-port wiring is still the harder constraint than raw throughput** — it's why Beryl AX remains a trap regardless of its higher throughput ceiling. Spitz AX/Puli AX are the models worth reaching for specifically *because* of their built-in cellular — i.e. exactly the multi-WAN-failover and WiFi-as-primary-link scenarios earlier in this document, where a wired drop isn't guaranteed or a genuine backup path matters more than shaving cost/size. **Worth revisiting if a fifth stream is ever proposed**: at that point Flint 2's much larger throughput headroom (24% combined vs. the Slate Plus's 128%) may stop being a "nice margin, not worth the size/weight/cost" trade-off and start being the safer default — this doc's own recommendation was written assuming two theatre monitoring streams, not an open-ended number of them.

The Comet KVM range

GL-iNet's separate KVM-over-IP product line — not used in this build (see `docs/glkvm-cloud.md` for why: GLKVM-Cloud here is being used purely for router administration, no physical KVM hardware involved), but worth documenting on its own merits since it's the same self-hosted-cloud philosophy applied to a genuinely different problem: out-of-band access to a physical machine's console — BIOS, boot failures, a hung OS — independent of whether that machine's own network stack is even up. Same benefit already described for GLKVM-Cloud's router-admin use here: no dependency on GL.iNet's vendor cloud, works without a public IP, reachable over Tailscale just like everything else in this design.

Model	Price (USD)	Video	Ports	Cellular	Rack/multi-server	Notes
Comet (GL-RM1)	$70–90	4K@30 claimed — one independent review reports the HDMI input actually caps around 2K@60; not	1× GbE, HDMI in, USB-C/USB2 for KB/mouse emulation	No	Single-server only	The base model — noted (and ruled out of scope) in glkvm-cloud.md.

Model	Price (USD)	Video	Ports	Cellular	Rack/multi-server	Notes
		independently confirmed either way here				
Comet Pro (GL-RM10)	$180	4K@30, hardware H.264 encode, ~30-60ms latency	1× GbE, USB for KB/mouse, HDMI passthrough	No (WiFi 6 only)	Single-server	Adds a small onboard touchscreen and a USB "fingerbot" accessory for physical power-button control.
Comet X (GL-RM4PE)	$270	4K@30	**Quad HDMI/USB — up to 4 servers per unit**, single PoE Ethernet port powers the whole unit	No	**Yes — 10"/19" rack-mount**	The one genuinely built for a server-room/rack deployment rather than one-off remote access — the closest fit if this line is ever used to manage a rack of machines instead of a single box.
Comet 5G (GL-RM10RC)	$300	Up to 4K, hardware H.264 encode	1× GbE, HDMI in/out (loop), 2× USB-C, 1× USB 2.0	**Built-in 5G RedCap + 4G LTE fallback**	Single-server	The cellular-backed model — genuine out-of-band access even if the target machine's own network *and* the venue's primary internet are both down, since the KVM's own uplink doesn't depend on either. Directly mirrors the multi-WAN failover reasoning above, just applied to a KVM unit instead of a theatre router.
Comet Q (GL-RMQ1)	$80–130	N/A — DisplayPort Alt-Mode passthrough	USB-C only	No	N/A	**Different category, not a straight comparison**: a mobile-device (phone/tablet/laptop) remote-control dongle, not a server HDMI KVM. Was still in crowdfunding as of research — confirm general-availability/shipping status before treating pricing as current retail.

If this line is ever adopted, Comet 5G is the standout for exactly the same reason Spitz AX/Puli AX stand out among the routers: a genuinely independent uplink, not sharing fate with whatever network the thing it's managing (or the venue) is on. Comet X is the pick if the target ever becomes "a rack of servers" rather than one box. Same purpose-built-over-general-purpose-workaround logic this repo already applies elsewhere — see `docs/birddog-play-rationale.md` for the same argument made about BirdDog Play vs. a laptop running VLC.

Summary

For this build specifically, GL-iNet A-1300s plus Tailscale earn their place on three independent grounds that all point the same direction: **segmentation** contains DHCP, multicast, and security faults to a single theatre instead of the whole event; **Tailscale** routes between those segmented subnets without depending on the venue's own physical network cooperating, and enforces cross-theatre isolation at the overlay regardless of how the underlying VLANs actually turn out; and **GLKVM-Cloud** gives centralized administration of all 14 units as one console, self-hosted, for free.

The features this build doesn't currently use — multi-WAN failover, WiFi-as-primary-link, built-in cellular, mesh/repeater backhaul — aren't hypothetical extras. They're exactly the reasons this is reusable hardware rather than single-event kit: the same units, the same Tailscale-mesh architecture, and the same self-hosted-cloud philosophy (GLKVM-Cloud for routers, the Comet line for out-of-band KVM) apply directly to remotely administering LED processors, running a WiFi AP for mixing-desk control apps, or cutting the number of wired drops a venue needs to provide — all without re-deriving the architecture from scratch each time.

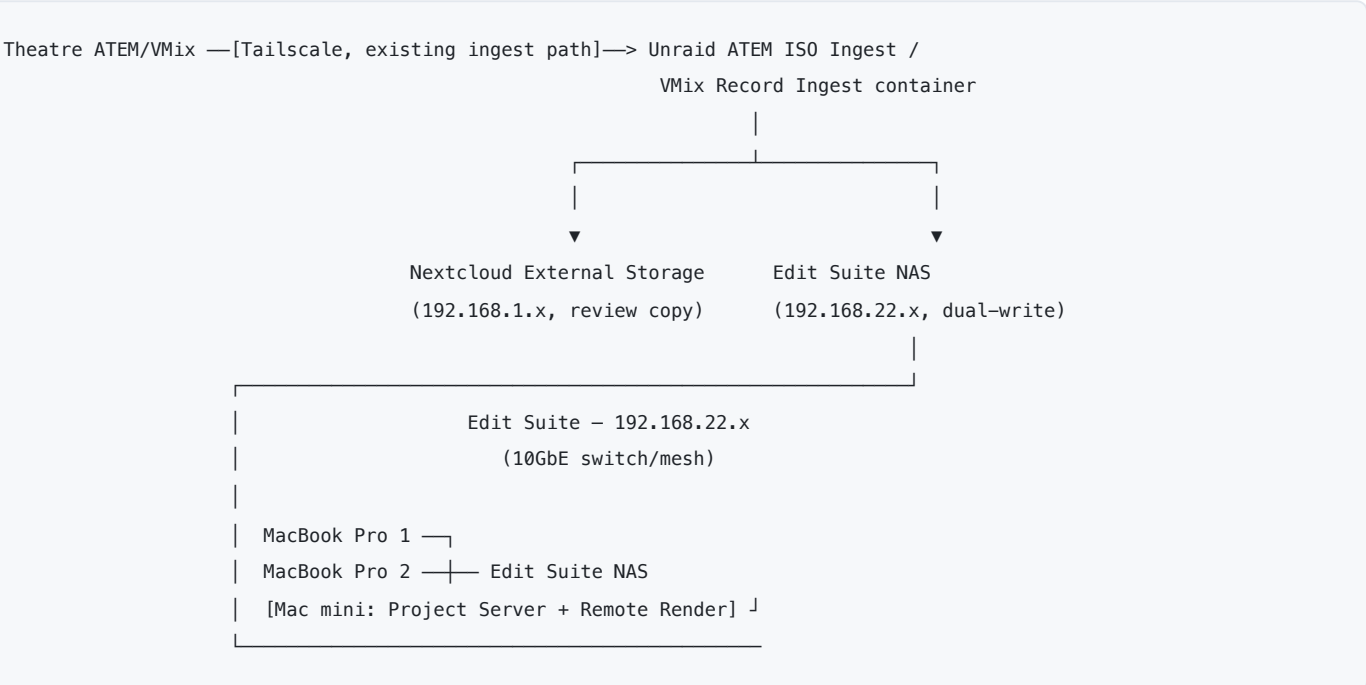
Live editing — post-production at the mothership

A separate subsystem at the mothership: 2× MacBook Pro editing workstations plus dedicated fast storage, editing event content as it arrives rather than waiting until after the event. Deliberately **not** part of the theatre-facing network this whole repo otherwise documents — different traffic pattern (sustained multi-gigabit scrubbing/export, not the bursty low-bitrate streams everything else here is sized for), different security boundary (no reason for 12 theatres to reach an editor's workstation or vice versa), and a genuinely different failure domain (an editing workstation crashing shouldn't touch the live show).

Why this needs its own network segment, not just a corner of the existing one

Every bandwidth figure elsewhere in this repo (`docs/bandwidth-analysis.md`) is sized around what the *live event* needs — SRT/NDI streaming, ATEM ISO ingest, file sync. Editing is a different animal: scrubbing or exporting 4K footage wants sustained throughput in the hundreds of Mbps to multiple Gbps *per editor*, not the tens of Mbps this design budgets per theatre. Putting that load on the mothership's existing 2× bonded 1GbE (`docs/server-specification.md`) would mean either starving the editors or risking contention with the live ingest pipelines still running during the event — the one thing this whole design goes out of its way to avoid elsewhere (see the NDI-fallback mitigation's own margin concerns in `docs/bandwidth-analysis.md`).

Recommendation: a dedicated 10GbE LAN for the edit suite, physically separate from the mothership's theatre-facing network, proposed as `192.168.22.0/24` (next free block after the VMix nodes — see `docs/ip-address-map.md`):



Same ingest containers already documented in `docs/atem-iso-ingest.md` and `docs/vmix-record-ingest.md`, writing to a **second destination** as well as Nextcloud — one pull off the ATEM/VMix source, two writes, not a chained re-sync (see Decision 1 below). No Tailscale within the edit suite itself — everything in this subsystem is in the same room as the mothership, unlike the theatre uplinks this repo's Tailscale mesh exists to solve for

(`docs/tailscale.md`). A simple routed connection back to `192.168.1.x` is enough for anything that still needs it; the edit suite doesn't need to be reachable *from* any theatre, and vice versa.

Standard MacBook Pros don't have built-in 10GbE — each needs a Thunderbolt-to-10GbE adapter or dock (e.g. OWC/Sonnet-class hardware) to actually hit these speeds; budget for that alongside whatever storage is chosen below.

See `diagrams/live-editing-dataflow.svg` for this same dual-write path as a diagram.

Decision 1: Nextcloud's storage vs. a dedicated NAS

Dedicated NAS, kept in sync via the same rclone/rsync pattern already built for ingest. Three reasons Nextcloud's own storage (the Unraid recording pool) is the wrong target to edit directly against:

- **It's sized and provisioned for the ingest workload, not editing.** The recording pool's whole design brief (`docs/server-specification.md`) is absorbing 16 concurrent low-bitrate append streams reliably — a completely different I/O shape from 2 editors scrubbing/exporting simultaneously.
- **Contention risk during the live event.** ATEM ISO Ingest and VMix Record Ingest are still actively writing to this same pool while editors would be reading from it — exactly the kind of shared-resource risk this repo's segmentation philosophy exists to avoid everywhere else.
- **Nextcloud's own serving stack (PHP/WebDAV) adds overhead a raw SMB/NFS-native NAS mount doesn't have** — fine for occasional file sync, not ideal for sustained media scrubbing.

A dedicated NAS, fed by the **same pull, writing to a second destination** — not a chained re-sync reading Nextcloud's own copy back out again, just the existing `config/atem-iso-ingest/` and `config/vmix-record-ingest/` containers' `rsync --append` step writing to both `/mnt/user/nextcloud-external/...` (Nextcloud's External Storage, for the near-real-time review path this repo already documents) and a second mount on the edit-suite NAS, in the same pass. One read off the ATEM/VMix source, two writes — cheaper and simpler than pulling twice, and the edit suite never has to wait on Nextcloud's own indexing (`occ files:scan`) to see new footage, since it's not going through Nextcloud at all. Same incremental-transfer reasoning as everywhere else in this repo: `rsync --append` only transfers new bytes as footage keeps growing, on both destinations. (The second write is a documented design, not yet a code change — the actual scripts currently write only the Nextcloud destination; tracked as `docs/open-questions.md` item 15.)

NAS options, roughly by cost:

Option	~Price	Notes
Blackmagic Cloud Dock 4 / Dock 2 / Pod (BYO drives)	$445-$1,535	Pure 10GbE network enclosure, no included storage — cheapest path to real throughput if drives are sourced separately
Generic 10GbE NAS (Synology/QNAP-class, populate with own drives)	Varies, comparable to Cloud Dock + drives	Not Resolve-specific, but plain SMB/NFS works identically; more flexible (can also run Docker containers — relevant to Decision 2)
Blackmagic Cloud Store Mini, 8TB	**$4,945**	4× M.2 NVMe, **RAID 0 — no redundancy**, 1×10GbE + 1×1GbE, up to 50 concurrent connections. Confirm the RAID0 risk is acceptable — a drive failure mid-event means falling back to a fresh pull from the mothership's own recording pool, not losing anything permanently, but it is a real mid-event disruption
Blackmagic Cloud Store Mini, 16TB	$8,245	Same, more headroom if editing against more than a curated subset of the event's footage

For 2 editors working from a curated subset of footage (not the full 8TB recording pool — see the open question below on how much actually needs to be pulled), the Cloud Store Mini 8TB is a reasonable fit and gets Resolve-

aware sync to Blackmagic Cloud "for free" if that's ever wanted later (see Decision 4). A generic 10GbE NAS is the cost-conscious or redundancy-conscious alternative. Either way, the Project Server is a separate dedicated Mac mini (Decision 2), not something hosted on the NAS itself — see below.

Decision 2: a dedicated Mac mini running the Project Server and Remote Render

The workflow this section is built around (see below) has **resolve-configurator build one shared show project** — a single set of Theatre/Date bins and per-session timelines both editors work against, populated by smart bins as footage lands — not two editors each on their own private copy. That design choice settles the "do we even need Collaboration" question from an earlier draft of this doc: **yes**, because both editors need to see the same live-updating bin/timeline structure, not separate forks of it. So this is a dedicated node, not an optional one.

What that node needs to run:

- **The DaVinci Resolve Project Server** — a small standalone app bundling a pinned PostgreSQL version, syncing project metadata (bins, timelines, locks) between editors in real time. **It does not move media** — actual footage still needs to be reachable by both editors via shared storage regardless (the NAS from Decision 1).
- **Resolve's Remote Render feature** — dispatches an export job from an editor's Deliver page to a separate networked Resolve Studio install, so the editor's own laptop isn't tied up rendering while cutting continues. Confirmed (Blackmagic's own Reference Manual): this is one job to one machine at a time — **not** a distributed render farm; Resolve never splits a single job's frames across multiple nodes.

Recommendation: a single dedicated Mac mini running both, rather than a Docker container on a NAS (dropped from an earlier draft — Blackmagic ships the Project Server as a native macOS/Windows/Linux app, not a container image, so this isn't a realistic option) and rather than either editor's own laptop. The most-cited failure mode across DIT/post-production forum threads is exactly the laptop case: if that machine sleeps, reboots, or the lid closes, *both* editors lose the shared project, not just one.

Combining Project Server and Remote Render duty on one Mac mini — confirmed workable, but thinly documented, so flagging the confidence honestly:

- Blackmagic's own documentation does not state a prohibition on co-locating the two roles, and architecturally there's no reason it wouldn't work — the Project Server is just a background PostgreSQL-backed service, Remote Render is just a licensed Resolve Studio instance running in a render role. But this specific combination isn't something Blackmagic explicitly confirms in writing, and it's a thin pattern in practitioner discussion — one forum thread asked this exact question ("could the project library be on this Mac mini as well?") and never got a direct answer; one real-world example found keeps a small Mac mini as project-server-only, separate from any render node. Worth treating as **very likely fine for a 2-editor event-scale deployment, not a Blackmagic-certified pattern** — a reasonable design decision, not a guaranteed one.
- **Both the render node and every editor's machine need DaVinci Resolve Studio**, not the free version — confirmed directly in Blackmagic's manual ("remote rendering does not work with the free version of DaVinci Resolve"). **$295 perpetual license per seat**, one-time cost. Blackmagic's own forum states an activation-code license can be active on 2 machines simultaneously, which in principle could cover an editor's laptop plus the render node from one seat — but this is a general Studio licensing term applied to this scenario, not a Remote-Render-specific clause, so budget one license per machine unless that's tested and confirmed to work as expected.

- **Remote Render requires a Network project library (PostgreSQL), not a Local one** — confirmed by a Blackmagic DaVinci support engineer on their own forum. In other words, Remote Render can't be added on top of two independent local projects; it only works once the Project Server (or Blackmagic Cloud) infrastructure already exists. That's further reason these two decisions are coupled here, not separable.
- **The media storage volume must be mounted on both the requesting machine and the render node** — confirmed in Blackmagic's manual. The Mac mini needs the same Edit Suite NAS mount the MacBook Pros use, not just database connectivity.
- **Performance is workable for 1080p H.264 but not fast** — no source benchmarks this exact scenario, so treat this as reasoned inference from adjacent data: Apple Silicon's H.264 hardware encode gap (base/Pro chips: one encode engine vs. two on Max, four on Ultra) matters far more at 4K/UHD than at 1080p, where independent reviews found little real difference between a Mac mini and a Mac Studio. But remote rendering itself runs slower than local rendering regardless of hardware — one documented case measured roughly half the fps remotely vs. locally. **Spec the Mac mini at M2 Pro/M4 Pro tier or better**, not the base chip, given it's doing double duty as database host and render node; a base-tier mini is a real risk for anything beyond light 1080p H.264 jobs.
- **Known gotcha to plan around**: submitting multiple render jobs to the same remote node at the same time can silently fail — send them one at a time. Fine at 2-editor scale, worth documenting as an operational note rather than something to engineer around.

Decision 3: using Blackmagic Cloud Store as the project server

No — not currently supported, and not just "not recommended." Blackmagic Cloud Store appliances are documented as SMB/NFS media storage plus a Resolve-aware sync target for the separate Blackmagic Cloud service (Decision 4) — nothing in Blackmagic's own documentation or independent DIT/post-production sources indicates they expose a general-purpose compute or container environment capable of hosting the Project Server's PostgreSQL database. Practitioner guidance consistently treats "where the media lives" (Cloud Store or any NAS) and "where the project database lives" (the dedicated Mac mini from Decision 2) as two separate questions with two separate answers — the project server needs its own host regardless of which storage option was picked.

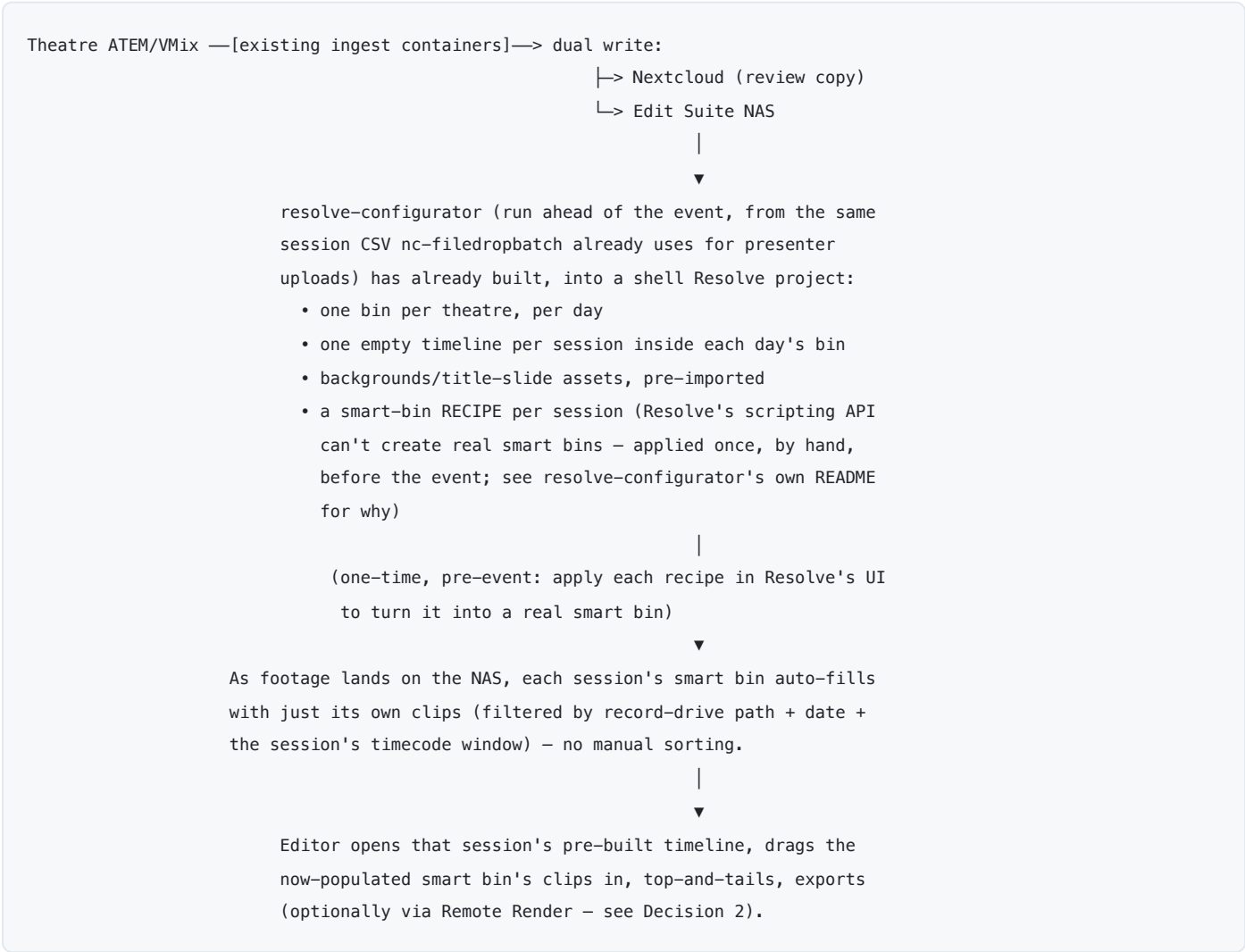
Decision 4: the cloud store's built-in internet sync functionality

Not needed for this design, and probably not worth turning on. "Blackmagic Cloud" is a genuinely separate product from the physical Cloud Store hardware — an internet-based sync service (own subscription: **~$5/month per project library** + **~$15/TB/month** for synced media) built for the case where collaborators are in *different physical locations* and don't want to deal with VPNs/firewalls to reach each other. Everything in this design is in one room at one venue — there's no second site to sync with, so this service would add an ongoing subscription cost and an internet-facing dependency for a problem this topology doesn't have. **Worth reconsidering only if there's a genuine need for someone off-site (a remote producer, a head-office stakeholder) to review cuts** — that's a real use case Blackmagic Cloud is built for, just a different one from what's being solved here. Track as a possible future need, not a current requirement.

Workflow: from ATEM pull to finished edit

Everything above (dedicated NAS, project-server decision) exists to support one actual pipeline — footage should already be sitting where an editor expects it, organized by session, by the time anyone sits down to cut.

The pieces:



What resolve-configurator actually builds (from its own README, not assumed): project format settings (frame rate/resolution, or any Resolve project setting via passthrough), a `Theatre / Date` bin structure, an empty timeline per session named from a template (e.g. `Start Time - Presenter`), pre-imported background/title-slide assets (global or per-theatre), and — since Resolve's scripting API genuinely cannot create smart bins, confirmed for the versions currently in use — a written recipe (`smart-bins.md` / `smart-bins.json`) giving the exact filter rule for each session (file path contains that theatre's record-drive name, date created is the session date, start timecode falls in the session's derived window). The tool builds everything the API *can* do automatically; the smart bins themselves are a one-time manual step per session, applied once ahead of the event from that recipe — after which they behave exactly like any other Resolve smart bin, auto-filtering as matching footage arrives.

Once that one-time setup is done, the editor's own job is genuinely just: open the session's timeline (already sitting in the right day/theatre bin, already named), the smart bin next to it already shows only that session's clips (no scrubbing through a whole day's unsorted recordings), drag them onto the timeline, top-and-tail to the actual session length, export. All of the organizing work happens ahead of time or automatically — editors are cutting, not filing.

See `diagrams/live-editing-project-structure.svg` for this whole pipeline as a diagram — from the session CSV through resolve-configurator's build, the smart-bin recipe's one-time hand-apply step, and the auto-fill down to the editor's own four-step job.

Ingest workflow: physical media (master drives, camera cards)

A separate, complementary path to the network-based ATEM ISO pull already built (`docs/atem-iso-ingest.md`) — that pipeline is optimized for near-real-time *review* access, but the ATEM's own USB SSD (and any standalone camera's SD/CFexpress cards, if this event uses cameras beyond the NDI-fed BirdDog P400s) remains the authoritative master, same "master vs. mirror" framing as the network ingest doc. Physically walking a drive/card to the edit suite for a proper checksummed offload — not just relying on the network pull — is standard DIT (Digital Imaging Technician) practice for anything that matters enough to edit.

Tool comparison

Tool	Vendor	Price	Role
ShotPut Pro	Imagine Products	$169 perpetual (+$59-70/yr updates after year 1), $60/30-day rental	Industry-standard dedicated checksummed offload + verify + PDF/CSV/MHL report — the classic "DIT cart" tool
OffShoot / OffShoot Pro	Hedge	$149-249 one-time, $49/30-day rental	Same core job (fast checksummed offload, multiple simultaneous destinations), simpler UI, no metadata/look-management layer
Silverstack Offload Manager	Pomfort	$139/yr, project licenses from $35	Pomfort's own cut-down offload-only tier, positioned against ShotPut/OffShoot for smaller productions
Silverstack XT / Lab	Pomfort	~$899/yr (project licenses ~$99-319)	Full DIT station: offload+verify+metadata/lens data+look management (CDL/LUT), and (Lab only) audio sync + transcode/dailies — a much bigger tool than this event likely needs unless additional standalone cameras with real production metadata needs are added
FoolCat	Hedge	$29/mo, $89-129/activation, $299 bundle	Companion **report generator**, not a copy tool — HTML/PDF camera reports with thumbnails, plugs directly into OffShoot's offload queue
o/PARASHOOT	OTTOMATIC GmbH	Free	Companion **card-safety** tool — confirms a card's files exist at the backup destination (filename+size, not checksum) then reversibly blanks the card so the camera prompts a clean reformat; pairs with OffShoot

Recommendation for this event's actual scale: ShotPut Pro or OffShoot Pro as the primary checksummed-copy tool — both are cheap, reliable, and exactly matched to "offload a drive/card, verify it, get a report" without paying for Silverstack's much larger metadata/look-management scope this design doesn't currently need (no standalone camera color pipeline in scope, no lens metadata being tracked). Add **FoolCat** only if stakeholders want visual per-session camera reports (director/producer-facing dailies-style PDFs) rather than just a copy-verification log. Add **o/PARASHOOT** (it's free) if any actual camera SD/CFexpress cards are in play and get reformatted/reused between sessions — cheap insurance against a card getting wiped before its footage is confirmed safe. **Silverstack XT/Lab is the right call instead** only if this event's scope grows to include standalone cameras needing real lens/metadata tracking or an in-house look/color pipeline — worth revisiting if that changes, not a default recommendation for the design as currently scoped.

A typical chained workflow, once decided: **ShotPut Pro/OffShoot (checksummed copy to the edit-suite NAS) → FoolCat (camera report, optional) → o/PARASHOOT (safe card reformat, optional)** — not competing alternatives, a real multi-stage pipeline other productions run this way.

Devices — new addition to the inventory

Device	Proposed IP	Notes
Edit suite NAS (Cloud Store or generic 10GbE NAS)	`192.168.22.2`	See Decision 1
MacBook Pro — Editor 1	`192.168.22.11`	10GbE via Thunderbolt adapter/dock

Device	Proposed IP	Notes
MacBook Pro — Editor 2	`192.168.22.12`	10GbE via Thunderbolt adapter/dock
Mac mini — Project Server + Remote Render	`192.168.22.20`	Dedicated always-on machine, not an editor's own laptop — see Decision 2

See `docs/ip-address-map.md` for how this fits the rest of the network's addressing.

Open questions this section introduces

- **How much of the event's footage actually needs editing access** — the full 8TB recording pool, or a curated/selected subset? Drives NAS capacity sizing in Decision 1.
- **Whether combining Project Server and Remote Render duty on one Mac mini holds up in practice** — Decision 2 flags this as architecturally sound but thinly documented by Blackmagic and practitioners alike; worth a real test before the event, not just before this doc.
- **Whether this event uses any standalone cameras with SD/CFexpress cards** beyond the NDI-fed BirdDog P400s — affects whether the physical-media DIT workflow has real cards to ingest beyond the ATEM's own USB SSD, and whether Silverstack's deeper metadata/lens-data features become worth their cost.
- **Whether the Cloud Store Mini's RAID 0 (no redundancy) is an acceptable risk** given the fallback is a fresh pull from the mothership's recording pool, not permanent data loss — but still a real mid-event disruption if it happens.
- **Implementing the dual-write in the actual ingest scripts** — `pull-iso.py` and `mount-and-sync.sh` currently write only the Nextcloud destination; the second write this whole section depends on is a real code change, tracked as `docs/open-questions.md` item 15.

File sync flow

Theatre laptops → Nextcloud (rclone)

PowerPoint Main/Backup and VT Main/Backup laptops in each theatre run `rclone`, syncing their theatre's folder (`/$TheatreName/`) up to the Nextcloud file server at the mothership.

```
Theatre laptop (PowerPoint/VT, Main/Backup) --rclone sync--> Nextcloud:/$TheatreName/
```

Ready room → Nextcloud (WebDAV)

Ready room PCs at the mothership write directly to Nextcloud via a WebDAV mount — a live local mount on the mothership LAN, not rclone, since there's no WAN hop involved.

```
Ready room PC --WebDAV mount--> Nextcloud (local to mothership LAN)
```

See `diagrams/file-sync-flow.svg`.

The other file flows into the same Nextcloud

This doc covers only the *document* sync — theatre laptops and ready-room PCs. Two recording pipelines write into the same Nextcloud by a completely different mechanism (near-real-time incremental pulls, not rclone sync of finished files), and one of them carries on to a second destination:

- `docs/atem-iso-ingest.md` — each theatre's ATEM ISO recordings, pulled over FTP
- `docs/vmix-record-ingest.md` — each VMix PC's recordings, pulled over SMB
- `docs/live-editing.md` — the same pulls dual-write to the edit suite's NAS alongside Nextcloud

ATEM ISO ingest — near-real-time, Nextcloud-aware

Goal: get each theatre's ATEM Mini Extreme ISO recordings (up to 9 H.264 streams — 8 camera ISOs + program — at ~10 Mbps each) onto the mothership's Nextcloud as close to real time as the hardware allows, without corrupting anything and without the theatre's uplink falling permanently behind.

The two hardware facts that shape this

- **The ATEM has a built-in FTP server**, and files can be browsed/transferred over it *while recording is still in progress* — confirmed via multiple independent accounts (aaronparecki.com, Directory Opus forum). This is also the only mechanism available — the unit has no NDI output and its Ethernet port doesn't expose the drive over SMB/NFS, only FTP.
- **The files are very likely safe to read mid-write.** Blackmagic markets editing an ISO recording in DaVinci Resolve *before the event even finishes* — that only works if the container format is structured so a partial file is valid up to whatever's been flushed so far (in practice, this means fragmented MP4 — periodic `moof`/`mdat` boxes rather than one index at the very end). Treat this as **very likely true given the marketed capability, but worth a quick empirical check** (copy a file mid-recording, confirm `ffprobe`/a media player can open it) before relying on it operationally.

Why not plain rsync, and why not plain rclone sync

Plain rsync can't connect at all — it needs SSH or an rsync daemon on the far end, and the ATEM only speaks FTP. There's no bridging that directly; the ATEM never exposes rsync or SSH.

A naive periodic `rclone sync`/whole-file re-copy doesn't survive the bandwidth math. Real numbers: at a realistic 4–6 active camera ISOs + program (~10 Mbps each), a 3-hour session is already 50–80 GB. Re-uploading the *entire current file* every 10 minutes would take longer than 10 minutes to transfer at the A-1300's ~170 Mbps ceiling — the sync falls permanently behind almost immediately for any session longer than about 15–20 minutes. Genuinely incremental transfer isn't a nice-to-have here, it's required.

Two ways to get genuinely incremental transfer — both implemented, pick one (see `config/atem-iso-ingest/`):

	Mechanism	Why it's incremental
`pull-iso.py` (default)	FTP's own `REST <offset>` command — the same "resume from byte offset" mechanism `curl --continue-at` / `lftp --continue` / `wget -c` use	Deterministic: explicitly requests only bytes past the last known offset. No cache layer involved.
`rclone-mount-rsync/` (alternative)	`rclone mount` presents the ATEM's FTP share as a local FUSE path; `rsync --append` (not rsync's general checksum-diff algorithm) transfers only the tail past the destination's current size	Also deterministic — `--append` seeks straight to the destination's size rather than reading/hashing the whole file, so it doesn't depend on rclone's VFS cache behaving well for a growing remote file — a real concern for mount-based approaches that `--append` sidesteps entirely

Both are legitimate. `pull-iso.py` has fewer moving parts (no persistent FUSE mounts to supervise); `rclone-mount-rsync/` uses tools you may already operate day to day, at the cost of 12 FUSE mounts to keep healthy. Neither uses rsync's *general* block-checksum algorithm (which genuinely would require reading the whole file

every pass) — that algorithm only saves network bytes when both ends speak the rsync wire protocol to each other, which isn't possible against an FTP-only source regardless of what sits in between.

Architecture

Centralized entirely on the Unraid server as its own container (`192.168.1.17`) — no software installed on any theatre laptop, consistent with how VMix/BirdDog Central/NDI Discovery Server/DERP are already consolidated there. Reaches each theatre's ATEM over Tailscale subnet routing (see `docs/tailscale.md` — this is exactly the "route between subnets where needed" case the subnet-router setup was built for). Described below for the default (`pull-iso.py`) — the `rclone-mount-rsync/` alternative differs only in the pull mechanism, writing into the same Nextcloud folders the same way.

```
ATEM (192.168.X.2, FTP) --[Tailscale subnet route]--> Unraid: atem-iso-ingest container
                                                                |
                                                  incremental REST-resume pull
                                                                |
                                                                v
                                            local mirror file, growing in lockstep
                                            (written directly into the folder Nextcloud
                                             has mounted as External Storage — Local)
                                                                |
                                                  targeted `occ files:scan` (short cycle)
                                                                |
                                                                v
                                                  visible in Nextcloud, growing
```

1. **Pull step** (`atem-iso-ingest` container, new — see `config/atem-iso-ingest/`): for each theatre, on a schedule (e.g. every 60–120s), list the ATEM's FTP directory, and for each media file, `REST`-resume from the last recorded byte offset and append the new bytes to a local mirror file. A small state file tracks per-file offsets so a restart doesn't re-pull from scratch.
2. **No separate upload/chunking step.** The pull step writes directly into `/mnt/user/nextcloud-external/TheatreN/ISO/` — the same path mounted into Nextcloud as an External Storage (Local) folder for that theatre. There's no WebDAV re-upload of the growing file at all, so the "whole-file re-sync is too slow" problem simply doesn't apply on this side — it's a same-host filesystem write, not a WAN transfer.
3. **Nextcloud awareness**: a short-interval, *targeted* `occ files:scan --path=...` (not `--all`) picks up the new file size on Nextcloud's side. Because this only touches one theatre's folder and runs locally on the same box, it's cheap even at a 1-2 minute cadence — unlike scanning the whole Nextcloud tree, which would not be.
4. **On session end**, a final pull + final targeted scan catches the last bytes and whatever moov/index finalization happens when the ATEM's recording actually stops.

Second write destination for live editing. The pull step also writes the same growing mirror file to a second mount — the edit suite's dedicated NAS (`192.168.22.x`), not just Nextcloud's External Storage. One read off the ATEM, two writes, not a separate re-sync reading Nextcloud's copy back out. See `docs/live-editing.md` for the full editing subsystem this feeds — not yet reflected in `config/atem-iso-ingest/pull-iso.py` itself, which currently only writes the one Nextcloud destination; adding the second write path is a real code change, tracked in `docs/open-questions.md`.

Bandwidth

Scenario	Aggregate	vs. A-1300 ceiling
Worst case, all 9 streams active	90 Mbps	tight but under ~170 Mbps
Realistic, 4 cams + program	50 Mbps	comfortable headroom
Realistic, 6 cams + program	70 Mbps	comfortable headroom

This rides the theatre's uplink in the *opposite direction* from the incoming SRT feed (~8–50 Mbps, see `docs/open-questions.md`) — if the link is genuinely full-duplex-capable at its rated throughput, those two shouldn't compete much. But the ingest does **not** have the upstream direction to itself: the theatre's ATEM Overseer monitoring stream and Flock SRT preview (~10.4 Mbps each) run theatre → mothership alongside it, plus bursty rclone. The authoritative per-router upstream model — ~129 Mbps worst case against the A-1300's ~170 Mbps ceiling — is in `docs/bandwidth-analysis.md`; the table above covers the ingest in isolation only. Worth validating the full-duplex assumption in practice rather than assuming it, since the ~170 Mbps A-1300 figure was a single-direction benchmark, not a confirmed simultaneous-bidirectional rating.

Master vs. mirror

Frame this as: **the ATEM's own SSD remains the authoritative final master** (complete, guaranteed-valid once recording stops) — pulled in full at the end of each session/day regardless. **Nextcloud holds a near-real-time, continuously-updated mirror** for early review/rough-cut purposes during the event. That framing resolves the tension between "want it fast" and "want it guaranteed correct": you get both, from two different consumers of the same underlying data.

Nextcloud versioning caveat

Nextcloud's versioning app may retain snapshots on repeated changes to the same file path. Configure version retention/expiry for the ISO folders (or disable versioning there specifically) so a multi-hour growing recording doesn't accumulate many near-duplicate historical versions before Nextcloud's own expiry curve catches up. Not a blocker, just worth tuning once this is running for real.

Open items

- **Confirm the 10 Mbps/stream figure and actual active-channel count** against the real ATEM recording settings, rather than relying on the estimate used for the bandwidth math above.
- **Confirm partial-file playability empirically** — copy a file mid-recording, check it opens/scrubs correctly, before treating the "safe to read mid-write" assumption as settled rather than "very likely."
- **Tune the scan interval and pull interval** against real session lengths and available Unraid CPU/disk headroom, once the server's full spec is known (see `docs/open-questions.md`).
- **Confirm the ATEM's actual FTP file/folder naming** (not assumed here — the ingest script lists whatever's present rather than hardcoding filenames, precisely because this wasn't confirmed against a real unit).
- **If using the `rclone-mount-rsync/` alternative**, test `rsync --append` against a genuinely growing file on a real ATEM before trusting it operationally — confirm it only transfers the new tail each pass (e.g. watch network throughput or `--itemize-changes` output), and budget for supervising 12 persistent FUSE mounts (auto-remount on failure), which `pull-iso.py` doesn't need at all.

VMix record ingest — near-real-time, Nextcloud-aware

Goal: same as `docs/atem-iso-ingest.md` — get each VMix PC's growing recording onto the mothership's Nextcloud as close to real time as possible, without corrupting anything and without falling permanently behind. Covers all 4 VMix PCs: VMix Node 1 (`192.168.20.21` / `.22` , off Theatre 1) and VMix Node 2 (`192.168.21.21` / `.22` , off Theatre 4) — see `docs/ip-address-map.md` .

Why this is actually simpler than the ATEM case

The ATEM Mini Extreme ISO only exposes its recording drive over FTP — a file-transfer protocol, not a filesystem-access protocol, so a mount-based approach there needs `rclone mount` as a FUSE bridge (that's the ATEM doc's *alternative* path; its default pulls via FTP `REST` resume with no mount at all). **VMix PCs run Windows, which natively serves SMB** — a real network filesystem protocol, designed to be mounted. There's no bridging trick needed here; SMB *is* the native way two machines share a filesystem on a LAN. That makes this the more natural fit for a mount-based approach, not a stretch of the ATEM pattern onto a different protocol.

Architecture

Same shape as the ATEM ingest, reusing the identical tools (`config/vmix-record-ingest/` mirrors `config/atem-iso-ingest/rclone-mount-rsync/`), as its own Unraid container (`192.168.1.18`):

```
VMix PC (192.168.2X.2X, SMB share) --[Tailscale, via that VMix node's own A-1300]--> Unraid
                                                        |
                                              rclone mount (SMB backend)
                                                        |
                                                        v
                                          rsync --append into Nextcloud's
                                          External Storage (Local) folder
                                                        |
                                             targeted `occ files:scan`
                                                        |
                                                        v
                                            visible in Nextcloud, growing
```

`rsync --append` (not rsync's general checksum-diff algorithm) is what makes this incremental — it seeks straight to the destination's current size and transfers only the new tail, rather than reading/hashing the whole file. Same reasoning as the ATEM ingest's `rclone-mount-rsync` alternative; see that doc for the full explanation of why this differs from rsync's usual delta-transfer mode.

Second write destination for live editing. Same as the ATEM ingest: the pull also writes to the edit suite's dedicated NAS (`192.168.22.x`) alongside Nextcloud's External Storage, one read off the VMix PC's SMB share, two writes. See `docs/live-editing.md` for the editing subsystem this feeds — not yet implemented in `config/vmix-record-ingest/mount-and-sync.sh` , tracked in `docs/open-questions.md` .

A native Linux CIFS mount (`mount -t cifs`) is a simpler alternative to `rclone mount` for SMB specifically — no FUSE bridge needed at all, since the kernel has first-class SMB client support. The tooling here uses rclone anyway, to reuse the exact same config/script pattern already built for the ATEM ingest rather than introduce a third mechanism. Worth reconsidering if operational simplicity outweighs consistency.

Why bandwidth isn't a shared concern with the ATEM ingest

Each VMix node has **its own dedicated GL-iNet A-1300 uplink and its own Tailscale connection** — it is not part of the theatre subnet it physically sits near (see `docs/topology.md`). So VMix Node 1's record-ingest traffic rides over Node 1's own link, entirely separate from Theatre 1's ATEM ISO ingest and SRT traffic. There's no contention between the two ingest pipelines sharing a link — each just needs to fit inside its own node's ~170 Mbps ceiling.

Open items

- **Confirm what VMix is actually recording** — program mix only, or per-input ISO the same way the ATEMs do — and at what resolution/codec/bitrate. None of this is assumed here; it directly determines the real bandwidth load on each VMix node's uplink.
- **Set up SMB sharing on the real PCs** — confirm the actual recording folder path and share name, and use a dedicated, minimal-privilege (read-only) local account for the ingest process rather than an admin/shared login.
- **Same partial-file-safety question as the ATEM ingest, but easier to answer**: VMix recording formats are more commonly designed for editability during capture than consumer camera formats, but this should still be empirically checked (open a currently-recording file's ingested copy in a player) rather than assumed.
- **Confirm the version-retention setting on Nextcloud** covers these folders too (see the same caveat in `docs/atem-iso-ingest.md`) — same risk of accumulating snapshots on a repeatedly-changing external-storage file.

GLKVM-Cloud — remote administration for the GL-iNet router fleet

Self-hosted GLKVM-Cloud (`gl-inet/glkvm-cloud`), run as Docker containers on the consolidated services server, gives centralized remote access to all **14 GL-iNet A-1300 routers** (12 theatres + 2 VMix nodes) — their web admin UI and an SSH terminal, both reachable through one browser session, without exposing any individual router's admin interface to the WAN or needing 14 separate tunnels. Same self-hosted-over-vendor-cloud rationale already used for the DERP server and the UniFi Controller: no dependency on GL.iNet's own `glkvm.com` cloud, no subscription, no third party in the path when administering the router fleet during a live event.

There is no physical KVM unit in this design. GLKVM-Cloud's flagship product is a KVM-over-IP hardware line (the Comet/GL-RM1) for out-of-band access to bare-metal servers — not used here. What's actually in scope is the platform's separately-documented **HTTP/HTTPS web proxy and device-management capability for embedded devices like OpenWrt and Raspberry Pi** — the A-1300s already run OpenWrt/UCI (see `config/gl-inet/`), which puts them squarely in that supported category.

Why centralize router administration at all

14 identical GL-iNet A-1300s (12 theatre routers, 2 VMix node routers) is exactly the kind of fleet that benefits from one console instead of 14 separate admin logins — same motivation as Flock for the BirdDog Play fleet (see `docs/birddog-play-rationale.md`). GLKVM-Cloud's documented features that apply directly:

- **Web-based SSH terminal** to any registered router, through the browser — no SSH client, no per-router WAN exposure, no VPN client needed on the admin's own device.
- **HTTP/HTTPS proxy to each router's own web admin (LuCI)** — same centralization for the GUI side.
- **Batch command execution across multiple devices** — push a config check or command to some or all of the 14 routers at once, rather than 14 individual sessions.
- **User group management + LDAP/OIDC** — if more than one person needs scoped admin access during the event, without sharing one shared router password.

Software: self-hosted GLKVM-Cloud — two containers

The upstream `docker-compose/docker-compose.yml` (reference) defines **two** services, both landing on the consolidated services server with their own macvlan IPs (same pattern as every other container there):

Service	Image	IP	Ports	Purpose
`rttys`	`glzhitong/glkvm-cloud:latest`	`192.168.1.20`	443/tcp (Web UI), 5912/tcp (device connection), 10443/tcp (WebSocket proxy)	The actual GLKVM-Cloud application
`coturn`	`coturn/coturn:edge-alpine`	`192.168.1.22`	3478 tcp+udp	TURN relay for WebRTC (separate off-the-shelf container, not GL.iNet's own code)

`coturn` exists in the upstream stack for GLKVM-Cloud's live remote-desktop/KVM feature, which doesn't apply to routers (there's no framebuffer to capture on an OpenWrt device) — deployed anyway as the standard

reference stack rather than assuming it's safe to drop; not confirmed whether `rttys`'s SSH-terminal/web-proxy features have any dependency on it internally. Revisit once actually running.

Manual Docker/docker-compose deployment supports both x86_64 and arm64 (deployment docs). Minimum self-host requirements per GL.iNet: 1 CPU core, ≥1 GB RAM, ≥40 GB storage, ≥3 Mbps — trivial next to what VMix/BirdDog Central already need on this box.

Registering the 14 routers

Each A-1300 registers to the self-hosted instance by running a connection script copied from the GLKVM-Cloud web UI (OpenWrt supports SSH/shell, so this is a normal `opkg`/script-based install, same mechanism as any other device type GLKVM-Cloud supports). Since every router already runs Tailscale and accepts routes back to `192.168.1.0/24` (see `docs/tailscale.md`), **that registration traffic rides the existing tailnet — no WAN port-forward needed for it.** WAN exposure is only about the *admin's* browser reaching the GLKVM-Cloud web UI itself from a device that isn't on the tailnet (see below).

WAN reachability — and the DERP port collision

Remote (off-venue) admin access to the GLKVM-Cloud web UI needs to be reachable from the internet, the same way `derp.example.net` is (`docs/tailscale.md`) — useful since, per this repo's own rule, only the routers and the mothership's Tailscale container are on the tailnet — no admin laptops or other client devices (see `docs/tailscale.md`). But the Cloud Gateway has one public IP, and DERP already owns WAN `443/tcp` and WAN `3478/udp` — a straight 1:1 forward of GLKVM-Cloud's own port numbers would collide with both. Resolution: forward different WAN-side port numbers to GLKVM-Cloud's real (internal, unchanged) ports —

WAN port	Proto	Forwards to	Why remapped
8443	TCP	`192.168.1.20:443`	WAN 443/tcp already goes to DERP
10443	TCP	`192.168.1.20:10443`	no collision, forwarded as-is
3479	TCP+UDP	`192.168.1.22:3478`	WAN 3478/udp already goes to DERP's STUN; kept TCP alongside it on the same alternate port for one consistent "DERP = 3478, GLKVM-Cloud = 3479" mental model, even though 3478/tcp alone was actually free

Suggested hostname: **`kvm.example.net`**, same domain as `derp.example.net`, pointed at the same public IP — reached as `https://kvm.example.net:8443`, with `GLKVM_ACCESS_IP` set to match so `rttys` advertises the right address/port back to the browser rather than its own LAN IP.

Not fully verified against a real deployment — confirm before relying on it: the existence and general purpose of `GLKVM_ACCESS_IP` is documented upstream, but its exact accepted format (hostname only vs. `host:port`) isn't confirmed here. If a plain env var doesn't cover the WAN-side port remap cleanly, the fallback is a second public IP or an SNI-routing reverse proxy in front of both DERP and GLKVM-Cloud on the literal 443 — more moving parts, only worth it if the simple remap doesn't work. Tracked in `docs/open-questions.md` (question 7).

See `config/glkvm-cloud/` for the docker-compose template, and `config/unifi/network-config.yaml` for the port forwards above.

Bandwidth analysis — venue-supplied VLANs

New constraint that changes the calculus: the 4 uplink VLANs are not our infrastructure — they're supplied by the venue over their existing structured cabling, each hard-capped at **1 Gbps**, and **each VLAN port is expensive**. The goal is minimum VLAN count at adequate performance, not the reverse. This replaces the earlier framing in `docs/open-questions.md` (which assumed we controlled VLAN addressing/count freely) — the *addressing* there is still fine, the *count* now has a real cost pressure behind it that this doc quantifies.

Assumption to confirm with the venue: the 1 Gbps figure is treated here as a standard switched-Ethernet full-duplex rating (1 Gbps *each direction*, not a shared aggregate) — true for virtually all modern structured cabling, but worth a one-line confirmation rather than assuming.

Tailscale/WireGuard overhead

Every figure below that crosses the tailnet (SRT, NDI, ATEM ingest, rclone) is wrapped in a WireGuard tunnel between the theatre's A-1300 and the mothership's Tailscale container — that adds a real, quantifiable tax on top of the raw payload. For IPv4 at full-size (1500B) packets, WireGuard's header overhead is 60 bytes (20B IP + 8B UDP + 32B WireGuard header) — **~4.0%** (header breakdown). All figures in this doc already have that 4% folded in.

Presenter internet does NOT get this tax. It's a separate path — presenter laptops go straight out to the internet via the A-1300's normal NAT/WAN, never touching the Tailscale tunnel at all (see `docs/tailscale.md`: only the 14 A-1300s and the mothership run Tailscale; nothing routes presenter traffic through it). It may still share the same physical VLAN link and the same router (unconfirmed — see the recommendation at the end of this doc), but it never shares the encrypted tunnel or its overhead.

Real-world per-traffic-type figures (WireGuard overhead included where it applies)

Traffic	Direction	Basis	Figure
SRT, baseline (static/motion-graphics)	Mothership → theatre	3-6 Mbps encoder for low-motion 1080p H.264, +25% SRT overhead (Haivision), +4% WireGuard	**~5.8 Mbps**
SRT, peak (opening/closing, higher motion)	Mothership → theatre	6-8 Mbps encoder + 25% overhead + 4% WireGuard	**~10.4 Mbps**
NDI backup, active (1080p50, full — not HX)	Mothership → theatre	Official NDI spec (docs.ndi.video) — **roughly constant regardless of content**, unlike SRT's H.264 long-GOP; the "mostly static graphics" discount does not apply here. +4% WireGuard	**~130 Mbps**
ATEM ISO ingest, realistic	Theatre → mothership	5-7 active streams × 10 Mbps (your figures) + 4% WireGuard	**~62 Mbps**
ATEM ISO ingest, worst case	Theatre → mothership	all 9 streams × 10 Mbps + 4% WireGuard	**~94 Mbps**
ATEM Overseer monitoring stream	Theatre → mothership	10 Mbps per theatre (your figure) + 4% WireGuard — flat, doesn't scale with ATEM channel count like ISO ingest does, see `docs/topology.md`	**~10.4 Mbps**
Flock BirdDog Play preview stream	Theatre → mothership	10 Mbps SRT per theatre (your figure) + 4% WireGuard — flat, same treatment as the Overseer stream, see `docs/topology.md`	**~10.4 Mbps**

Traffic	Direction	Basis	Figure
rclone file sync, steady-state	Theatre → mothership	sporadic post-ingest updates only — see below	**~5 Mbps avg**
rclone file sync, worst case	Theatre → mothership	multiple laptops syncing simultaneously (was already implicit in the per-theatre worst-case upstream total below; made explicit here)	**~15 Mbps**
rclone file sync, initial bulk ingest	Theatre → mothership	10-20 GB/theatre, **one-time**	not a sustained-load concern — see below
Presenter internet	Both, shared VLAN link, **no WireGuard tax**	web/slides/occasional demo video — **unconfirmed whether it shares this VLAN or is separate infrastructure**	~10-25 Mbps allowance

Initial bulk ingest isn't a bandwidth risk if scheduled right. 15 GB at a generously throttled 100 Mbps takes ~20 minutes; even at a conservative 50 Mbps, ~40 minutes. Run it during load-in/setup, not concurrent with a live session, and it's a non-issue regardless of VLAN count. Don't let this drive the VLAN-count decision.

Per-theatre sustained budget (during a live session, excluding the one-time bulk ingest)

	Realistic	Worst case
Upstream (theatre → mothership)	**~88 Mbps**	~129 Mbps
Downstream (mothership → theatre)	**~21 Mbps**	35 Mbps

Upstream is dominated by ATEM ISO ingest, with Overseer's and Flock's monitoring/preview streams together now a real secondary contributor rather than noise — **~88 Mbps** breaks down as ATEM ISO (~62 Mbps) + Overseer (~10.4 Mbps) + Flock (~10.4 Mbps) + rclone (~5 Mbps); **~129 Mbps** worst case as ATEM ISO (~94 Mbps) + Overseer (~10.4 Mbps) + Flock (~10.4 Mbps) + rclone worst-case (~15 Mbps). Both monitoring streams are flat regardless of ATEM channel count, unlike ISO ingest — see `docs/topology.md` for what they're for.

New finding: check the A-1300's own ceiling, not just the shared VLAN

The GL-iNet A-1300's own published WireGuard throughput (~170 Mbps, see `docs/open-questions.md`) is a **separate, tighter bottleneck** from the shared VLAN — every byte a theatre sends/receives over the tailnet has to pass through that one router's crypto engine, regardless of how much headroom the VLAN itself has.

Scenario	Upstream	Downstream	Combined	vs. ~170 Mbps ceiling
Normal (SRT active)	88.2 Mbps	5.8 Mbps	94.0 Mbps	55% — still comfortable, though the two monitoring streams together have eaten roughly a fifth of the headroom this used to have (was 43%)
NDI fallback, this theatre only, ATEM still ingesting	88.2 Mbps	130.0 Mbps	**218.2 Mbps**	**128% — exceeds the router's own ceiling** (was 116% before Overseer, 122% with just Overseer)

This holds **regardless of VLAN consolidation** — it's not a VLAN-count problem at all, it's a single-router problem that shows up the moment one theatre's ATEM ingest and NDI fallback run at the same time; Overseer and Flock's streams each widen that gap further, since both are genuinely new loads riding the same router regardless of what else is happening — and unlike ATEM ISO ingest, neither is something you'd necessarily want to pause during the exact moment (an NDI fallback) when knowing what's actually on screen matters most. Two caveats worth being honest about: the 170 Mbps figure is a vendor single-direction benchmark, not a confirmed simultaneous-bidirectional rating (flagged already in `docs/open-questions.md`), and even under a more

favorable per-direction reading, NDI alone (130 Mbps) is still 76% of that ceiling on its own, with little room left for anything else sharing the same crypto engine.

Practical mitigation: if a theatre's SRT feed fails and it falls back to NDI, pause or throttle *that theatre's* ATEM ISO ingest for the duration — near-real-time review footage is a lower priority than the live show staying on air, and the ATEM's own SSD keeps recording the full-quality master regardless of whether the ingest pipeline is running. This is a cheap, config-only fallback response, not a design change. **Still sufficient on its own, but with noticeably less margin than before both monitoring streams existed** — pausing just the ATEM ISO ingest leaves Overseer (10.4 Mbps) + Flock (10.4 Mbps) + rclone

- NDI (130 Mbps) at roughly **92% of the router's ceiling** (was 86% with just Overseer, 79% before either existed). Still under the ceiling, but the margin this mitigation buys back has shrunk each time a new monitoring stream was added — worth watching if a fourth one is ever proposed, since this specific fallback stops working once the two monitoring streams alone plus NDI exceed 170 Mbps on their own (they don't yet: 10.4+10.4+130=150.8, 89% — comfortable headroom even before rclone is added back in).

Worked through fully — NDI fallback + ATEM paused, at every level this design has a bottleneck at, since "still under the ceiling" at one figure doesn't tell the whole story on its own:

Level	Without the mitigation (ATEM still ingesting)	With it (ATEM paused)
Single theatre's own A-1300 (upstream + downstream combined, vs. 170 Mbps)	218.2 Mbps — **128%, exceeds the ceiling**	155.8 Mbps realistic (**92%**) / **165.8 Mbps worst case (97%)** — both under, but worst case leaves only ~7 Mbps of headroom
That theatre's VLAN group, upstream side only (NDI itself doesn't touch upstream — full-duplex, separate capacity)	n/a — this is exactly what the mitigation removes	Drops to just rclone + Overseer + Flock per theatre (25.8-35.8 Mbps) — even a fully-loaded 6-theatre group sits at 15.5-21.5% upstream, nowhere near a concern
Mothership's own bonded NIC, mass-fallback disaster case (all 12 theatres on NDI at once)	Downstream 1,560 Mbps + upstream ~1,553 Mbps worst case — both sides genuinely loaded	Downstream unchanged at 1,560 Mbps (NDI doesn't care about the ATEM side), but **upstream drops to just 310-430 Mbps** — the mitigation's biggest payoff shows up here, not at the single-router level

The one number worth remembering from this table: 97% at worst case, single theatre. The mitigation reliably resolves the per-router ceiling problem (128% → under 100% either way), but under the pessimistic rclone assumption it leaves only about 3% headroom — comfortably safe on paper, uncomfortably close in practice if anything else about that theatre's traffic runs a little hotter than modeled. Pausing ATEM ingest fleet-wide during a genuine mass NDI fallback is unambiguously the right call regardless — that's where the mitigation actually earns its keep, cutting the mothership's own upstream load from ~1,553 Mbps to ~310-430 Mbps (a ~72-80% reduction depending on realistic vs. worst-case rclone) at exactly the moment the network is already under the most stress.

Alternative: a higher-throughput GL-iNet model. Would fix this numerically, but not cleanly for either mid/high-tier option:

Model	WireGuard (vendor)	LAN ports	Form factor	~Price	NDI-fallback+ATEM+Overseer+Flock utilization
A-1300 (current)	170 Mbps	2	Pocket travel router	~$100	128%
Beryl AX (GL-MT3000)	300 Mbps	**1 only**	Pocket travel router	~$100-140	73%
Flint 2 (GL-MT6000)	900 Mbps	5 (2×2.5GbE+4×1GbE, minus WAN)	233×137×53mm, 761g	~$170	24%

Beryl AX is a trap for this design specifically — better throughput, but only 1 LAN port total, so it can't replicate the dedicated-ATEM-port wiring (`docs/topology.md`) without an external switch ATEM would then have to share anyway, undoing exactly the isolation that wiring was for.

Flint 2 genuinely solves it — ports to spare, comfortable margin (24% vs. 128%) — but it's ~3x the volume and ~5x the weight of the A-1300, no longer pocket-travel-router class for a 14-unit touring kit, at ~70% more per unit. Its 900 Mbps figure carries the same caveat as the A-1300's 170 Mbps: a vendor benchmark, not a confirmed real-world simultaneous-bidirectional rating under this exact mixed traffic.

Given the software mitigation above already fixes the same problem at zero hardware cost and no footprint change, a router swap isn't the recommended fix — it's a genuine option if the touring-kit footprint and cost increase are acceptable trade-offs, not a clear upgrade.

Is the current plan (4 VLANs, 3 theatres each) sufficient?

Yes, with large margin — at the VLAN level. Per-VLAN upstream aggregate: 3 × 88 = **~265 Mbps realistic (27% of 1 Gbps)**, 3 × 129 ≈ 388 Mbps even at worst case (39%). Downstream is lower still. The VLAN was never actually at risk — the per-router ceiling above is the one that actually bites first, and no VLAN count changes that.

Consolidating further (fewer, more expensive VLANs handling more theatres each)

VLANs	Theatres/VLAN	Upstream, realistic	Upstream, worst case	Verdict
4 (current)	3	265 Mbps (27%)	388 Mbps (39%)	Comfortable, as designed
3	4	353 Mbps (35%)	518 Mbps (52%)	Comfortable, but now over half the link at worst case
2	**6**	**529 Mbps (53%)**	**776 Mbps (78%)**	**Workable, real margin eroding — over half the link even at realistic load, no longer the wide 65% worst-case margin it had before either monitoring stream existed**
1	12	**1058 Mbps (106% — exceeds the link even at realistic load)**	**1553 Mbps (155%)**	**Not viable, full stop** — this used to only fail at worst case; it now fails under normal expected conditions

The worst-case column is normally the one that matters for a go/no-go call, but **1 VLAN has crossed a real threshold here**: it no longer needs an unlucky worst-case day to fail, it exceeds the link under the *realistic*, expected-case assumptions this whole document otherwise treats as the comfortable baseline. **2 VLANs (6 theatres each) is still the furthest safe consolidation**, but between Overseer and Flock, the two monitoring streams have eaten a real chunk of its margin — 78% worst-case (was 65% before either existed), and now over half the link (53%) even at realistic load, which wasn't true before. Worth re-weighing whether 2 VLANs is still the right call now that "comfortable" and "worst case" are closer together than they used to be. This is still **at the VLAN level; the per-router finding above is tighter and applies regardless**. This holds for normal operation (SRT primary) — **see the NDI fallback scenario below, which revises this** for the case where the backup path has to carry real load.

If the NDI backup path actually activates — very different math

The bandwidth model above assumes SRT is what's actually flowing downstream. NDI (full, not HX) is a fundamentally different animal: **~130 Mbps at 1080p50 with WireGuard overhead, roughly constant regardless of content** (official NDI spec) — NDI's compression doesn't get cheaper for static/motion-graphics content the way SRT's H.264 long-GOP does. That's ~12-22x the SRT figures above, and it changes which scenario the VLAN-count decision actually needs to survive.

A single theatre falling back is a non-event at the VLAN level — 130 Mbps extra on one theatre's downstream fits comfortably even inside a fully-loaded 6-theatre VLAN group (though see the per-router finding above — that theatre's own A-1300 is a tighter constraint than the VLAN). The real VLAN-level question is a **mass fallback**: since BirdDog Central/NDI is a separate path from Restreamer/SRT, a Restreamer failure (a single container, single point of failure) would push *every* theatre onto NDI simultaneously — that's the scenario worth sizing for, not the single-theatre case.

VLANs	Theatres/VLAN	Downstream if ALL theatres in the group fall back to NDI at once (incl. presenter internet)	Verdict
4 (current)	3	450 Mbps (45%)	Still comfortable
3	4	600 Mbps (60%)	Workable, less margin than 4 but not tight
2 (bandwidth-only recommendation above)	**6**	**900 Mbps (90%)**	**Tight — no real margin left**
1	12	1800 Mbps (180%)	Not viable at any scale

This is the direct trade-off the earlier 2-VLAN recommendation glossed over: it's safe for *normal* operation, but a mass NDI fallback at 2 VLANs leaves almost no headroom (90%, combined with whatever ATEM upstream is doing on the other direction at the same time). At the current 4-VLAN split, the same event is still comfortable (45%).

Revised recommendation: if the design needs to gracefully survive a mass simultaneous NDI fallback (Restreamer going down, most plausible trigger), **stay at 3-4 VLANs rather than consolidating to 2** — the bandwidth-only case for 2 VLANs assumed the backup path stays rare and low-volume, which doesn't hold if you actually need it to work at full scale. If a mass fallback event is judged acceptable to handle operationally instead (e.g. running degraded/reduced-quality NDI, or accepting some theatres go dark until Restreamer recovers, rather than provisioning full-bandwidth NDI for all 12 at once), 2 VLANs remains viable — but that's now an explicit choice being made, not a free consequence of the SRT numbers alone.

Splitting further instead (more, cheaper-per-theatre but more numerous VLANs)

Going the other direction (6 or 12 VLANs, 1-2 theatres each) buys essentially nothing on the bandwidth side — even the current 4-VLAN plan runs at 27-39% utilization under normal SRT operation, and a comfortable 45% even under a mass NDI fallback (see above), nowhere near needing more headroom. It also does nothing for the per-router NDI-fallback finding above, since that's a single-router problem, not something more VLANs can fix. The only genuine benefit of more/smaller VLANs is **fault isolation**: a failed venue port or switch takes out fewer theatres. Given the explicit cost pressure on VLAN count, the bandwidth numbers don't support this direction — it's a resilience trade-off to make consciously if at all, not one the traffic model asks for.

The highest-leverage lever: real-time ATEM ingest is the dominant cost

ATEM ISO ingest is ~71% of realistic upstream load per theatre — still the dominant single item, but down from ~92% before either monitoring stream existed (~80% with just Overseer), now that Overseer's and Flock's flat 10.4 Mbps apiece are real contributors, not noise. It exists purely for near-real-time review during the event — see `docs/atem-iso-ingest.md`'s "master vs. mirror" framing: **the ATEM's own SSD remains the authoritative master regardless**, pulled in full after each session either way. If VLAN count/cost turns out to be the binding constraint rather than a one-time design choice, deferring ATEM ingest to end-of-session pulls (instead of continuous near-real-time) drops per-theatre upstream from ~88 Mbps to **~25.8 Mbps realistic / ~35.8 Mbps worst case** (rclone + Overseer's 10.4 Mbps + Flock's 10.4 Mbps, none of which go away — **unlike ATEM ISO, neither monitoring stream is deferrable, both are live feeds**) — at which point **1 VLAN for all 12 theatres sits at ~31% realistic / ~43% worst-case utilization**. Real, workable margin, but a genuinely different number from the "trivially safe ~12%" this lever used to buy before either monitoring stream existed — it's still the single biggest lever available (bigger than any VLAN-topology change), and it's still enough to make 1 VLAN viable where it otherwise now isn't (see the table above), just not quite the near-zero-risk free pass it once was. Losing same-day review access to footage (not the final deliverable) is still the cost. Has nothing to do with the per-router NDI-fallback finding above — that's purely a downstream/NDI issue.

VMix nodes' uplinks

Not yet assigned to a VLAN group in the existing docs. Given the cost pressure, they shouldn't get dedicated VLANs of their own for just 2 nodes — fold each into whichever group its adjacent theatre already belongs to (Node 1 → Theatre 1's group, Node 2 → Theatre 4's group). VMix record-ingest bitrate is unconfirmed (see `docs/vmix-record-ingest.md` open items), but even at a generous assumed 20-30 Mbps/PC (comparable to a single ATEM channel), adding 2 PCs' worth of traffic to a 6-theatre VLAN group (529 Mbps realistic) still leaves room, if less than it used to — worth re-checking against the 2-VLAN group's now-thinner margin above once VMix's real bitrate is confirmed. The same per-router NDI-fallback caveat above would apply to the VMix node's own A-1300 too, if its record ingest and an NDI fallback ever coincide on that same router.

Recommendation

- **Confirm the full-duplex assumption with the venue** before finalizing any count.
- **Set up the per-theatre NDI-fallback mitigation regardless of VLAN count** — pause/ throttle that theatre's ATEM ingest when it falls back to NDI. This isn't optional the way the VLAN-count trade-offs are; without it, one theatre alone can exceed its own A-1300's ~170 Mbps ceiling (128% combined) even on a fully over-provisioned VLAN. It still works with both monitoring streams added, but with less margin doing so than it used to (92% post-mitigation, was 79% before Overseer/Flock existed) — worth being aware this mitigation has less room to absorb a *fifth* new upstream/downstream load if one ever gets proposed.
- **1 VLAN is no longer a real option, full stop — not just a worst-case risk.** Before Overseer and Flock existed, 1 VLAN was viable if ATEM ingest was deferred to end-of-session pulls. It still is (~31% realistic / ~43% worst-case with ATEM deferred), but with real-time ATEM ingest kept, 1 VLAN now exceeds the link even under normal, expected-case assumptions (106%), not just an unlucky worst-case day. If a single VLAN is genuinely the target, real-time ATEM ingest has to be deferred — that's now a requirement rather than the discretionary lever it used to be.
- **Decide how much a mass NDI fallback needs to survive at full quality** — the actual fork in the VLAN-count recommendation, not a footnote:

    - If a Restreamer failure pushing all 12 theatres onto NDI simultaneously must keep working at full 1080p50 quality: **stay at 3-4 VLANs**. 2 VLANs puts that scenario at 90% utilization with no real margin.
    - If that event is acceptable to handle operationally instead (degraded/reduced-quality NDI, or some theatres going dark until Restreamer recovers, rather than full-bandwidth NDI for all 12 at once): **2 VLANs (6 theatres each)** is still viable for normal SRT-primary operation, but weigh this carefully now — halves current VLAN cost, keeps real-time ATEM ingest, but margin has eroded a real amount since this recommendation was first made: 78% worst-case (was 65% before either monitoring stream existed) and now over half the link (53%) even at realistic load, which wasn't true before. This is the trade-off most worth revisiting given everything added since the original 2-VLAN call.
- **Fold both VMix nodes into their adjacent theatre's VLAN group** rather than requesting dedicated VLANs for them, but re-check their real bitrate against the thinner margin above once it's confirmed.
- **If VLAN cost pressure is severe enough to want a single VLAN**, deferring ATEM ISO ingest to end-of-session pulls is now a requirement, not an optional lever — real-time ATEM ingest alone pushes 1 VLAN over capacity at realistic load, before worst case even enters into it. Note this doesn't fix the NDI downstream problem, though — 1 VLAN is not viable for a mass NDI fallback (180%) regardless of what's done on the ATEM side, since that's a downstream, not upstream, load.
- **Resolve the presenter-internet question** (shared with production VLANs or separate venue circuit) before treating any of the above as final — it's folded into the downstream figures above as a conservative shared-infrastructure assumption, and downstream isn't the binding constraint under normal SRT operation, but it should still be confirmed.

Tailscale — DERP relay avoidance

Who's actually on the tailnet

Only **15 nodes** run the Tailscale client: the 14 GL-iNet A-1300 routers (12 theatres + 2 VMix nodes) and the mothership's Tailscale container. Each advertises its own subnet (`--advertise-routes`) and accepts the others' (`--accept-routes`) — see `config/tailscale-up-all-devices.sh` .

No individual client device runs Tailscale. Not the PowerPoint/VT laptops, not the control laptops, not the ATEM, not BirdDog Play/Central, not the VMix PCs or P400 cameras. They all reach the rest of the network purely through their GL-iNet's normal LAN gateway — subnet routing makes `192.168.1.0/24` (and every other theatre's subnet, where ACLs allow it) reachable transparently, with zero Tailscale software installed on any of them.

Why direct connections are likely

Every node ultimately sits behind one shared mothership internet connection — a favorable case for direct (non-relayed) connections, since Tailscale can either use the true private LAN path or hairpin through the single shared public IP.

What determines it in practice:

1. **Whether the Ubiquiti firewall allows UDP between VLANs/subnets.** If yes → direct private-path connections, no internet round-trip.
2. **If inter-VLAN UDP is blocked**, Tailscale falls back to NAT hairpin via the shared public IP — usually works, but more fragile with GL-iNet's own NAT stacked underneath (double-NAT).
3. **DERP relay** is only the fallback when both of the above fail. Tailscale continuously retries and upgrades to direct when possible.

Verifying after setup

```
tailscale status        # shows direct vs relay per peer
tailscale ping <peer>
```

Self-hosted DERP server (decided — running on the mothership)

Runs as a Docker container on the consolidated Unraid server — `derp-server` , `192.168.1.16` (see `docs/ip-address-map.md`) — using Tailscale's own `derper` binary/image (official docs).

Why this still needs a real, reachable hostname. DERP isn't reached over an already-established Tailscale tunnel to the peer that needs relaying — a node holds a persistent connection to its DERP server over its own normal WAN path, independent of any specific peer, precisely so it can bootstrap connectivity when direct WireGuard isn't available yet. That means the venue's shared-internet-connection hairpin trick (see above) is exactly what makes this work here: every theatre's GL-iNet reaches the DERP server's hostname over its *own*

WAN path, which loops back through the same shared public IP without the traffic ever actually leaving the venue's router. This requires:

- A real DNS hostname pointed at the mothership's public IP (a domain you control, or a dynamic-DNS hostname if you don't have one). Named **`derp.example.net`** — needs to actually be registered/pointed (or swapped for whatever domain/DDNS host you do control) before it resolves to anything.
- A port forward on the Cloud Gateway: WAN `443/tcp` → `192.168.1.16:443`, and WAN `3478/udp` → `192.168.1.16:3478` (STUN). See `config/unifi/network-config.yaml`. **443** for the DERP listener — matches Tailscale's own DERP fleet and is the most firewall-friendly port (many restrictive networks allow outbound 443 and nothing else), though it doesn't matter much for reachability here specifically since it's an internal hairpin, not roaming clients behind a hotel firewall.
- Let's Encrypt issues the cert automatically against that hostname (`-certmode=letsencrypt`) since the port forward makes it genuinely reachable for the ACME challenge.

```
docker run -d --name derp-server \
  --network macvlan-mothership --ip 192.168.1.16 \
  -v /mnt/user/appdata/derper/certs:/app/certs \
  -v /var/run/tailscale/tailscaled.sock:/var/run/tailscale/tailscaled.sock \
  tailscale/derper \
  -hostname=derp.example.net \
  -certmode=letsencrypt -certdir=/app/certs \
  -stun -stun-port=3478 \
  -verify-clients
```

(Attached to the `macvlan-mothership` network at `192.168.1.16` — the same macvlan setup every other mothership container uses, so no `-p` port publishes, which are inert under macvlan anyway. The canonical definition of this service lives in `config/docker-compose.yml`; this standalone command is just the same thing spelled out.)

`-verify-clients` restricts relay use to nodes already on this tailnet — needs the container to reach the host's `tailscaled`, which is exactly what the `tailscaled.sock` mount above is for. Recommended so the DERP server isn't an open relay; drop both the flag and the mount if that's not a concern.

Register it with the tailnet by adding a custom region to the ACL policy — see the `derpMap` block in `config/tailscale-acl.json`. Kept `OmitDefaultRegions: false` (the safer default) — Tailscale prefers this DERP node when it's faster, but still falls back to the public regions if the self-hosted one goes down, rather than losing relay fallback entirely.

Verify with `tailscale netcheck` — it reports which DERP region a node prefers.

Config sketch

See `config/tailscale-up-commands.sh` for the per-role `tailscale up` invocations and `config/tailscale-acl.json` for the ACL policy.

Routes still need approving in the Tailscale admin console (or via `autoApprovers` keyed to tag/CIDR) before they take effect — advertising a route alone is not sufficient.

Open questions

Resolved

- **Subnet numbering.** Spec said 12 theatres on "192.168.2-12.x", which is only 11 subnets. Confirmed mapping: mothership on `.1.x`, Theatre 1 → `.2.x` … Theatre 12 → `.13.x` (contiguous after mothership). Used throughout this repo.
- **Mothership router: Ubiquiti Cloud Gateway.** Cloud Gateway (UniFi-OS-based, same family as the Dream Machine) generally can't run Tailscale natively without an unofficial container hack. Decision: the mothership subnet-router role runs as a container on the consolidated services server instead of the Cloud Gateway itself — see `docs/topology.md`.
- **Mothership service consolidation: single physical server running Unraid.** Nextcloud, Restreamer, the VMix instance, BirdDog Central, and the NDI Discovery Server all move onto one box instead of being separate machines — the router and the 4 ready-room PCs stay physical. VMix and BirdDog Central are both Windows-only, so each runs as its own Windows VM (kept separate so one hanging doesn't risk the other); only the VMix VM needs GPU passthrough (hardware encode) — BirdDog Central is routing/control, not encode. Everything else (Nextcloud, Restreamer, NDI Discovery Server, Tailscale subnet router) runs as Docker containers. Chose **Unraid over TrueNAS SCALE** for its more mature GPU-passthrough track record and more polished one-click container experience, accepting the $129 one-time Lifetime license cost that TrueNAS (free) doesn't have. See `docs/topology.md` for the full breakdown.
- **Theatre router: GL-iNet A-1300 (Slate Plus).** Published client-mode WireGuard throughput ≈ 170 Mbps (GL.iNet A-1300 datasheet). Each theatre's router only ever carries *its own* theatre's traffic — one incoming SRT feed to that theatre's BirdDog Play plus up to 4 laptops' rclone sync — not the aggregate across all 12 theatres (that 12x aggregate converges at the mothership's WAN link and Restreamer, not at any single theatre router). So the real per-router budget is roughly 1× SRT bitrate (~8–50 Mbps depending on resolution/codec) + bursty rclone traffic from up to 4 laptops — comfortably inside the A-1300's 170 Mbps headroom. *(Caveat: that traffic list predates ATEM ISO ingest and the Overseer/Flock monitoring streams, all of which now ride the same router. The current per-router model — 97% of the ceiling at worst-case normal load, 128% during an NDI fallback with ingest running — is in `docs/bandwidth-analysis.md`'s "check the A-1300's own ceiling" section; the router choice still stands, but "comfortably" no longer describes the margin.)*
- **VMix node routers: also GL-iNet A-1300.** Same model as the 12 theatre routers — 14 A-1300s total across the network. Both LAN ports bridge together (no dedicated-port split needed here, unlike the theatre routers' BirdDog Play case). Configs generated: `config/gl-inet/vmix-node-1.uci` and `vmix-node-2.uci`.
- **NDI backup path discovery.** Confirmed Tailscale can't help directly — it's point-to-point WireGuard and doesn't forward multicast/mDNS (a still-open Tailscale feature request, not something MagicDNS works around). Plan: run an **NDI Discovery Server** on the mothership (unicast TCP, default port 5959 — built into NDI 5+ specifically for WAN/VPN cases like this), and point BirdDog Central plus all 12 BirdDog Play units at it. See `docs/streaming-flow.md` for the full plan.

- **BirdDog Play Discovery Server support — confirmed.** BirdUI (PLAY's web admin) has a Network panel with an NDI Discovery Server toggle: switch it on, enter a comma-delimited list of server IP(s), apply. BirdDog's own docs describe this as the mechanism for finding sources "on different subnets." Sources: BirdDog PLAY User Guide, BirdUI User Guide. Default admin password (`birddog`) should be changed on all 12 units before the event.

- **Consolidated server: hardware already on hand.** Physical box already exists — no procurement needed. Known so far: 2× gigabit NICs, a "decent" discrete GPU. Full spec (CPU/motherboard, RAM, exact GPU model) to be confirmed later — a calculated **target** minimum spec to check it against (storage pool layout/sizing, RAM, CPU, GPU tier, NIC validation, all worked from this box's actual workload rather than guessed) is now in `docs/server-specification.md`.

- **Full device inventory + configs generated.** Every device on the network now has a concrete IP — see `docs/ip-address-map.md` (134 devices total) and the redrawn `diagrams/topology.svg`. Configs generated to match: `config/gl-inet/` (UCI network config for all 14 routers — 12 theatres + 2 VMix nodes), `config/tailscale-up-all-devices.sh` (all 15 tailnet nodes instantiated), and `config/unifi/` (Cloud Gateway VLANs/DHCP/firewall reference).

  Two things introduced *during config generation* that weren't decided before — flagged here since they're new, not previously confirmed:

  - **Uplink VLAN transit addressing** (`10.10.1-4.0/24`, VLAN tags 101–104) for the 4 physical VLAN groups' GL-iNet WAN ports. Doesn't affect anything inside the theatres (still NAT'd behind each A-1300) or Tailscale (rides over whatever WAN IP is handed out) — purely a Cloud Gateway-side detail. Change freely.
  - **Docker networking mode per mothership service**: Nextcloud/Restreamer/NDI Discovery Server/DERP server assigned dedicated IPs via macvlan; Tailscale subnet router uses host networking (required for route advertisement, shares the Unraid host's `.2`). Both are standard Unraid patterns, not unusual choices, but worth confirming once you're actually configuring Docker on the box.

- **2 gigabit NICs: bonded (802.3ad/LACP), not split.** SRT bandwidth isn't constant, and this box handles many simultaneous flows (12 theatres' SRT fan-out to distinct destinations, plus rclone/Nextcloud/BirdDog Central/NDI Discovery Server/DERP traffic) — LACP's hash-based distribution spreads those across both links for real aggregate headroom, rather than a static split that leaves one NIC idle whenever traffic doesn't match the assumed pattern. See `docs/topology.md` for the full reasoning, the Unraid setup steps, and the LACP rate/hash-policy gotcha with Ubiquiti gear.

- **Self-hosted DERP server added.** Runs as a Docker container on the Unraid box (`192.168.1.16`), registered with the tailnet as DERP region 900, `RegionCode: "example"`, hostname `derp.example.net`, port **443** (matches Tailscale's own DERP fleet, most firewall-friendly choice) — see `config/tailscale-acl.json`. Kept `OmitDefaultRegions: false` so Tailscale's public DERP regions remain available as a fallback if this one goes down. See `docs/tailscale.md` for the full setup (container flags, port forward, cert handling).

- **Cloud Gateway assumed to support LAG.** Proceeding on that assumption rather than confirming the exact model first. Fallback already agreed if it turns out not to: drop an intermediate LACP-capable switch in between and bond the Unraid server's NICs through that instead of directly into the Cloud Gateway — the bond itself doesn't care which device terminates it, only that *something* in the path speaks LACP.

- **ATEM ISO ingest architecture decided, two documented approaches.** Each ATEM's built-in FTP server (confirmed live-accessible during recording) is pulled continuously by a new Unraid container (`192.168.1.17`) over Tailscale subnet routing. Plain rsync can't connect to the ATEM at all (no SSH/rsync daemon), and a naive whole-file re-sync is ruled out by the bandwidth math for any real session length — so genuinely incremental transfer is required, achieved either via **FTP's `REST` /resume command directly** (`pull-iso.py`, the default — fewer moving parts) or via **`rclone mount` + `rsync --append`** (`rclone-mount-rsync/` — off-the-shelf tools, at the cost of 12 FUSE mounts to supervise). Both write directly into Nextcloud's External Storage (Local) mount, so there's no separate upload step and no need to slice/chunk the video file itself either way. Full comparison in `docs/atem-iso-ingest.md`.

- **VMix record ingest added, same mechanism as the ATEM ingest.** All 4 VMix PCs' recordings pulled by a new Unraid container (`192.168.1.18`) via `rclone mount` + `rsync --append`, targeting each PC's Windows SMB share instead of an FTP server. SMB is a native network filesystem protocol, so this is arguably a better fit for a mount-based approach than the ATEM's FTP-only case was, not a stretch of the same pattern. Each VMix node has its own dedicated A-1300 uplink (not shared with the theatre it sits near), so this traffic never competes with that theatre's ATEM ingest or SRT feed. Full design in `docs/vmix-record-ingest.md`, tooling in `config/vmix-record-ingest/`.

- **Self-hosted UniFi Controller added to the consolidated server.** A UniFi Network Application container (`192.168.1.19`) joins the rest of the Docker stack, same self-hosted-over-cloud.ui.com rationale already used for DERP. The Cloud Gateway has its own built-in controller and doesn't require this — adopting it in is optional, purely a local/independent management console, not a change to any traffic path in `config/unifi/network-config.yaml`. See `config/unifi/README.md`.

- **Self-hosted GLKVM-Cloud added for centralized administration of the 14 GL-iNet routers.** No physical KVM hardware involved — this uses GLKVM-Cloud's separately documented HTTP/HTTPS web-proxy and device-management support for embedded OpenWrt devices (which the A-1300s are), giving one browser-based SSH terminal + web-admin proxy for all 14 routers instead of 14 separate sessions. Same self-hosted-over-vendor-cloud rationale as DERP and the UniFi Controller, avoiding GL.iNet's own `glkvm.com`. Two containers (`rttys` on `192.168.1.20`, `coturn` on `192.168.1.22`); router registration rides the existing tailnet (routers already accept routes to `192.168.1.0/24`), no WAN exposure needed for that part. Surfaced a real conflict during design: GLKVM-Cloud's documented ports (`443/tcp`, `3478/tcp+udp`) collide with the DERP server's existing WAN forwards on those same numbers — resolved by forwarding different WAN-side port numbers (`8443`, `3479`) to GLKVM-Cloud's unchanged internal ports (for the admin's own browser access), rather than moving DERP off 443 and losing its firewall-friendliness rationale. Not yet confirmed against a real deployment: whether `GLKVM_ACCESS_IP` (the env var that tells the app what address to hand back to clients) accepts this WAN-side remap cleanly, or whether `coturn` needs its own separate external-address setting — also unclear whether `coturn`/TURN is needed at all for the SSH-terminal/web-proxy features actually in use here, versus only for GLKVM-Cloud's KVM-specific remote-desktop feature, which doesn't apply to routers. Full design in `docs/glkvm-cloud.md`, stack in `config/glkvm-cloud/`.

- **Bandwidth modeled against venue-supplied VLANs — 2 VLANs (not 4) recommended, but reconsider given how thin that margin has gotten.** The 4 uplink VLANs turned out to be venue-supplied, 1 Gbps each, with expensive ports — changes the goal from "isolate cleanly" to "minimize VLAN count at adequate performance." Real numbers (last updated after adding the Flock BirdDog Play preview stream, ~10.4 Mbps/theatre, on top of ATEM Overseer's — see below): per-theatre upstream is dominated by ATEM ISO ingest (~62 Mbps realistic of ~88 Mbps total) — at the current 3-theatres-per-VLAN split, that's ~27% utilization, with room to consolidate to 2 VLANs (6 theatres each, ~53% realistic / 78% worst-case) before it stops being comfortable; **1 VLAN for all 12 theatres now exceeds capacity even at realistic load**

(106%), not just worst case — it's only viable at all if ATEM ingest is deferred to end-of-session pulls instead of real-time. Full model in `docs/bandwidth-analysis.md`.

- **ATEM Overseer + ATEM Fleet Admin added to the mothership.** Two of the user's own separate projects, deployed as Docker containers (`192.168.1.21` and `192.168.1.23`) alongside everything else — Overseer for fleet-wide monitoring/tally (every theatre's ATEM sends it a dedicated 10 Mbps monitoring stream, folded into the bandwidth model above as a flat per-theatre addition that doesn't scale with ATEM channel count), Fleet Admin for bulk provisioning (reaching all 12 theatres' ATEMs the same way ATEM ISO Ingest does, over the existing Tailscale subnet routing). Neither is built from source in this repo — `config/docker-compose.yml` uses placeholder image references pending confirmation of each project's actual published container image.

- **Flock added to the mothership.** A third of the user's own projects, deployed alongside the other two (`192.168.1.24`) — the BirdDog Play fleet manager already referenced elsewhere in this design (LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits — see `docs/birddog-play-rationale.md`). Each theatre's BirdDog Play sends Flock a 10 Mbps SRT preview stream (confirmed by the user, not assumed), folded into the bandwidth model the same way as the Overseer stream — another flat per-theatre addition. Same not-built-from-source caveat as Overseer/Fleet Admin above.

- **Live editing subsystem added — new `192.168.22.0/24` subnet, own 10GbE LAN, not on the Tailscale mesh.** 2× MacBook Pro editing workstations plus dedicated fast storage at the mothership, editing event content as it arrives rather than after the event. Deliberately separate from the theatre-facing network — different traffic pattern (sustained multi-gigabit editing I/O vs. the bursty low-bitrate streams everything else here is sized for) and a genuinely different failure domain. Key decisions made: dedicated NAS (not Nextcloud's own storage), dual-write from the same rclone/rsync pull already built for ingest rather than a chained re-sync; a dedicated Mac mini running both the Resolve Project Server and Remote Render (needed because resolve-configurator builds one shared show project both editors work against, not independent copies — confirmed **not** possible on a Blackmagic Cloud Store appliance itself, storage-only, no general compute); Blackmagic Cloud's internet sync service not needed for a single-venue setup. Full design, including the ATEM-pull-to-finished-edit workflow and the DIT physical-media ingest tool comparison (ShotPut Pro/OffShoot vs. Silverstack vs. FoolCat/o/PARASHOOT as companion tools), in `docs/live-editing.md`.

Still open — verify before building

1. **Check the existing server's real spec against the calculated target in `docs/server-specification.md`.** In particular: does the CPU/motherboard support IOMMU (Intel VT-d or AMD-Vi) for GPU passthrough, is the GPU model VFIO-passthrough-compatible and which vMix camera-count tier does it actually meet, and is there enough RAM/cores (target: 16 cores/32 threads, 96-128 GB RAM) to run 2 Windows VMs (VMix + BirdDog Central) plus 12 Docker containers concurrently without contention during a live event.

2. **Actually register/point `derp.example.net`** (or substitute whatever domain/DDNS host you end up controlling) at the mothership's public IP, and confirm the WAN port forward (443/tcp, 3478/udp) is in place before relying on it — the name and port are picked, but nothing resolves until the DNS record and port forward both exist.

3. **Verify the DERP hairpin path doesn't get caught by the uplink-VLAN firewall block.** `config/unifi/network-config.yaml` blocks the 4 uplink VLANs from reaching Mothership-LAN directly (defense-in-depth for cross-theatre isolation) — that should be a different path from the DERP hairpin (which arrives via the WAN zone after NAT, not directly from an Uplink-VLAN zone), but exact zone/hairpin

behavior is Cloud-Gateway-firmware-specific. Confirm with `tailscale netcheck` on a theatre router once built, before assuming the self-hosted DERP server is actually reachable.

4. **Confirm the real ATEM ISO bitrate/active-channel count, and empirically verify partial-file playability.** The bandwidth plan above uses 10 Mbps/stream and an assumed 4-6 active channels — worth checking against actual ATEM recording settings. Whether a file copied mid-recording is genuinely valid (rather than just "very likely, given Blackmagic's marketed edit-while-recording capability") should be tested against a real unit before relying on it operationally. See `docs/atem-iso-ingest.md` for the full open-items list, including confirming the ATEM's actual FTP credentials/file layout and tuning the pull/scan intervals and Nextcloud version-retention settings once this is running for real.

5. **Confirm what VMix is actually recording, and set up real SMB shares/credentials.** Neither the recording mode (program mix vs. per-input ISO) nor bitrate/codec, nor the actual share name/folder path/account on the 4 VMix PCs, is assumed here — see `docs/vmix-record-ingest.md` for the full open-items list.

6. **Confirm with the venue: final VLAN count, the full-duplex-1Gbps assumption, and whether presenter internet shares the production VLANs or is separate infrastructure.** `docs/bandwidth-analysis.md` recommends 2 VLANs over the current 4, but that's a real cost/procurement decision, not something this repo can finalize alone. If real-time ATEM ingest turns out incompatible with the venue's final VLAN allocation, deferring it to end-of-session pulls is the fallback — same doc.

7. **Confirm `GLKVM_ACCESS_IP`'s exact accepted format, and whether `coturn` needs its own separate external-address setting**, before trusting the WAN-remapped port forwards (`8443` , `3479`) to actually work end-to-end for remote GLKVM-Cloud admin access. See `docs/glkvm-cloud.md` for the fallback (second public IP or an SNI-routing reverse proxy) if the plain env var doesn't cover it. Also confirm whether `coturn` is needed at all for the SSH-terminal/web-proxy features actually in use here — it may only matter for GLKVM-Cloud's KVM-specific remote-desktop feature, unused since there's no KVM hardware in this design.

8. **New assumptions introduced by `config/docker-compose.yml`**, none confirmed against a real deployment: the NDI Discovery Server image (`pnxr/ndi-discovery-minimal` , a third-party community build, not NDI/NewTek-published — confirm it's still maintained), the MongoDB version the UniFi Controller needs (drifts with the controller image's own version, not fixed here), and MariaDB + Redis as Nextcloud's DB/cache backend (chosen over SQLite — this box has a lot of concurrent writers across 12 theatres' rclone syncs plus both ingest pipelines, which official Nextcloud guidance says SQLite handles poorly; not benchmarked here either way).

9. **Confirm actual event duration (day count × active-recording hours/day)** before treating the 8 TB recording pool in `docs/server-specification.md` as settled — that document calculates it covers roughly 15.5-23.5 hours of continuous worst-to-realistic-case load (~1.5-3 typical event days), but this repo doesn't establish the event's real length anywhere. If it runs longer without a periodic archive-off step, either the pool needs to grow or an offload step needs adding to the ingest design.

10. **Confirm what the mothership's own VMix instance VM actually does** — how many camera/input channels, what resolution — before treating the GPU tier in `docs/server-specification.md` as settled. The 2 VMix *nodes* (physical PCs at the theatres) are well-documented (`docs/topology.md`); this separate VM's role isn't.

11. **Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's actual published container images** (or that they need to be built from source instead) before deploying `config/docker-compose.yml` — all three

use placeholder `ghcr.io/...` references. Also confirm the actual mechanism behind both monitoring streams `docs/bandwidth-analysis.md` takes as given inputs, not verified against real hardware: Overseer's 10 Mbps ATEM stream (does the ATEM Mini Extreme ISO genuinely support two independent simultaneous stream outputs — its own hardware streaming engine feeding both Restreamer and Overseer at once?), and Flock's 10 Mbps BirdDog Play preview stream (same question for BirdDog Play — decoding its primary program feed while simultaneously *originating* a second SRT stream back to Flock is a different capability than the receive-only role it plays everywhere else in this design).

12. **Decide how much of the event's footage the live-editing subsystem actually needs access to** — the full 8TB recording pool, or a curated subset — before sizing the edit-suite NAS in `docs/live-editing.md` Decision 1.

13. **Confirm combining Project Server and Remote Render on one Mac mini holds up under real load** — `docs/live-editing.md` Decision 2 treats this as architecturally sound but thinly documented by both Blackmagic and practitioners; worth a real pre-event test, not just a design-time assumption.

14. **Confirm whether this event uses any standalone cameras with SD/CFexpress cards** beyond the NDI-fed BirdDog P400s — affects whether the DIT physical-media ingest workflow in `docs/live-editing.md` has real camera cards to process beyond the ATEM's own USB SSD, and whether Silverstack's deeper metadata/lens-data feature set becomes worth its cost over ShotPut Pro/OffShoot.

15. **Implement the second write destination in the actual ingest scripts.** `docs/live-editing.md` documents `pull-iso.py` and `mount-and-sync.sh` writing to the edit-suite NAS alongside Nextcloud — that's a real code change neither script has yet (`config/atem-iso-ingest/pull-iso.py`, `config/vmix-record-ingest/mount-and-sync.sh` currently write one destination each). Not done as part of documenting the design.

Alternative: plain switches + Tailscale on generic Linux, no GL-iNet routers

Same theatre topology (isolated `/24` per theatre, subnet routing, Tailscale ACLs doing cross-theatre isolation) — different hardware. Instead of a GL-iNet A-1300 doing router+NAT+DHCP+Tailscale in one appliance, use a plain unmanaged switch for LAN fanout plus a small generic Linux box (mini-PC/NUC, or the theatre's existing control laptop) running Tailscale as a subnet router with NAT/DHCP built from standard Linux tools (nftables + dnsmasq). Config in `config/alt-tailscale-switch/`.

Why a router-equivalent device is still required, not eliminated: the ATEM and BirdDog Play are closed appliances — they can't run the Tailscale client themselves. Some device must still advertise the theatre's subnet on their behalf. "Pure Tailscale, no router" only works if literally every device can run Tailscale; since two can't, this alternative just moves the subnet-router role onto generic Linux instead of GL-iNet's firmware. The `tailscale up` invocations themselves are identical either way — see `config/tailscale-up-all-devices.sh` — Tailscale doesn't care what hardware runs it.

Comparison

	GL-iNet A-1300 (chosen)	Generic Linux box + switch (alternative)
Setup effort	Config file, done (see `config/gl-inet/`)	Build + harden NAT/DHCP/Tailscale from scratch per box
Throughput confidence	Vendor datasheet, ~170 Mbps WireGuard	Unverified until benchmarked on whatever's chosen
Physical footprint	One compact unit per theatre	Separate PC + separate switch + 2 power supplies
Dedicated ATEM port	Built-in (2 LAN ports)	Needs a 2nd NIC/dongle to replicate
Wi-Fi	Built-in	Extra hardware if needed at all
Spares/swap-in	Re-flash identical UCI config	Re-provision a whole OS image
Maintenance	Vendor firmware updates	Ongoing OS patching, our responsibility
Flexibility/CPU headroom	Fixed, embedded-class	Higher, if it matters later
Unit cost at required throughput	~$100-class travel router	Often more, once a real NIC + case + PSU are added

Recommendation: GL-iNet

This is 12 identical small router/switch appliances for a touring event that need to just work — exactly the product category a travel router is built for. A generic Linux box gives more flexibility and CPU headroom we don't need here, at the cost of more setup work, more failure modes, no vendor throughput guarantee, and a bulkier kit per theatre. The one real advantage of the alternative — no vendor lock-in — doesn't outweigh those costs for this deployment. Keep GL-iNet as the primary; this alternative is documented for completeness, not as an equally-weighted option.

Deployment runbook

Ordered build sequence synthesizing every doc/config in this repo into one checklist. Each step links to the doc with the actual detail — this is a map, not a duplicate.

Phase 0 — Before touching anything

- [ ] Confirm the consolidated server's full spec (CPU/motherboard IOMMU support, GPU passthrough compatibility, RAM/cores) — `docs/open-questions.md` #1
- [ ] Confirm the Cloud Gateway model (LAG support assumed, not confirmed) — `docs/open-questions.md` resolved-items section
- [ ] Decide a real hostname/DDNS for the DERP server and confirm you can port-forward on the Cloud Gateway — `docs/tailscale.md`
- [ ] Confirm VMix's actual recording mode/bitrate and get SMB sharing set up on all 4 VMix PCs — `docs/vmix-record-ingest.md`
- [ ] Confirm ATEM FTP credentials and recording path on a real unit — `docs/atem-iso-ingest.md`
- [ ] Decide a real hostname/DDNS for GLKVM-Cloud and confirm `GLKVM_ACCESS_IP`'s exact format for the WAN-remapped ports — `docs/glkvm-cloud.md`
- [ ] Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's real container images, and whether the ATEM and BirdDog Play can actually originate their monitoring/preview streams alongside their primary feeds — `docs/open-questions.md` #11
- [ ] Settle the live-editing open questions — footage-volume sizing, the Mac mini's combined Project Server + Remote Render role, standalone cameras in scope or not — `docs/open-questions.md` #12-14

Phase 1 — Mothership networking (Cloud Gateway)

1. Apply `config/unifi/network-config.yaml` via the UniFi Network UI/API — 4 uplink VLANs, firewall baseline, LAG port profile, DERP + GLKVM-Cloud port forwards. Steps in `config/unifi/README.md`.
2. Wire the 4 VLAN uplink cable runs (3 theatres per group) — `docs/topology.md`.
3. Bond the consolidated server's 2 NICs (802.3ad/LACP) into the Cloud Gateway (or an intermediate switch if LAG isn't supported) — `docs/topology.md`, watch the LACP rate/hash-policy gotcha with Ubiquiti gear.

Phase 2 — Consolidated services server (Unraid)

1. Install Unraid, confirm IOMMU/passthrough works for the GPU.
2. Create the VMix instance and BirdDog Central Windows VMs (separate VMs; only VMix gets GPU passthrough) — `docs/topology.md`.
3. Bring up every Docker container in one shot with `config/docker-compose.yml` — Nextcloud, Restreamer, NDI Discovery Server, DERP, ATEM ISO Ingest, VMix Record Ingest, UniFi Controller, GLKVM-Cloud (`rttys` + `coturn`), ATEM Overseer, ATEM Fleet Admin, Flock, Tailscale subnet router. See `config/README.md` for the `.env` setup and prerequisites first — DERP and GLKVM-Cloud specifically need their hostnames/ port-forwards from Phase 0/1 in place before they're actually useful, and ATEM Overseer/Fleet Admin/Flock need their real container images confirmed first (see `docs/open-questions.md` #11) — DERP and GLKVM-

Cloud will start but be non-functional without their hostnames/forwards, and the three placeholder-image services won't start at all until real images are filled in.
4. Optionally adopt the Cloud Gateway into the UniFi Controller for local management — `config/unifi/README.md`. Purely a local console; the network config in Phase 1 doesn't depend on it.
5. Router registration into GLKVM-Cloud happens later, in Phase 4, once the routers themselves exist.

Phase 3 — Tailscale tailnet

1. Load `config/tailscale-acl.json` into the tailnet admin console — cross-theatre isolation ACLs + the self-hosted DERP region.
2. Run the mothership's `tailscale up` (see `config/tailscale-up-all-devices.sh`) from the Tailscale subnet-router container.
3. Approve routes in the admin console (or configure `autoApprovers`) — nothing routes until approved, regardless of what's advertised.

Phase 4 — Theatre + VMix node routers (GL-iNet A-1300 ×14)

1. Flash/reset all 14 units, apply Wi-Fi lockdown per venue policy (not covered by the generated configs).
2. Apply each theatre's UCI config from `config/gl-inet/` — replace every `REPLACE_WITH_MAC_nn` with real device MACs first (config won't do anything useful otherwise). Steps in `config/gl-inet/README.md`.
3. Run each router's `tailscale up` line from `config/tailscale-up-all-devices.sh`.
4. Confirm the physical wiring matches the current plan: ATEM on the dedicated LAN 1 port, everything else (including BirdDog Play) via the Netgear switch on LAN 2 — `docs/topology.md`.
5. Register each router against GLKVM-Cloud (brought up in Phase 2) using the connection script from its web UI, run over SSH on each A-1300 — rides the tailnet already set up in step 3, no WAN exposure needed for this part — `docs/glkvm-cloud.md`.

Phase 5 — BirdDog Play + NDI discovery

1. Change the default `birddog` admin password on all 12 PLAY units.
2. Point BirdDog Central and all 12 PLAY units at the NDI Discovery Server (`192.168.1.15:5959`) via BirdUI's Network panel — `docs/streaming-flow.md`.

Phase 6 — Nextcloud external storage + ingest pipelines

1. Run `config/atem-iso-ingest/setup-nextcloud-external-storage.sh` and `config/vmix-record-ingest/setup-nextcloud-external-storage.sh` — mounts the External Storage folders and prints the cron lines for targeted rescans. Add those cron entries.
2. Fill in real credentials and start `pull-iso.py` (or the `rclone-mount-rsync/` alternative) — `config/atem-iso-ingest/README.md`.
3. Fill in real SMB credentials/share names and start `mount-and-sync.sh` — `config/vmix-record-ingest/README.md`.
4. Configure Nextcloud version-retention for both ingest folders — both ingest docs flag this as worth tuning once running for real.
5. **Dual-write caveat**: the second write destination (the edit-suite NAS, see `docs/live-editing.md`) is documented but **not yet implemented** in either ingest script — implement it or consciously defer it before

the event (`docs/open-questions.md` #15). As shipped, both scripts write only the Nextcloud destination.

Phase 6b — Live editing subsystem

The edit suite (`docs/live-editing.md`) is its own 10GbE LAN, off the tailnet, so it can be built any time after Phase 2 — it only depends on the ingest pipelines (Phase 6) for its footage feed:

1. Stand up the `192.168.22.0/24` 10GbE LAN — switch, edit-suite NAS (`.22.2`), routed connection back to the mothership LAN for the dual-write (no Tailscale).
2. Set up the Mac mini (`.22.20`): install DaVinci Resolve Studio + the Project Server app, create the shared project library, enable it as a Remote Render node.
3. Connect both MacBook Pros (`.22.11` / `.22.12`) via Thunderbolt-to-10GbE, mount the NAS on all three machines with identical paths (Resolve's Remote Render requires the media volume mounted on every machine).
4. Run resolve-configurator against the event's session CSV to build the shell project; apply each smart-bin recipe once, by hand, in Resolve's UI.

Phase 7 — Verify before the event

- [ ] `tailscale status` / `tailscale ping` from a few nodes — confirm direct vs relay — `docs/tailscale.md`
- [ ] `tailscale netcheck` on a theatre router — confirm the self-hosted DERP region is reachable and not blocked by the uplink-VLAN firewall rule — `docs/open-questions.md` #3
- [ ] Copy an ATEM file mid-recording and confirm it opens/scrubs — empirically verify the "safe to read mid-write" assumption — `docs/atem-iso-ingest.md`
- [ ] Confirm ingested files actually appear in Nextcloud's UI (not just on disk) for both pipelines
- [ ] Confirm cross-theatre isolation: from one theatre's subnet, verify you cannot reach another theatre's devices, only the mothership — this is the core ACL guarantee the whole design depends on (`config/tailscale-acl.json`)
- [ ] From off-venue (a network that isn't the mothership's own), confirm the GLKVM-Cloud web UI is actually reachable through `https://kvm.example.net:8443`, and that all 14 routers show up registered and their SSH terminal/web-proxy access works — `docs/glkvm-cloud.md`
- [ ] Confirm all 12 theatres' monitoring/preview streams actually reach ATEM Overseer and Flock and show up live in each dashboard — this is the first real-hardware test of the dual-stream assumption flagged in `docs/open-questions.md` #11
- [ ] Live editing: confirm footage lands on the edit-suite NAS as it's ingested (once the dual-write is implemented — see Phase 6 step 5), both editors can open the shared show project simultaneously, the smart bins auto-fill as clips arrive, and a test export dispatched to the Mac mini via Remote Render completes — `docs/live-editing.md`

Troubleshooting

Known gotchas surfaced during design — check here before assuming something's broken.

"Theatre can't reach the mothership at all"

- **Routes not approved.** Advertising a route with `--advertise-routes` isn't enough — it must be approved in the Tailscale admin console (or via `autoApprovers`). Check first; this is the single most common reason a fresh subnet router doesn't work.
- **Check `tailscale status`** on the theatre's A-1300 — confirms whether the peer connection is up at all before debugging further.

"Two theatres can talk to each other" (should be impossible)

- This should never happen if `config/tailscale-acl.json` is loaded correctly — theatres can only reach `tag:mothership`. Confirm the ACL is actually applied in the admin console, not just present in this repo.
- Remember: physical VLAN grouping provides **zero** isolation on its own — it's cabling organization only. If ACLs are missing/misconfigured, cross-theatre reachability is the default, not a bug in the VLANs.

"Tailscale is using DERP relay instead of direct"

- Expected sometimes — see `docs/tailscale.md` for why direct connections are likely but not guaranteed here (depends on whether the Cloud Gateway allows inter-VLAN UDP).
- Run `tailscale netcheck` to see which DERP region is preferred. If it's not the self-hosted `example` region, check: is `derp.example.net` actually resolving? Is the WAN port forward (443/tcp, 3478/udp) actually in place? Neither works until both exist — see `docs/open-questions.md`.
- If the self-hosted DERP is reachable from the mothership's own LAN but not from a theatre, suspect the uplink-VLAN firewall block in `config/unifi/network-config.yaml` catching the hairpin path — see the caveat attached to that rule.

"The NIC bond won't form / only shows one active link"

- **LACP rate mismatch.** Ubiquiti gear hardcodes LACP rate `fast`; Unraid's bonding default is `slow`. Both ends must match — see `docs/topology.md`.
- **Cloud Gateway doesn't support LAG at all** on base (non-Pro/SE/Pro Max) models. If so, bond through an intermediate LACP-capable switch instead — already the agreed fallback, not a new problem.

"BirdDog Play isn't finding the NDI backup source"

- Confirm the Discovery Server IP is actually entered in BirdUI's Network panel on *both* BirdDog Central and every PLAY unit — one-sided config does nothing.

- Plain mDNS will never work across theatres; if Discovery Server isn't configured, this is expected behavior, not a fault. See `docs/streaming-flow.md`.

"Ingested files (ATEM or VMix) aren't showing up in Nextcloud"

- Writing to the folder isn't enough — Nextcloud needs `occ files:scan` to index it. Check the cron job from `setup-nextcloud-external-storage.sh` is actually installed and running, not just printed once during setup.
- If files appear but seem stuck at an old size, check the ingest container's logs — a failed FTP/SMB reconnect will silently stall the pull loop rather than crash it.

"Ingest is falling behind / theatre uplink saturated"

- Confirm the real ATEM bitrate/active-channel count and VMix recording settings — the bandwidth math in `docs/atem-iso-ingest.md` and `docs/vmix-record-ingest.md` assumes figures that were never verified against real hardware.
- If using the `rclone-mount-rsync` alternative, confirm `rsync --append` is actually being used (not falling back to a full re-copy) — check `--itemize-changes` output.

"A copied ATEM/VMix file won't play"

- Confirmed only "very likely" safe to read mid-write, never empirically tested — see the open item in both ingest docs. If it fails, the safest fallback is to only pull once recording has fully stopped (loses the near-real-time property, but guarantees a valid file).

"ATEM Overseer / Flock isn't showing a theatre's live preview"

- Confirm the ATEM/BirdDog Play in that theatre is actually configured to originate its second monitoring/preview stream, not just its primary program feed — this is a genuinely unconfirmed capability, not assumed working out of the box, see `docs/open-questions.md` #11.
- Check the stream is actually reaching the container's IP (`192.168.1.21` for Overseer, `192.168.1.24` for Flock — see `docs/ip-address-map.md`) over the tailnet, same routing path as every other theatre-to-mothership stream in this design.
- If every other theatre works but one doesn't, suspect that theatre's own A-1300 hitting its combined-load ceiling rather than anything Overseer/Flock-side — see the per-router finding in `docs/bandwidth-analysis.md`.

"GL-iNet static DHCP leases aren't applying"

- Every `REPLACE_WITH_MAC_nn` placeholder in `config/gl-inet/` must be swapped for the device's real MAC first — the generated configs ship with placeholders deliberately, not real defaults.